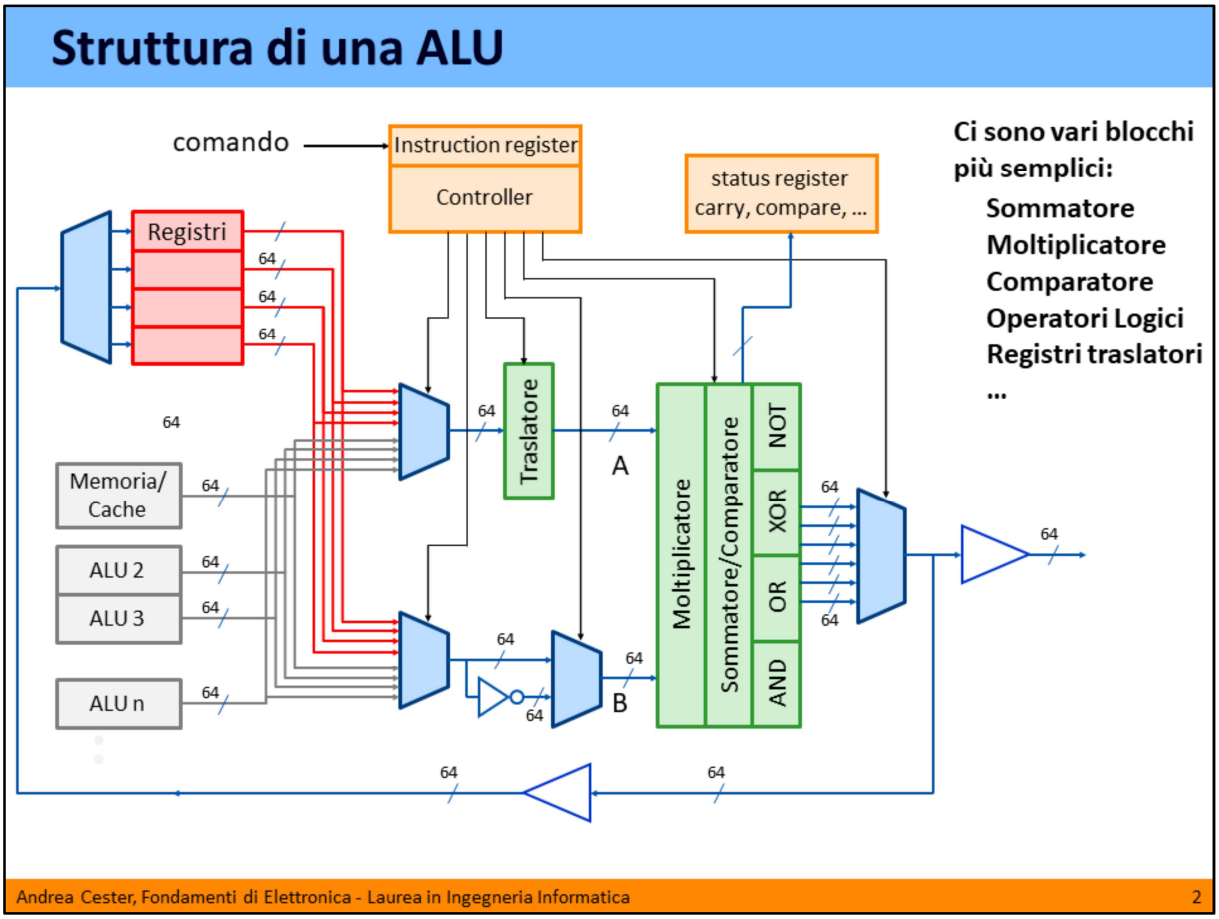
# Blocchi logico-aritmetici

*Rabaey, Circuiti Integrati Digitali 2°Ed: Cap. 11 p.597-642*

- Traslazione e rotazione
- Sommatore
- Sottrattore
- Comparatore
- Moltiplicatore
- Floating Point Unit




**Struttura di una ALU**

Una ALU è costituita da vari blocchi concatenati in modo programmabile per svolgere diverse funzioni logiche e aritmetiche.

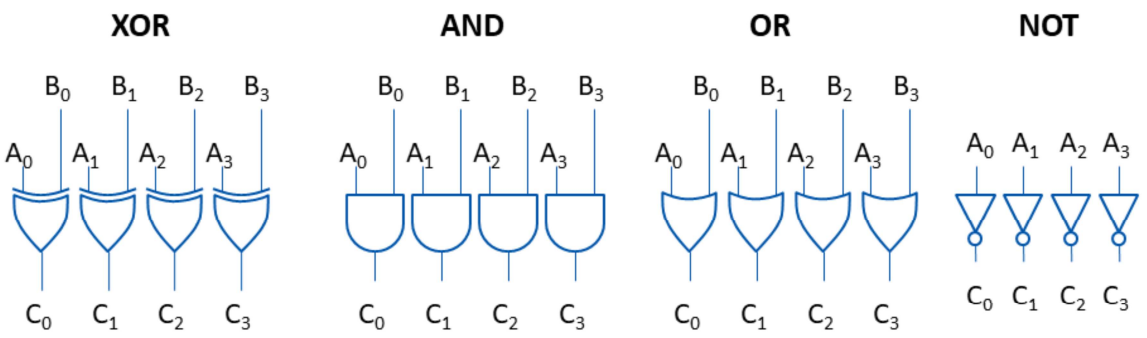
Il cuore dell'ALU (blocco verde) contiene diversi blocchi, quali: roto-traslatori, operatori logici (AND, OR, XOR, NOT…), operatori aritmetici (somma, sottrazione, moltiplicazione, comparazione…). Ciascuno di questi blocchi riceve in ingresso due operandi (vettori binari) che rappresentano numeri interi o in virgola mobile.
Gli operandi sono normalmente contenuti in registri, dalla memoria (RAM) o dall'uscita di altre ALU. Il risultato in uscita dalla ALU viene a sua volta memorizzato in uno dei registri interni.
Un sistema di multiplexer seleziona l'origine dei dati di ingresso e dove memorizzare il risultato di uscita.
Infine, i segnali di controllo dei multiplexer e dei vari blocchi logici sono forniti da una circuiteria di apposita che si occupa di decodificare l'istruzione e gestirne l'esecuzione. Questa circuiteria è fatta da porte logiche che hanno come ingresso la codifica binaria dell'istruzione macchina e producono in uscita i valori opportuni dei segnali di controllo.

Nel seguito vediamo come realizzare i blocchi logico-aritmetici che costituiscono il cuore della ALU. Successivamente analizzeremo come implementare i registri e la memoria esterna.

# OPERATORI LOGICI

## Operatori Logici

- Semplice funzione logica tra due vettori di ingresso
- Operazione eseguita elemento per elemento
- Array di porte logiche elementari

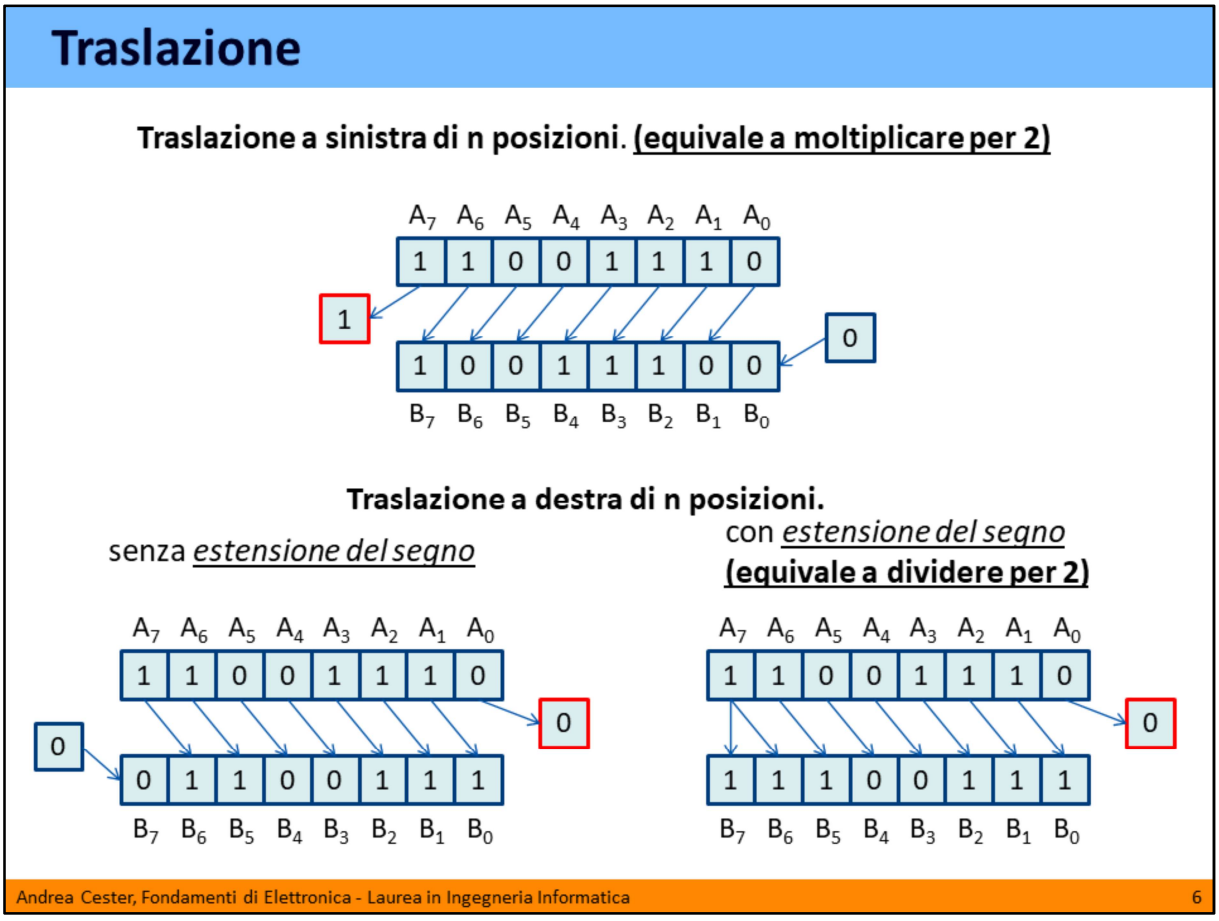



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 4

**Operatori logici**

Gli operatori logici di una ALU eseguono semplicemente una funzione logica bit a bit tra i due operandi.
L'operatore logico è semplicemente un vettore di N porte logiche elementari (AND, OR, XOR, NOT, ecc), dove N è il numero di bit. Abbiamo già visto alcune delle possibili strategie di implementazione di tali porte logiche.

# TRASLAZIONE E ROTAZIONE




**Traslazione**

Un blocco traslatore esegue uno spostamento dei bit di un numero intero di una certa quantità verso destra o verso sinistra.

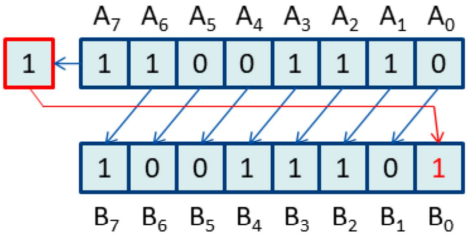
*Traslazione a sinistra di n posizioni*. Tutti i bit sono spostati verso sinistra (verso il MSB). Gli n bit più significativi escono e sono persi. Dalla parte del LSB entrano n zeri. Questa operazione è equivalente alla moltiplicazione per 2

*Traslazione a destra di n posizioni.* I bit sono spostati verso destra (verso il LSB). Gli n bit meno significativi sono persi. Dalla parte del LSB possono entrare:
- *n zeri*.
- *n bit uguali al MSB.* Se il vettore di bit rappresenta un numero intero in complemento a due il MSB non è altro che il bit del segno (0=numero positivo o nullo, 1 = numero negativo). Per questo motivo la traslazione con il riporto del MSB è detta anche traslazione con *estensione del segno*. Da un punto di vista matematico equivale alla divisione per 2.

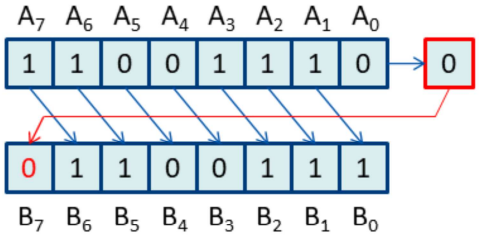
## Rotazione

- Un blocco di rotazione esegue uno spostamento dei bit di un numero intero di una certa quantità verso destra o verso sinistra. I bit uscenti vengono riportati dal lato opposto

**Rotazione a sinistra di *n* posizioni.**




**Rotazione a destra di *n* posizioni.**



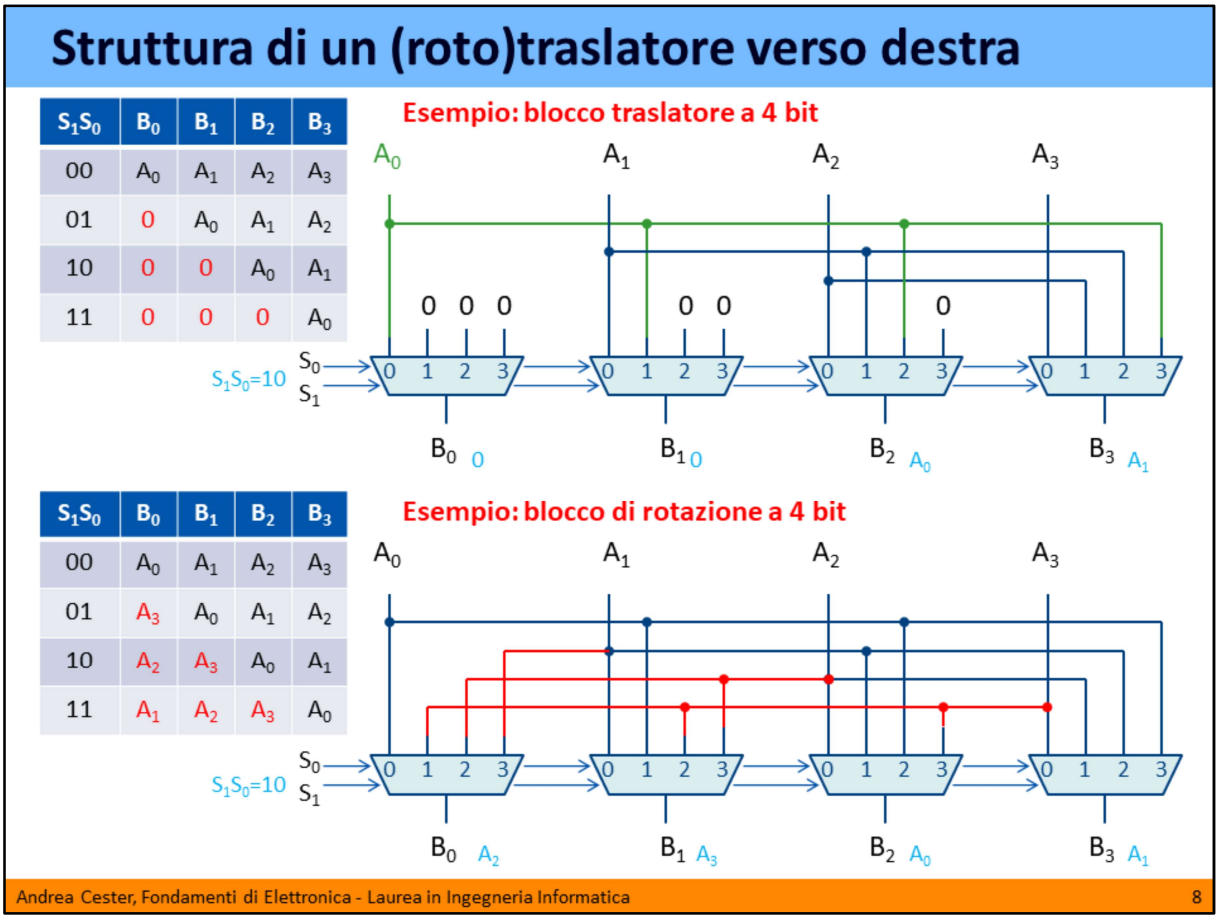

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 7

**Rotazione**

Un blocco di rotazione esegue uno spostamento dei bit di un numero intero di una certa quantità verso destra o verso sinistra. I bit uscenti vengono riportati dal lato opposto

*Rotazione a sinistra di n posizioni*.
tutti i bit sono spostati verso sinistra (verso il MSB).
Gli n bit più significativi escono a sinistra e rientrano a destra (dalla parte del LSB).

*Rotazione a destra di n posizioni.*
I bit sono spostati verso destra (verso il LSB).
Gli n bit meno significativi escono  a destra e rientrano a sinistra (dalla parte del LSB)




| $S_1S_0$ | $B_0$ | $B_1$ | $B_2$ | $B_3$ |
|---|---|---|---|---|
| 00 | $A_0$ | $A_1$ | $A_2$ | $A_3$ |
| 01 | 0 | $A_0$ | $A_1$ | $A_2$ |
| 10 | 0 | 0 | $A_0$ | $A_1$ |
| 11 | 0 | 0 | 0 | $A_0$ |

| $S_1S_0$ | $B_0$ | $B_1$ | $B_2$ | $B_3$ |
|---|---|---|---|---|
| 00 | $A_0$ | $A_1$ | $A_2$ | $A_3$ |
| 01 | $A_3$ | $A_0$ | $A_1$ | $A_2$ |
| 10 | $A_2$ | $A_3$ | $A_0$ | $A_1$ |
| 11 | $A_1$ | $A_2$ | $A_3$ | $A_0$ |

**Architettura di un traslatore/roto-traslatore**

I due schemi mostrano l'architettura di un traslatore (sopra) e un roto-traslatore (sotto). In entrambi si esegue la traslazione/rotazione a sinistra (cioè dal LSB al MSB) a N=4 bit.
Ciascun bit del vettore B è generato da un multiplexer a N ingressi. All'ingresso k-esimo del MUX è connesso l'ingresso k-m del vettore A (dove m è il numero di bit di cui traslare).
Nel traslatore, se m>k viene inserito uno 0.
Nel roto-traslatore, se m>k viene inserito l'elemento N+k-m. in modo da eseguire la rotazione.
Ad esempio se vogliamo traslare/ruotare di 2 bit (m=2), al bit $B_3$ (k=3) viene collegato l'ingresso k-m = 1 ($A_1$) in entrambi i casi.
Se invece consideriamo il bit $B_1$ (k = 1) abbiamo k-m = -1. In tal caso, in un traslatore l'ingresso 2 del MUX è collegato a 0. Invece nel roto-traslatore è collegato l'ingresso N+k-m = 4 + 1 – 2 = 3.

L'operazione di rotazione e traslazione sono molto simili tra loro e si differenziano solo per le connessioni tra LSB e MSB.
Esistono due strutture possibili per i blocchi traslatori e rotatori.

- *Architettura a matrice*
- *Architettura ad albero o logaritmica*

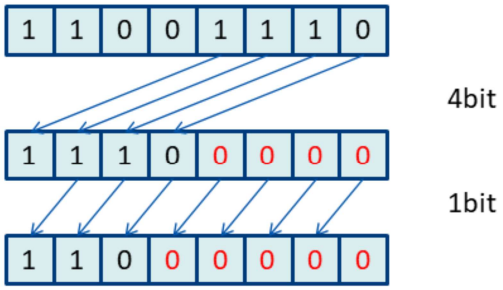
L'architettura a matrice consente la roto-traslazione in un unico stadio (più veloce) ma a un costo maggiore in termini di area.
L'architettura logaritmica consente un risparmio di area, ma per un vettore a N bit il ritardo è proporzionale a $\log_2 N$.
Nel seguito, come esempio, illustreremo solo l'architettura ad albero.

## Struttura logaritmica

- L'approccio alla base della struttura logaritmica è il fatto che due traslazioni consecutive di m e k bit corrispondono a una traslazione unica di n = m+k bit
- È possibile descrivere una traslazione di n bit come la somma di potenze di 2:

**Esempio: Numero di 8 bit**

**Traslazione di 5 bit = Traslazione di 4 bit + 1 bit**



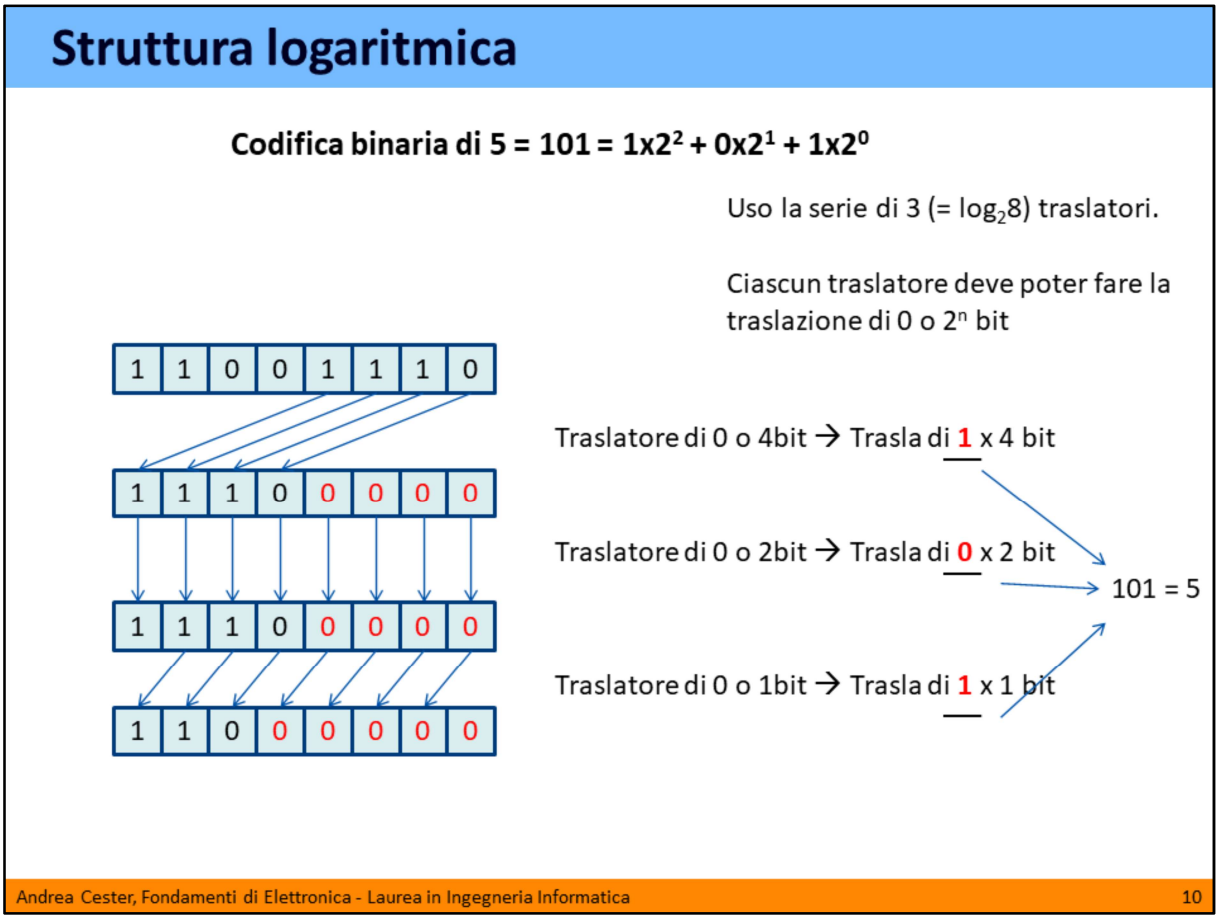

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 9

*Architettura logaritmica*

L'approccio della struttura logaritmica si basa su due considerazioni:

1. Due traslazioni/rotazioni consecutive di m bit e k bit, rispettivamente, corrispondono a un'unica traslazione/rotazione di m+k bit.
2. Ciascun numero intero può essere scritto come somma di potenze di 2.
3. Una qualunque traslazione di k bit può essere eseguita sommando un numero adeguato di traslazioni di un numero di bit pari a una potenza di due la cui somma sia k.
4. La traslazione di 0 bit corrisponde alla funzione identità

Ad esempio se si vuole traslare di 5 bit è possibile eseguire 2 traslazioni di 4 e 1 bit.




Il concetto del roto-traslatore logaritmico a N = $2^m$ bit sta nel mettere in cascata m stadi di traslatori in cui lo stadio k-esimo può essere programmato per eseguire la traslazione di 0 bit o di $2^k$ bit.
Prendiamo come esempio la traslazione di 5 bit eseguita in un numero a 8 = $2^3$bit mostrata in figura.
Il numero 5 può essere scritto come somma di tre potenze di due:

$$5 = 1 + 4 = 2^0 \cdot 1 + 2^1 \cdot 0 + 2^2 \cdot 1$$

Dal punto di vista logico la traslazione di 5 bit può essere svolta in tre stadi:

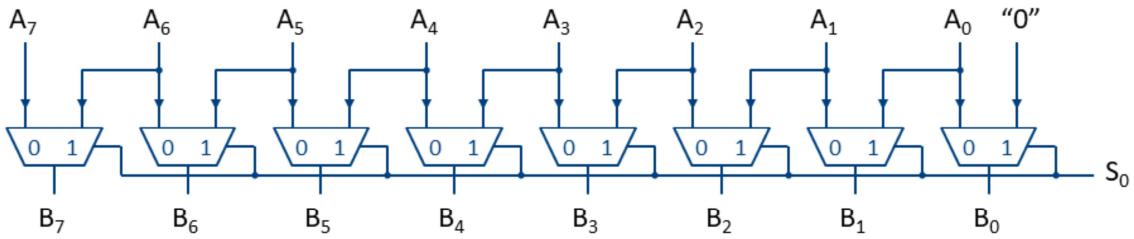
Primo stadio: traslazione di 0 o 4 bit. Programmato per la traslazione di **4 bit**

Secondo stadio: traslazione di 0 o 2 bit. Programmato per la traslazione di **0 bit**

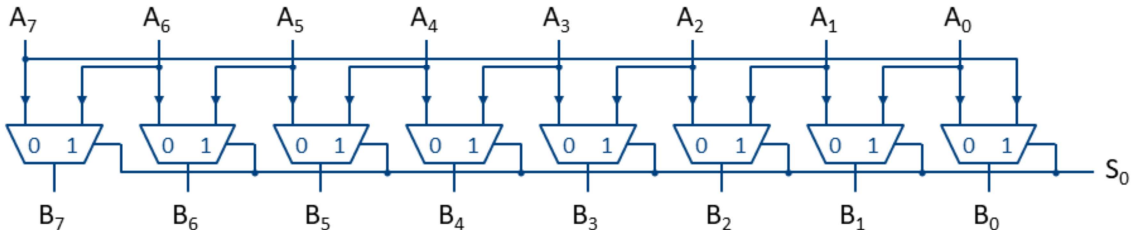
Terzo stadio: traslazione di 0 o 1 bit. Programmato per la traslazione di **1 bit**

# Traslazione e rotazione

**Traslazione a sinistra di 1bit (Moltiplicazione x 2)**

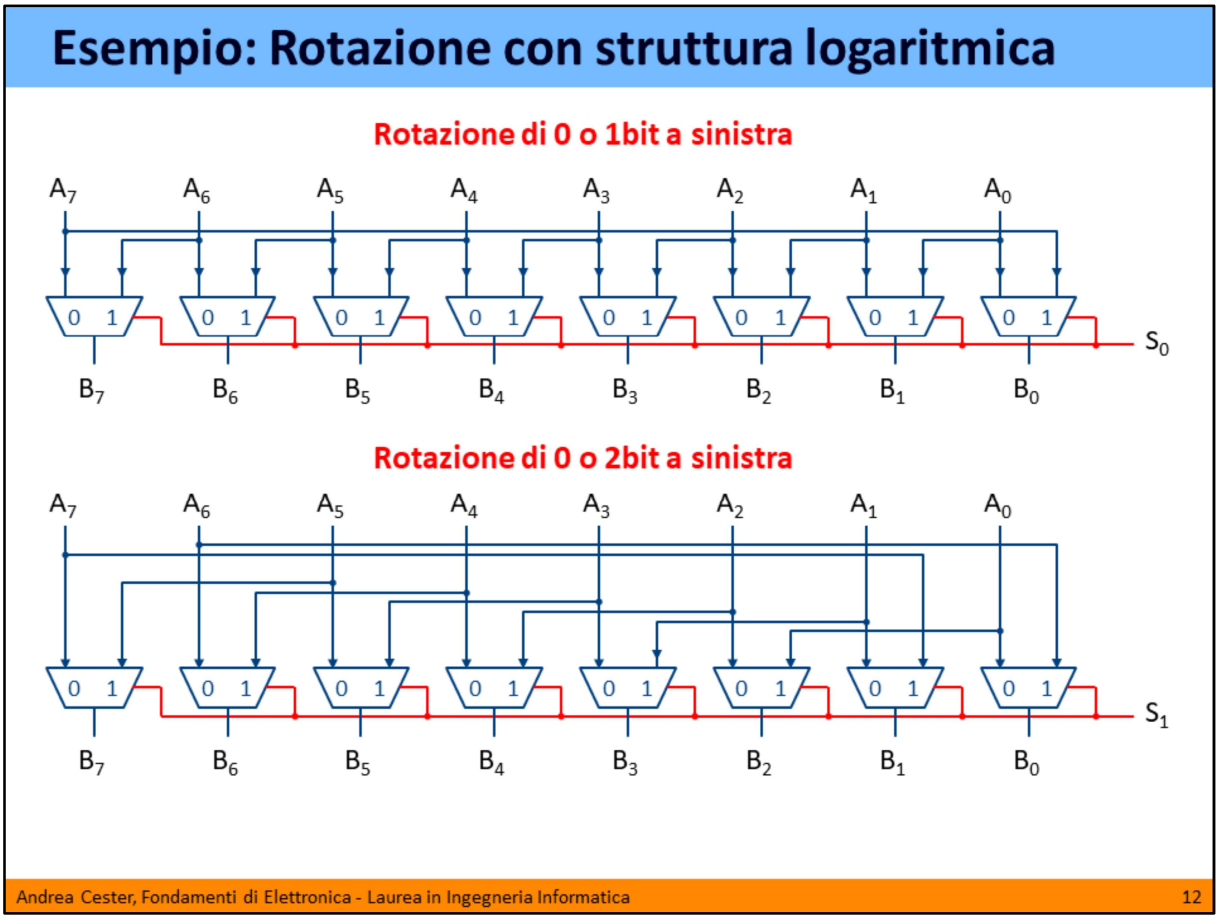



**Rotazione a sinistra di 1bit**



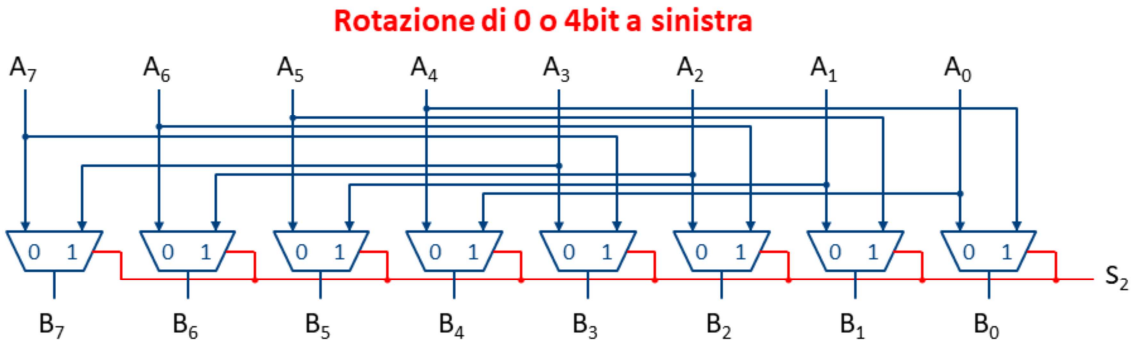

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 11

Nei due schematici vediamo lo stadio di traslazione e rotazione di 0 o 1 bit.
In entrambi i casi, se S = 0 abbiamo B = A e si ha la traslazione di 0 bit. Se S = 1 si si ha la traslazione o la rotazione verso sinistra di 1 bit. Nel caso della traslazione $B_0$ viene posto a 0. Nel caso della rotazione viene posto uguale a $A_7$.



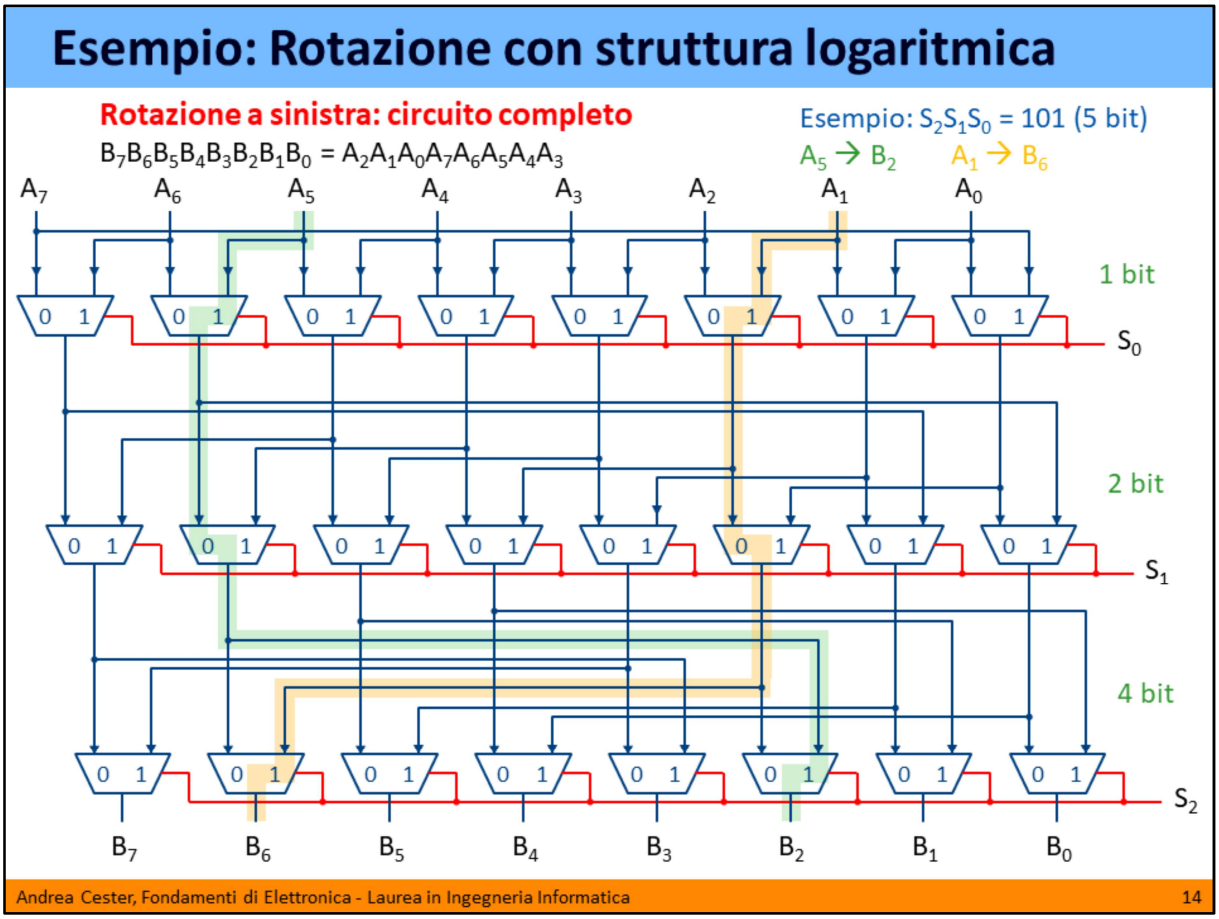

Prendiamo come esempio il caso della rotazione. In figura vediamo i due blocchi di rotazione 0 o 1 bit (in alto) e 0 o 2 bit (in basso).
Il tipo di circuito è lo stesso. L'unica differenza sta nei collegamenti: il MUX k-esimo riceve all'ingresso 0 $A_k$ e all'ingresso 1 $A_{k-2}$ invece che $A_{k-1}$.

# Esempio: Rotazione con struttura logaritmica



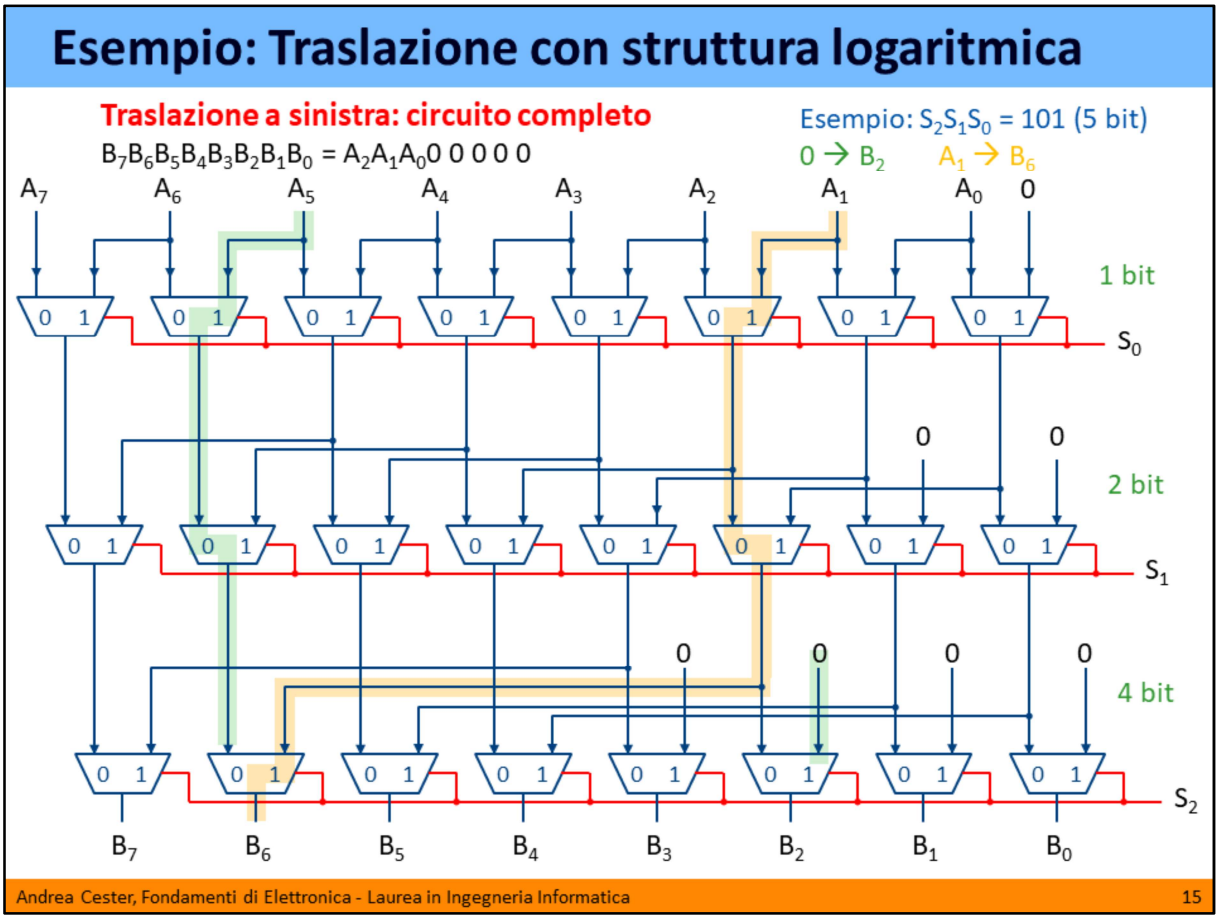



Analogamente si procede per la traslazione 0 o 4 bit.




Mettendo insieme i circuiti otteniamo la rotazione verso sinistra di un numero arbitrario di bit da 0 a 7.

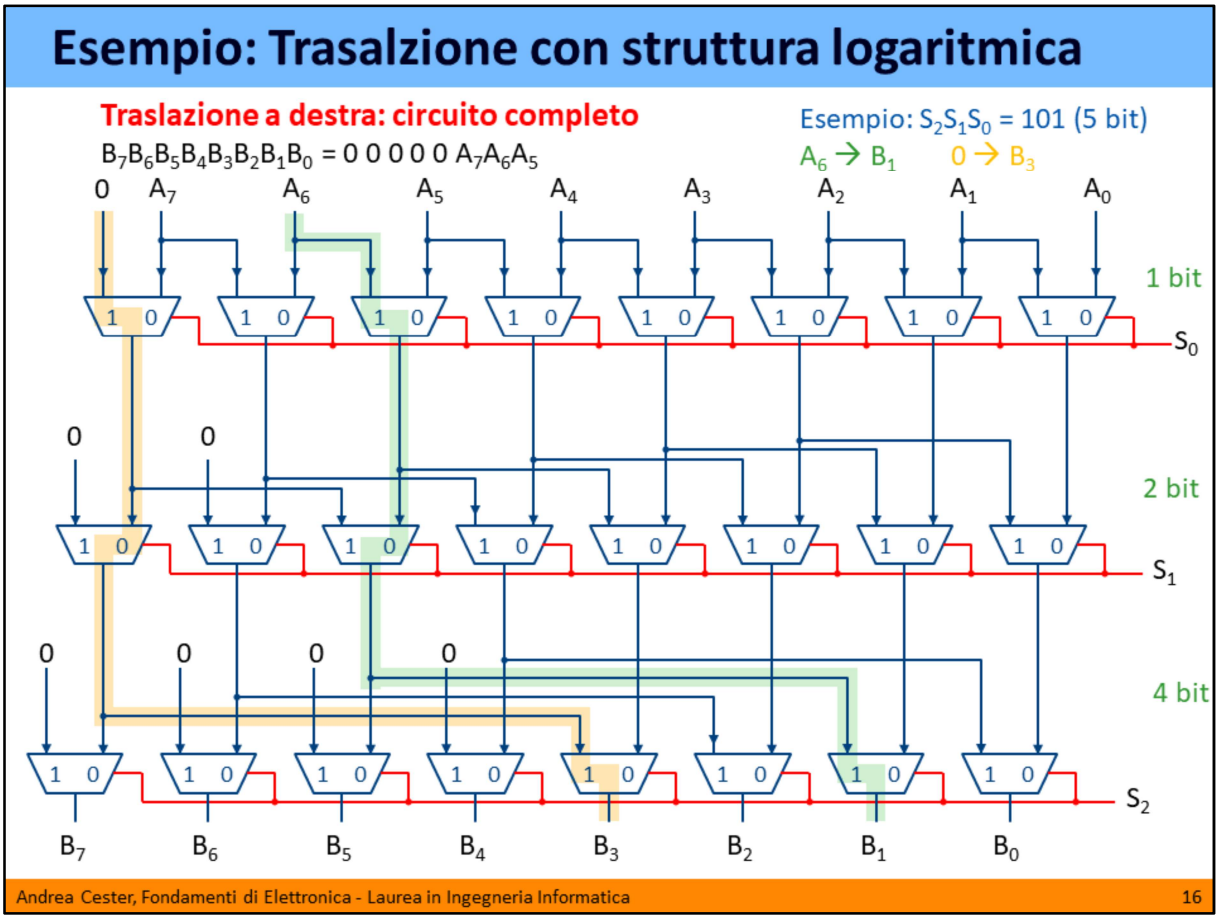
Nell'esempio è mostrata la rotazione a sinistra di 5 bit e sono evidenziati i percorsi dei bit $A_1$ (che viene traslato in $B_6$) e $A_5$ che è traslato verso l'MSB, esce, rientra dalla parte dell'LSB e arriva fino a $B_2$.




Il circuito in figura è la variante che esegue la traslazione a sinistra di un numero arbitrario di bit da 0 a 7. Si osserva facilmente che:

- Primo stadio: l'ingresso 1 del primo MUX a destra (LSB) è posto a 0, invece cha essere collegato a $A_7$.
- Secondo stadio: Gli ingressi 1 dei primi due MUX dalla parte dell'LBS sono posti a 0.
- Terzo stadio: Gli ingressi 1 dei primi quattro MUX dalla parte dell'LBS sono posti a 0.

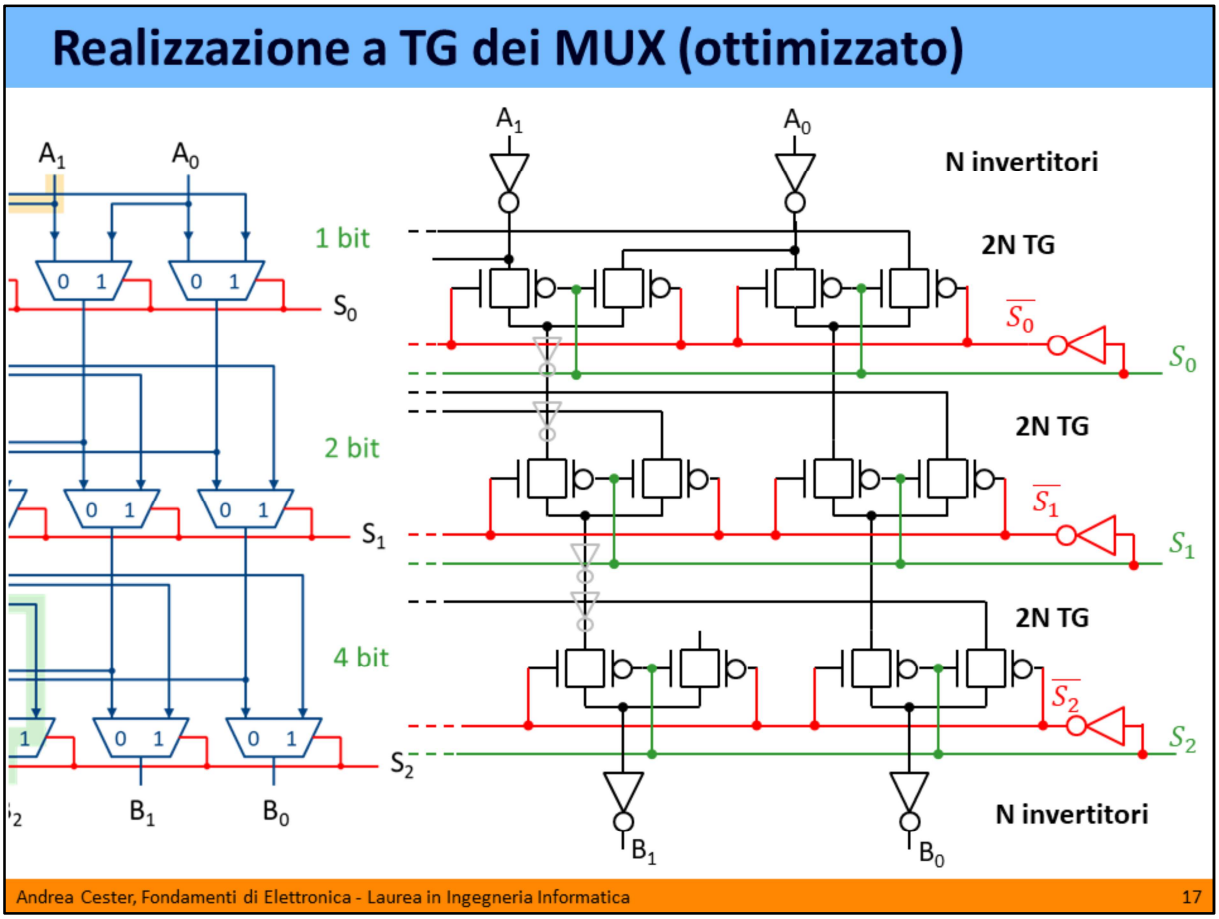
Nell'esempio è mostrata la traslazione a sinistra di 5 bit e sono evidenziati i percorsi dei bit $A_1$ (che viene traslato in $B_6$ come nel caso della rotazione) e $A_5$ che è traslato verso l'MSB, esce, ma non rientra. Contrariamente alla rotazione $B_2$ viene collegato a 0.




Il circuito in figura è la variante che esegue la traslazione destra di un numero arbitrario di bit da 0 a 7 senza riporto del segno. Si osserva facilmente che la struttura è simmetrica rispetto all'esempio precedente:

- Primo stadio: l'ingresso 1 del primo MUX a sinistra (MSB) è posto a 0.
- Secondo stadio: Gli ingressi 1 dei primi due MUX dalla parte dell'MBS sono posti a 0.
- Terzo stadio: Gli ingressi 1 dei primi quattro MUX dalla parte dell'MBS sono posti a 0.

Nell'esempio è mostrata la traslazione a destra di 5 bit e sono evidenziati i percorsi dei bit $A_6$ (traslato in $B_1$) e $B_3$ che riceve 0 (estensione di $A_8$ senza segno).




A livello di transistor, ciascun MUX può essere realizzato mediante tre invertitori e due porte di trasmissione (come illustrato a destra).

Con questo tipo di implementazione ritroviamo una coppia di invertitori in cascata tra uno stadio e il successivo. Per ottimizzare il numero di MOS e i tempi di ritardo è possibile eliminare queste coppie di invertitori. Ricordiamo che in un MUX 2:1 gli invertitori in ingresso e in uscita sono stati inseriti per rendere il MUX rigenerativo e unidirezionale.

Per rendere il traslatore unidirezionale è possibile eliminare tutti gli invertitori ad eccezione di quelli agli ingressi primari ($A_0$, $A_1$, …) e alle uscite ($B_0$, $B_1$,…).

In questo modo si ottiene una serie di $\log_2 N$ porte di trasmissione con un tempo di ritardo proporzionale a $(\log_2 N)^2$.

Se N è troppo grande e il numero di porte di trasmissione in serie risultasse eccessivo è possibile inserire dei buffer rigeneratori ogni 3-4 stadi per ridurre il temo di ritardo, come abbiamo visto precedentemente.

## Stima dei MOS richiesti per un traslatore

| Traslatore | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Numero di stadi | 3 | 4 | 5 | 6 | log(N) |
| Numero di TG | 48 | 128 | 320 | 768 | 2N·logN |
| N° di NOT | 16 | 32 | 64 | 128 | 2N |
| N° di NOT (segnali S) | 3 | 4 | 5 | 6 | log(N) |
| **Tot. MOS (2xTG + 2xNOT)** | **134** | **328** | **778** | **1804** | (2N+1)·logN+2N |

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 18

A titolo di esempio la tabella mostra una stima del numero di MOS necessari per realizzare un MUX ad albero a 8, 16, 32 e 64 bit.

In questo esempio si assume di realizzare un albero di porte di trasmissione, senza invertitori tra l'uscita di uno stadio e l'ingresso del successivo. Gli invertitori sono collocati solo agli ingressi e all'uscita del MUX.

Detto N è il numero di ingressi dati abbiamo $\log_2 N$ stadi.
I componenti necessari sono:

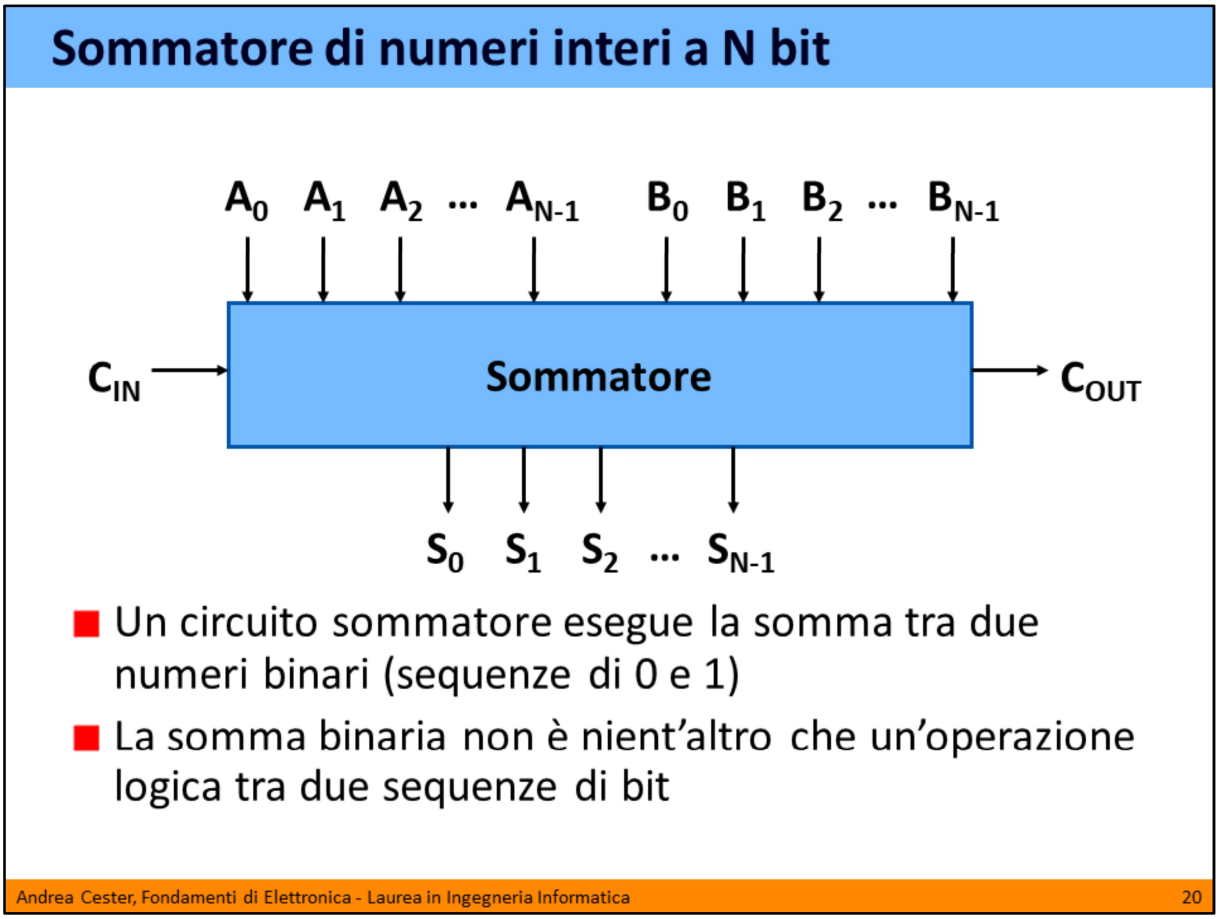
- Invertitori di ingresso e di uscita: uno per ciascun segnale di ingresso e uno per ciascuna uscita. In totale sono 2N invertitori.
- Porte di trasmissione: Abbiamo N MUX per stadio. Ciascun MUX richiede 2 TG. In totale $2N \cdot \log_2 N$ TG
- Invertitori per i segnali di selezione. Ciascun TG ha bisogno del segnale di gate diretto e negato. Poiché tutti i TG di uno stadio sono pilotati dallo stesso segnale di controllo, possono condividere lo stesso invertitore (uno per stadio). In totale abbiamo $\log_2 N$ invertitori. A tale scopo vale la pena osservare che il carico di ciascun invertitore è proporzionale a N. Se N è molto grande potrebbe essere necessario un buffer per gestire il carico. Per questo semplice esempio, trascuriamo l'effetto del carico.

In totale sono $2N+\log_2 N$ invertitori e $N \cdot \log_2 N$ TG. Tenendo conto che ciascun TG e ciascun invertitore richiede due MOS (uno n e uno p), in totale il MUX richiede un numero di MOS:

$$4N \cdot \log_2 N + 4N$$

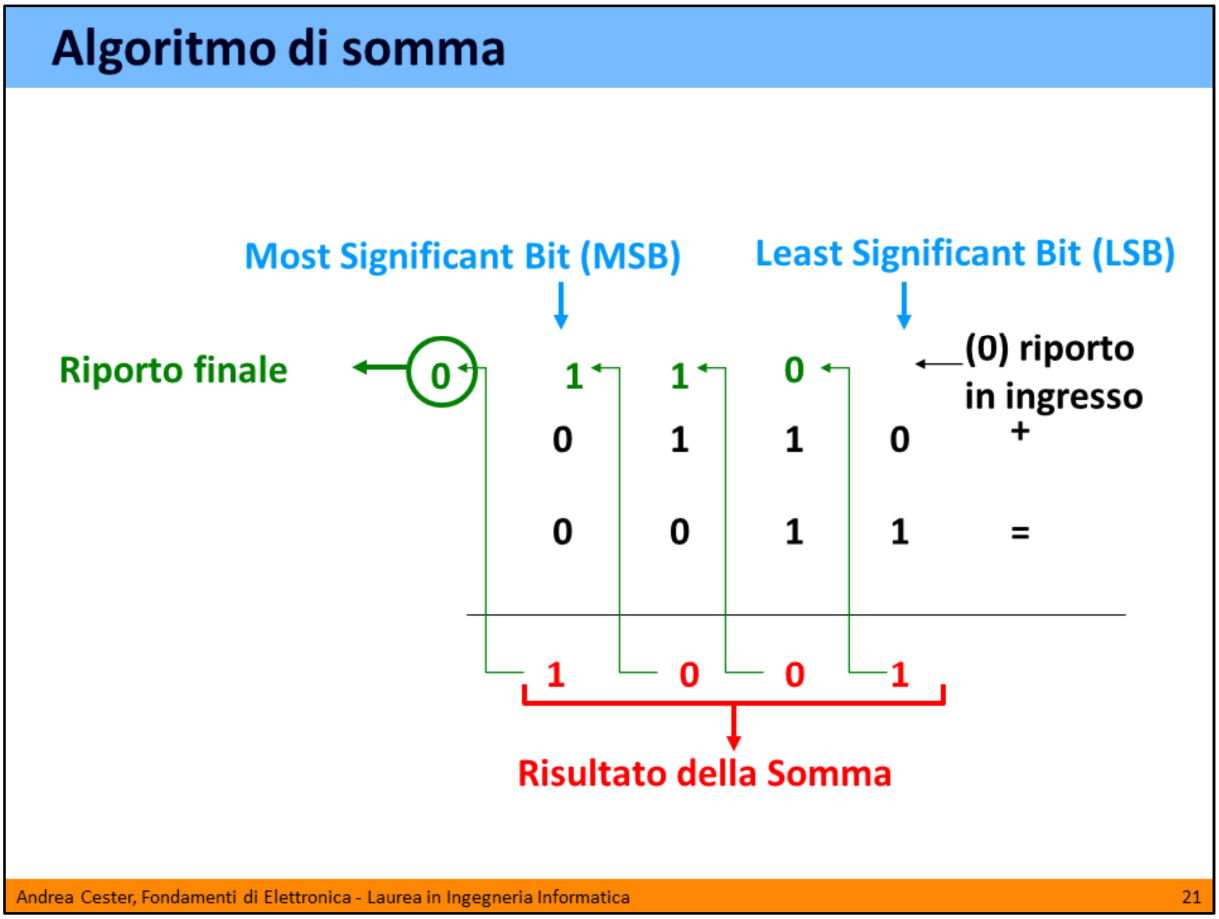
I valori richiesti per un traslatore a 8, 16, 24 e 32 bit sono riportati nell'ultima riga e va da poco più di cento a quasi 2000 MOS.

# SOMMATORE




**Il sommatore**

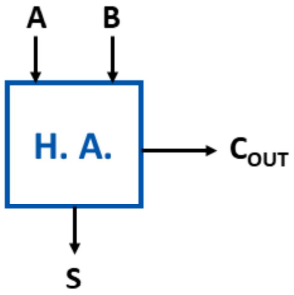
In elettronica digitale un sommatore a N bit è nient'altro che un circuito combinatorio con 2N+1 ingressi e N+1 uscite.
Gli ingressi sono due vettori di N bit A e B che rappresentano gli interi da sommare, e l'eventuale riporto in ingresso $C_{IN}$.
Le uscite sono il vettore S di N bit corrispondente alla somma e il riporto di uscita $C_{OUT}$, che può essere preso come estensione dei bit di somma.

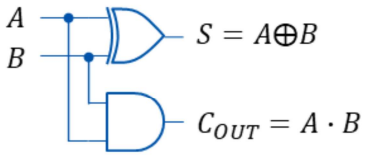



L'algoritmo base della somma procede dall'LSB fino al MSB. Il calcolo del bit k-esimo consiste nella somma dei due bit dei numeri in ingresso ($A_k$ e $B_k$) e il riporto della somma dei bit precedenti ($A_{k-1}$ e $B_{k-1}$). La somma del bit k-esimo genera anche un riporto che deve essere sommato alla somma dei bit $A_{k+1}$ e $B_{k+1}$.

## Sommatore a 1 bit (Half Adder)

- Un half-adder somma due bit A e B
- Ha due uscite:
  - la somma (S)
  - il riporto dell'operazione di somma ($C_{OUT}$)



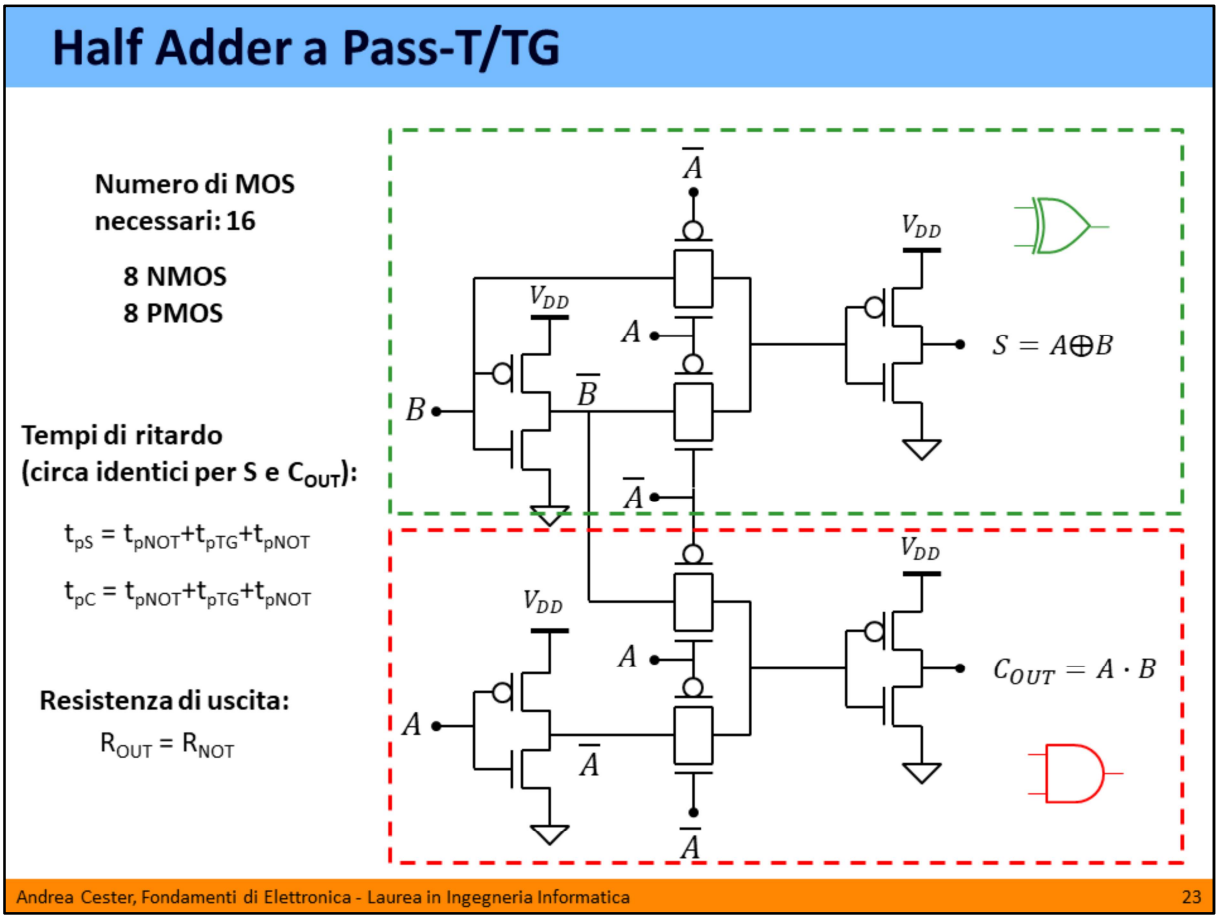




| A | B | S | $C_{OUT}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 22

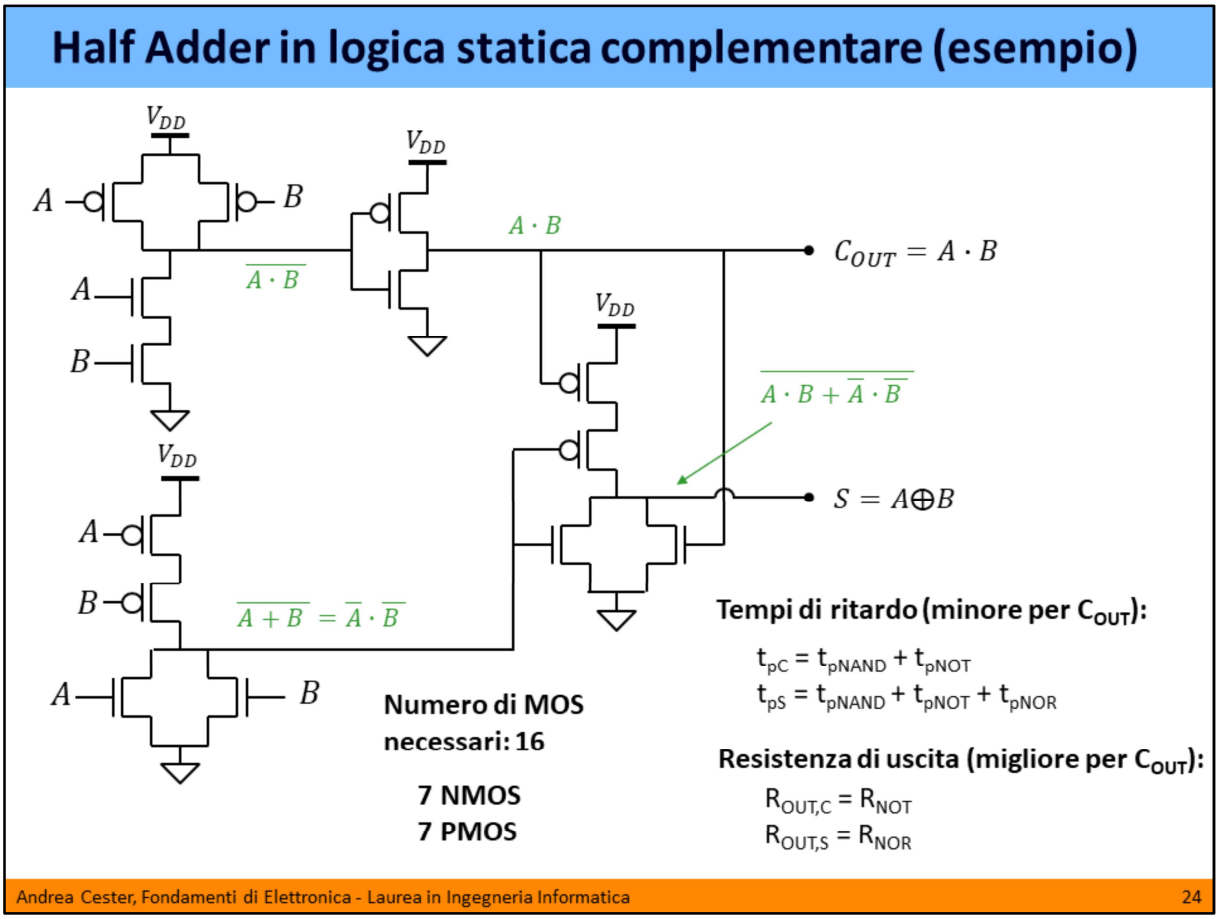
**Somma di due bit: half-adder (HA)**

Un Half-Adder (HF) è un circuito con due ingressi e due uscite. Esso riceve in ingresso due segnali A e B (interpretati come numeri binari a una cifra) e restituisce in uscita una coppia di segnali corrispondenti alla somma S = A + B (intesi come cifre binarie) e al riporto della comma $C_{OUT}$.
In figura vediamo la tabella di verità corrispondente alla somma e al riporto.
Si verifica facilmente che la somma è la funzione XOR e il riporto la funzione AND.
Le due funzioni possono essere realizzate mediante porte di trasmissione e invertitori come mostrato precedentemente.




*Esempio: implementazione dell'HA con TG.*

Ad esempio, se implementiamo le die porte con architettura a MUX tramite due porte di trasmissione, otteniamo gli schemi riportati in figura. Ciascun MUX impiegherebbe 10 MOS (5n + 5p). Tuttavia i due invertitori (quelli che negano A e B) possono essere condivisi tra le due porte. Il numero totale di MOS è 16.
I tempi di ritardo dei due circuiti (a parità di carico esterno) sono molto simili, avendo la stessa topologia.




*Esempio: implementazione dell'HA in logica statica complementare.*

In logica statica complementare, possiamo realizzare la funzione AND che genera $C_{OUT}$ con una porta NAND seguita da un invertitore. Invece la porta XOR può essere realizzata riscrivendo la funzione XOR in questo modo:

$$A \oplus B = \overline{\overline{A \oplus B}} = \overline{A \cdot B + \overline{A} \cdot \overline{B}} = \overline{A \cdot B + \overline{A + B}}$$

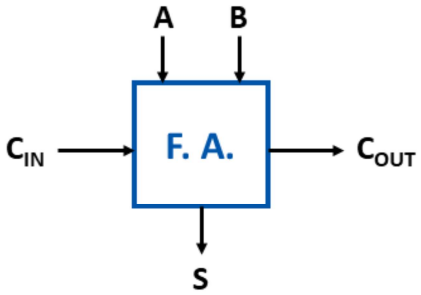
Notiamo che: $A \cdot B = C_{OUT}$ è l'uscita della porta AND. Ponendo $X = \overline{A + B}$, realizziamo S come NOR do X e $C_{OUT}$.

Sono necessarie quindi 2 porte NOR, 1 porta NAND e un invertitore per il calcolo di S e $C_{OUT}$. In totale impieghiamo 14 MOS.

I tempi di ritardo del circuito sono diversi per S e $C_{OUT}$. Tra i due, il minore è quello relativo a $C_{OUT}$.

## Sommatore a 1 bit con riporto (Full Adder)

- Il full-adder somma due bit A e B e il riporto $C_{IN}$
- Ha due uscite:
  - la somma (S)
  - il riporto dell'operazione di somma ($C_{OUT}$)




| A | B | $C_{IN}$ | S | $C_{OUT}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

$$S = A \oplus B \oplus C_{IN}$$

$$C_{OUT} = A \cdot B + (A \oplus B) \cdot C_{IN}$$

oppure

$$C_{OUT} = A \cdot \overline{(A \oplus B)} + (A \oplus B) \cdot C_{IN}$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 25

**Somma di due bit con riporto**

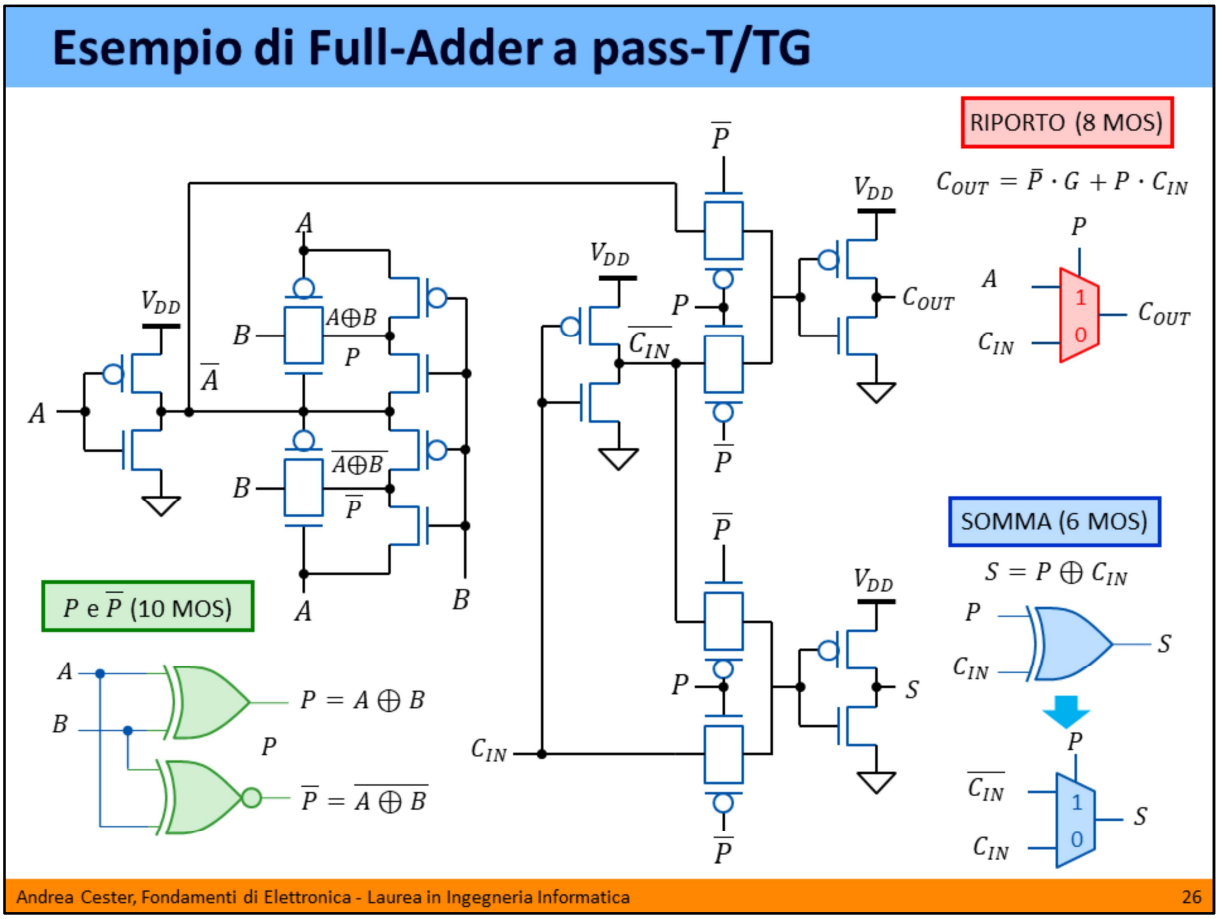
Un full-adder (FA) è un circuito a tre ingressi e due uscite. I segnali di ingresso rappresentano tre cifre binarie: A e B sono due numeri interi a 1 bit; $C_{IN}$ è il bit di riporto (quello dell'eventuale somma precedente).
In figura vediamo la tabella di verità.

La somma realizza la funzione XOR di tre ingressi:

$$S = A \oplus B \oplus C_{IN}$$

Il riporto di uscita $C_{OUT}$ può essere ricavato dalla mappa di Karnaugh:

$$C_{OUT} = A \cdot B + A \cdot \bar{B} \cdot C_{IN} + \bar{A} \cdot B \cdot C_{IN} = A \cdot B + (A \cdot \bar{B} + \bar{A} \cdot B) \cdot C_{IN}$$




*Esempio : realizzazione del FA a pass-transistor/porte di trasmissione.*

Studiando la l'implementazione di funzioni logiche a pass-T abbiamo già dimostrato che S e $C_{OUT}$ possono essere ricavati in funzione del segnale ausiliario:

$$P = A \oplus B$$

come segue:

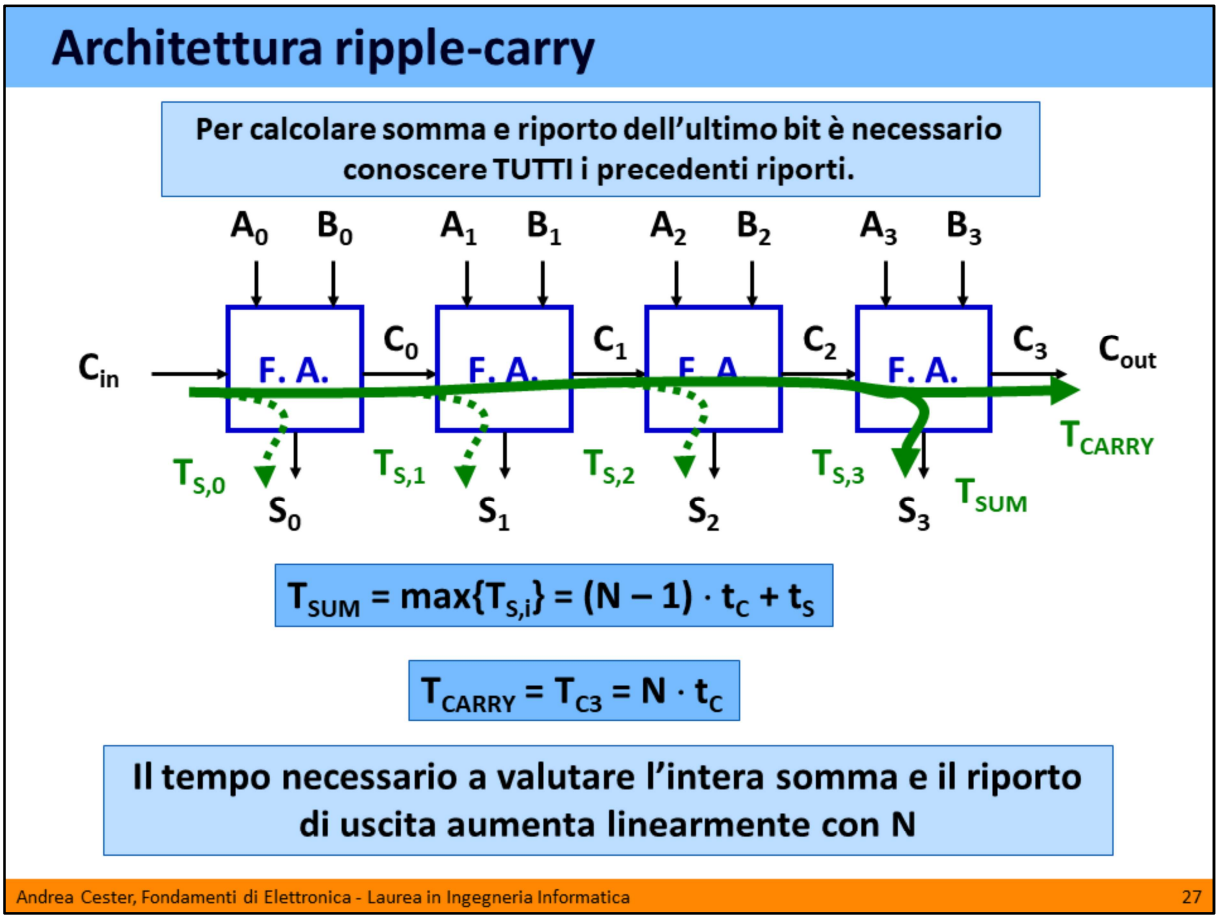
$$S = P \oplus C_{IN}$$
$$C_{OUT} = \bar{P} \cdot A + P \cdot C$$

Il segnale P e il suo negato possono essere realizzati mediante una coppia di porte XOR e XNOR che condividono lo stesso invertitore per generare A (non rappresentato in figura). Il numero totale di MOS è 4 + 4 + 2 = 10.

La somma è ottenuta con una porta XOR tra P e $C_{IN}$. In questo caso avendo P sia in forma diretta, sia negata non è necessario l'invertitore. Quindi sono sufficienti 2 porte di trasmissione e due invertitori (8 MOS in totale).

$C_{OUT}$ è realizzato con una coppia di porte di trasmissione e due invertitori. In totale bastano 6 MOS perché quelli dell'invertitore pilotato da A sono già stati inclusi nel computo della porta XOR/XNOR (Parte 1).

L'intero full-adder richiede 24 MOS.
Tra le caratteristiche importanti di questa architettura ricordiamo che il tempo di ritardo dagli ingressi all'uscita S e $C_{OUT}$ sono uguali e che la resistenza di uscita è quella di un singolo invertitore.




**Architettura ripple carry**

La somma di due interi A e B rappresentati su *N* bit come numeri senza segno o in complemento a due può essere ottenuta nel modo più semplice mediante *N full adder* connessi in modo tale che l'unità *i*-esima riceva in ingresso i due bit corrispondenti all'*i*-esima cifra binaria degli addendi e fornisca in uscita il bit *i*-esimo della somma. Il *riporto* calcolato dall'*i*-esimo *full adder* viene fornito come riporto di ingresso per il *full-adder* (*i*+1)-esimo.

Per determinare le prestazioni in termini di velocità della struttura è necessario considerare il tempo massimo che essa impiega per generare correttamente tutti i bit della parola d'uscita.
Infatti la parola di uscita è valida quando tutti i bit che la rappresentano hanno raggiunto una configurazione stabile.
In generale per calcolare la somma dell'n-esimo bit è necessario conoscere il valore di:

- $A_n$ e $B_n$ (bit di ingresso)
- $C_{n-1}$ (riporto della somma dei bit precedenti)

Per conoscere il riporto $C_{n-1}$ è necessario conoscere:

- $A_{n-1}$ e $B_{n-1}$ (bit di ingresso)
- $C_{n-2}$ (riporto della somma dei bit precedenti)

Di conseguenza il percorso critico per la somma è quello che parte dal primo bit e arriva fino all'ultimo. Il tempo di calcolo della somma per N bit è:

$$T_{SUM} = (N-1) \cdot t_C + t_S$$

dove con $t_C$ e $t_S$ indichiamo il tempo di propagazione di riporto e somma per ciascun FA
Analogamente per il riporto di uscita abbiamo:

$$T_{CARRY} = N \cdot t_C$$

Per calcolare somma e riporto dell'ultimo bit è necessario conoscere TUTTI i precedenti riporti.
*Il tempo necessario a valutare l'intera somma e il riporto di uscita aumenta linearmente con N*

# SOTTRAZIONE

# Sottrazione binaria

**Sottrarre due numeri interi significa sommare il primo con il negato del secondo.**

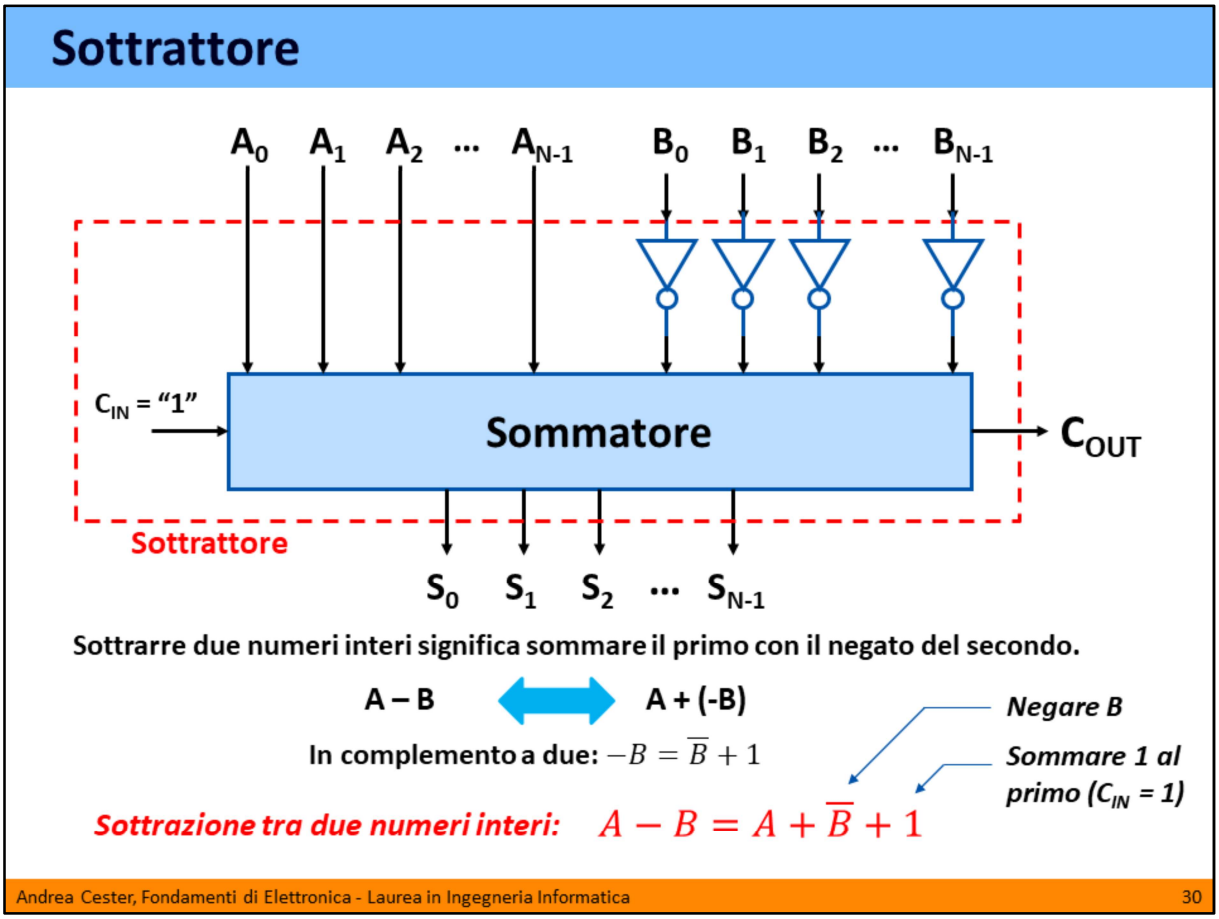
**A – B** **A + (-B)**

In complemento a due:

$$-B = \overline{B} + 1$$

***Sottrazione tra due numeri interi:***

$$A - B = A + \overline{B} + 1$$




**Sottrazione**

L'operazione di sottrazione può essere facilmente ricondotta a una somma:

$$A - B = A + (-B)$$

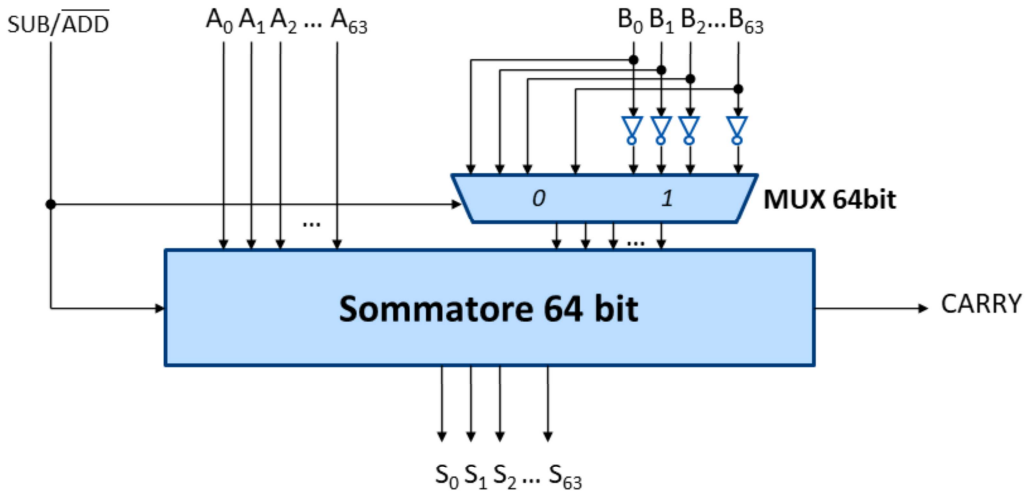
Sappiamo che la negazione di un numero in complementi a due equivale a complementare tutti i bit e sommare 1.

Quindi la sottrazione A-B equivale alla seguente somma:

$$A - B = A + (-B) = A + (\bar{B} + 1) = A + \bar{B} + 1$$

A livello circuitale è possibile usare un sommatore in cui i bit di B sono negati e in cui il riporto di ingresso vale 1.

## Esempio: Sommatore/Sottrattore a 64 bit




SEL = 0 → $C_{IN}$ = 0, in uscita alle XOR ci sono i segnali $B_n$ (SOMMA)

SEL = 1 → $C_{IN}$ = 1, in uscita alle XOR ci sono i segnali $B_n$ negati (SOTTRAZIONE)

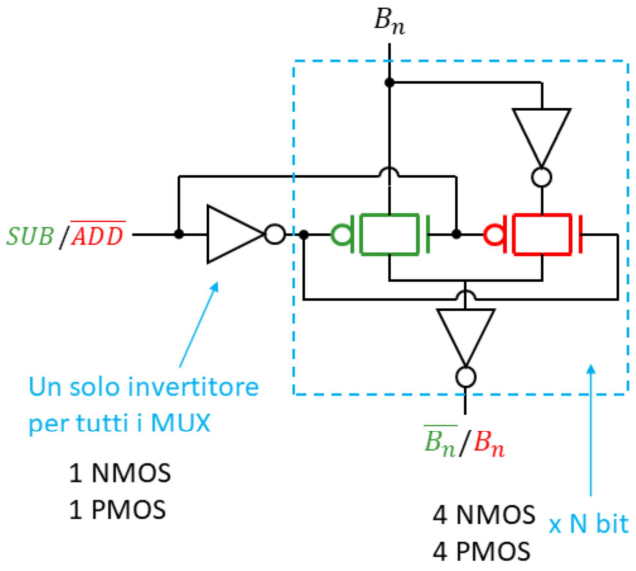


In modo molto semplice un sommatore può essere trasformato in un circuito sommatore/sottrattore come mostrato in figura.
In figura il MUX simboleggia un vettore di N multiplexer a 2 ingressi. Ciascun MUX è pilotato dallo stesso segnale di selezione SUB/ADD. Lo stesso segnale di selezione viene usatocome riporto di ingresso.

Se SUB/ADD = 0, abbiamo $C_{IN}$ = 0 e ciascun MUX porta in uscita B in forma diretta. Il sommatore esegue l'operazione S = A+B.

Se SUB/ADD = 1, abbiamo $C_{IN}$ = 1 e ciascuna MUX porta in uscita B in forma negata. Il sommatore esegue la somma di A, B negato e 1, che equivale all'operazione A – B.

## MUX per generare selezionare B diretto o negato




- Equivale a una porta XOR
- N MUX per N bit
- 1 NOT per generare il segnale negato di controllo per i MUX
- Totale MOSFET:

  8N + 2
  - 8N per i MUX
  - 2 per la NOT

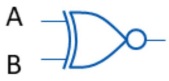


Ciascun elemento della batteria di multiplexer esegue, di fatto la funzione XOR tra B e SUB/ADD. Un esempio di implementazione è mostrato in figura. Se è attivo il canale di destra (rosso) il bit $B_n$ passa diretto. Se è attivo il canale di sinistra (verde), $B_n$ passa negato. In questo caso è stato eliminato il doppio invertitore per ottimizzare l'occupazione di area.
Il numero di MOS necessario per la realizzazione del MUX è 8 per bit, quindi 8N (4 per i due invertitori e 4 per i due TG). Poiché il segnale di selezione è il medesimo per tutti gli N MUX, l'invertitore che nega il segnale SUB/ADD viene condiviso tra tutti i MUX.
Il numero totale di MOS da aggiungere a quelli del sommatore è 8N+2.

# COMPARATORE

## Comparatore di uguaglianza tra due bit

**L'uguaglianza tra due bit è realizzata da una semplice porta XNOR**




EQ = 1 se A=B=1 o X=B=0

| A | B | Uguaglianza (EQ) |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |



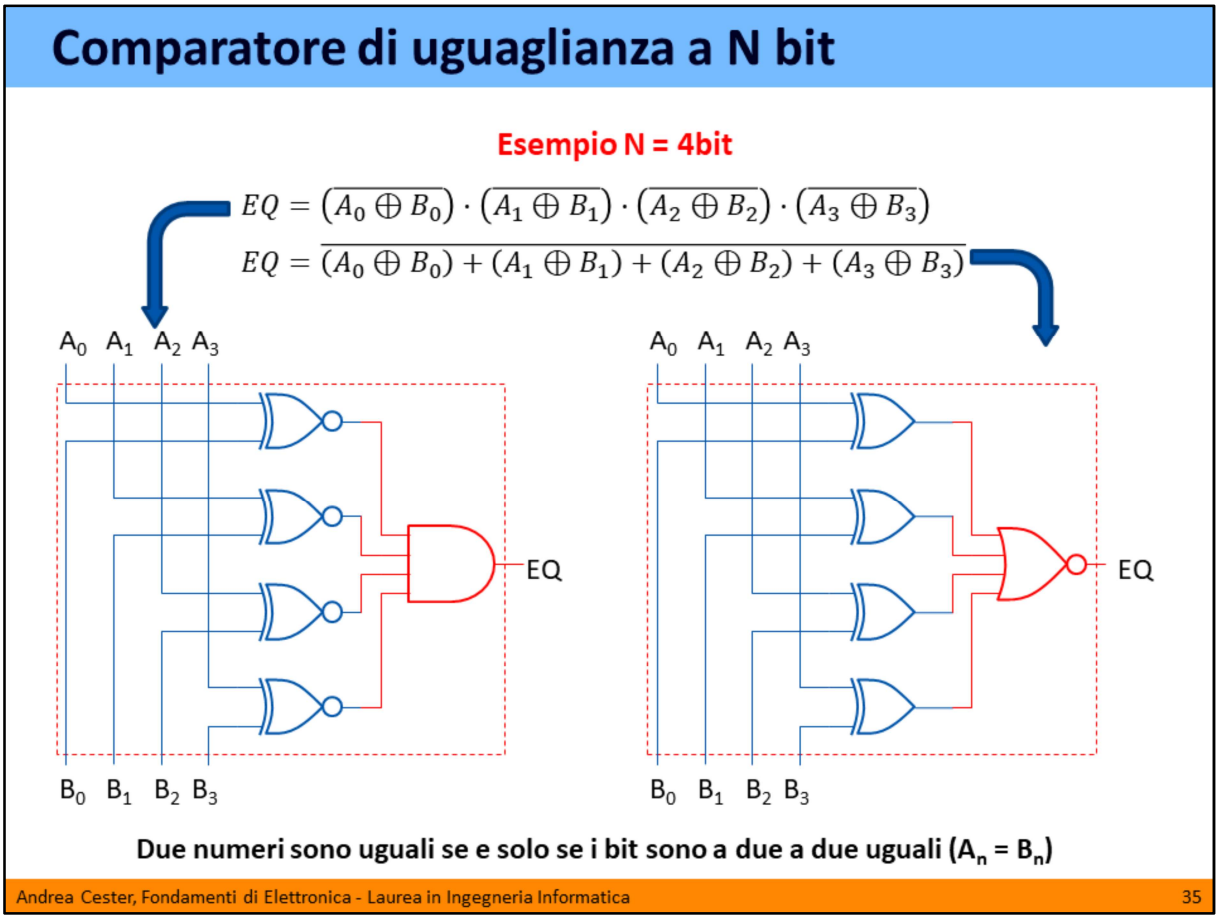
**Comparatore**

Scopo di un circuito comparatore è il confronto tra due numeri binari.
Il confronto può essere effettuato per verificare:

- L'uguaglianza
- La relazione d'ordine (maggiore, minore, maggiore o uguale, minore o uguale)

*Comparatore di uguaglianza a N bit*

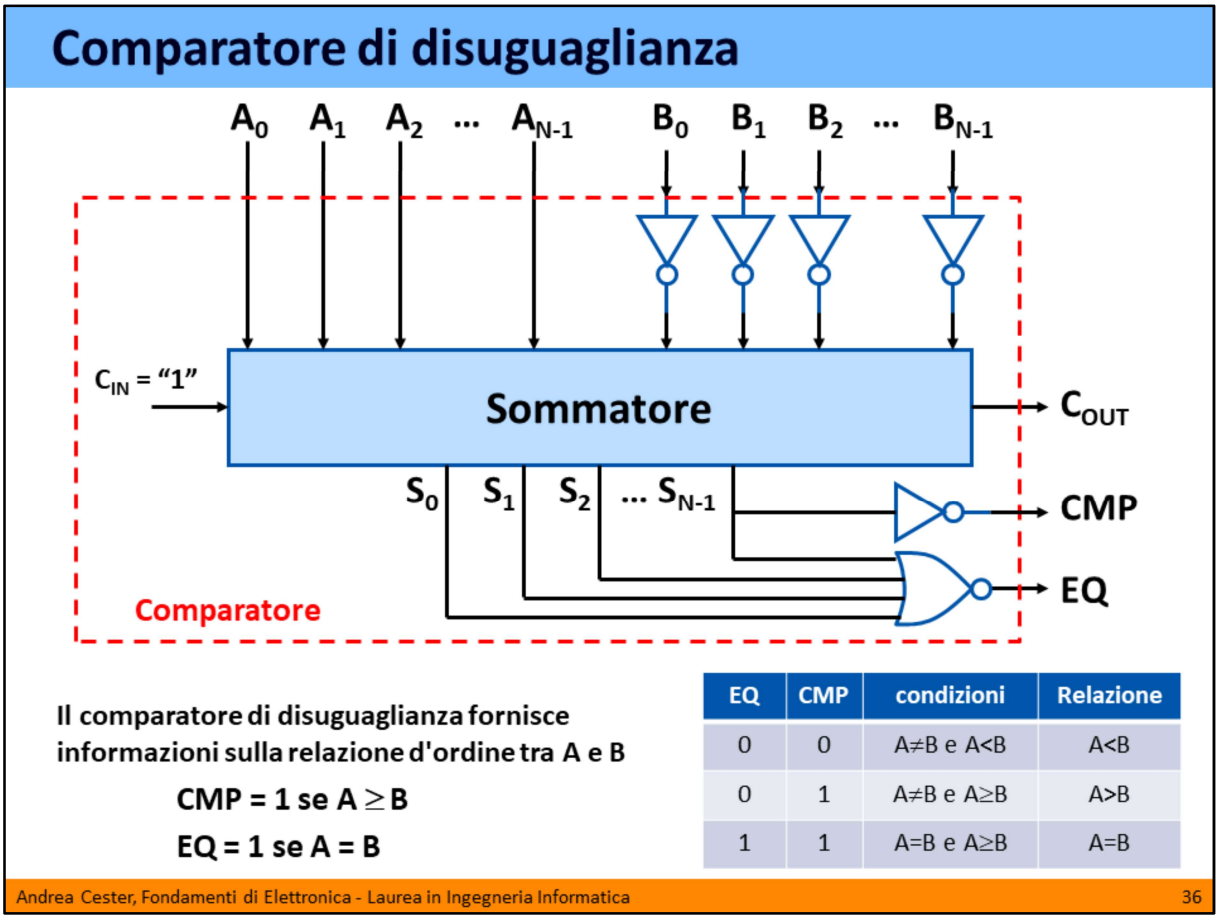
La funzione di comparazione tra una coppia di bit A e B è la semplice porta XNOR, la cui uscita vale 1 se e solo se A=B=0 o A=B=1.




*Comparatore di uguaglianza a N bit*

Per verificare l'uguaglianza di due numeri a N bit è necessario un circuito la cui uscita deve essere a 1 solo se $A_n$=$B_n$ per ogni n = 0, 1, …, N-1.

Il comparatore a N bit può essere fatto mediante N porte XNOR che verificano l'uguaglianza bit a bit, le cui uscite entrano in una porta AND a N ingressi, eventualmente implementata a più stadi per ottimizzare il tempo di propagazione.
Il circuito in figura riporta l'esempio di un comparatore di uguaglianza a 4 bit.
Alternativamente, usando il teorema di De Morgan, è possibile usare il circuito a destra con N porte XOR e una porta NOR.




*Comparatore di disuguaglianza*

Per verificare la relazione d'ordine è necessario identificare tre condizioni:

- A<B
- A=B
- A>B

Sono quindi indispensabili due bit per identificare quale delle tre relazioni esistono tra A e B.
Un modo per trovare la relazione d'ordine tra A e B è farne la differenza e controllare i bit di somma.

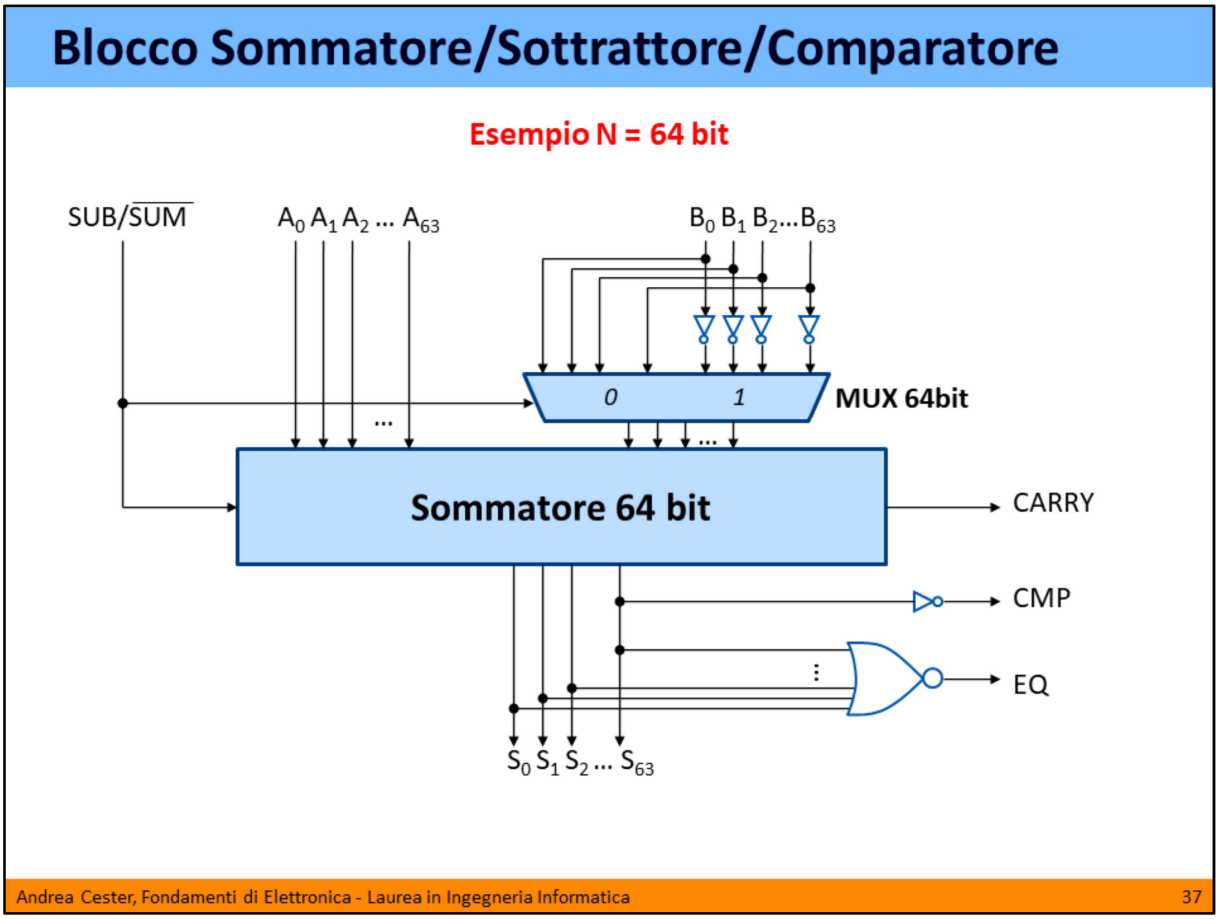
Se A < B → S = A – B < 0. Per verificare questa condizione è sufficiente controllare il bit di segno (cioè il più significativo). Definiamo:

$$CMP = \overline{S_{N-1}}$$

se CMP = 0 ($\overline{S_{N-1}} = 1$) allora S < 0.
Se CMP = 1 allora S $\geq$ 0 e A $\geq$ B. Per discriminare quale delle due condizioni è vera è necessario verificare se A = B o A $\neq$ B. La condizione A = B equivale a S = 0, cioè tutti i bit della somma devono essere 0. Si può usare una porta NOR che riceve come ingressi gli N bit di S. La porta NOR restituisce 1 in uscita se e solo se tutti i bit sono a 0.
Definiamo quindi il segnale:

$$EQ = \overline{S_0 + S_1 + \cdots + S_{N-1}}$$

Se EQ = 0 allora A $\neq$ B; se EQ = 1 allora A = B.
Nella tabella sono riassunti le tre combinazioni possibili per EQ e CMP. Notiamo che la combinazione EQ = 1 e CMP = 0 non è ammissibile. Infatti, se CMP = 0, allora A è strettamente minore di B ed EQ risulterà forzatamente 0.



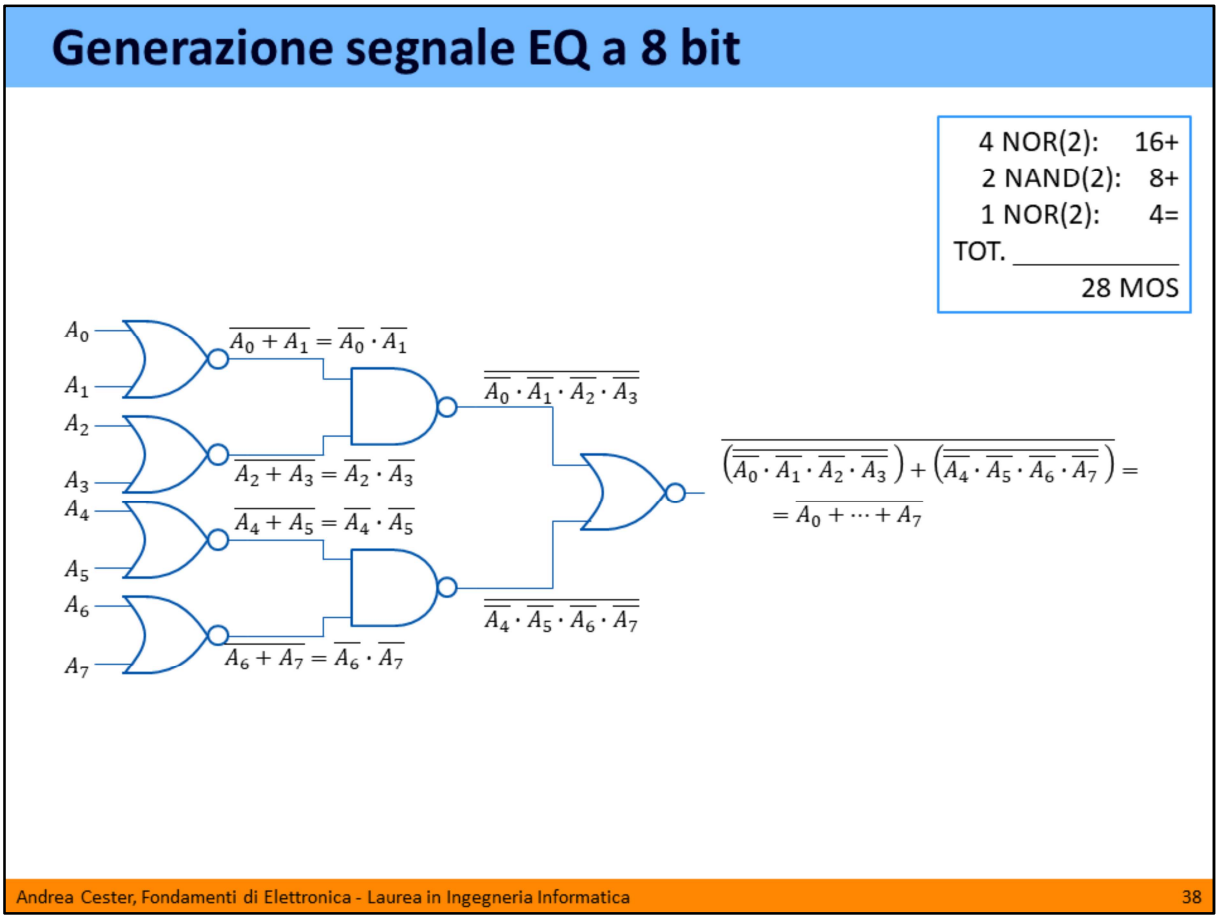

*Esempio somamtore/sotrattore/comparatore a 64 bit*

Con lievi modifiche è possibile realizzare un blocco sommatore/sottrattore/comparatore come mostrato in figura.
Il segnale $SUB/\overline{SUM}$ indica se l'operazione da svolgere è una somma (0) o una sottrazione (1).
Il segnale CMP è 1 se il risultato è positivo o nullo.
Il segnale EQ (cioè la NOR di tutti i bit di S) va a 1 se e solo se tutti i bit sono a 0 e indica l'uguaglianza.

Quindi eseguendo la differenza A-B possiamo avere:

- A > B se CMP = 1 e EQ = 0
- A = B se CMP = 1 e EQ = 1
- A < B se CMP = 0 e EQ = 0

Vale la pena notare che, sebbene la porta NOR sia disegnata come un'unica porta a N ingressi (64 in questo caso), nella pratica deve essere fatta con più stadi di porte logiche a meno ingressi, per mantenere i tempi di ritardo sufficientemente bassi.
Come abbiamo già detto, il numero massimo di MOSFET in serie nel PDN o PUN di una porta logica, così come il numero massimo di pass-T/TG in serie è limitato tra 3 e 5.
Tenendo conto che in una porta NOR a N ingressi il pull-up è formato da N PMOS in serie, questo limita il numero di ingressi a 3-5.
Nel seguito vediamo quattro esempi di porte logiche NOR realizzati in più stadi utilizzando porte a 2 o 4 ingressi.

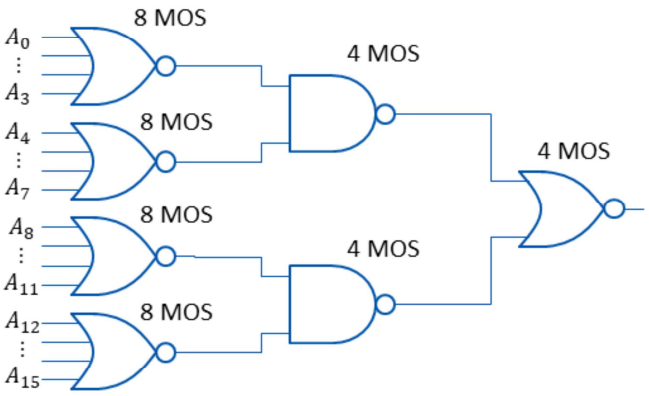



*Porta NOR a 8 ingressi*

L'esempio mostra uno dei modi possibili per realizzare la porta NOR. In questo esempio, si è deciso di adottare porte NAND e NOR a due ingressi con una struttura ad albero. Nella tabella è mostrato il calcolo del numero dei MOS richiesti, se si adotta la logica statica complementare.

## Generazione segnale EQ a 16 bit

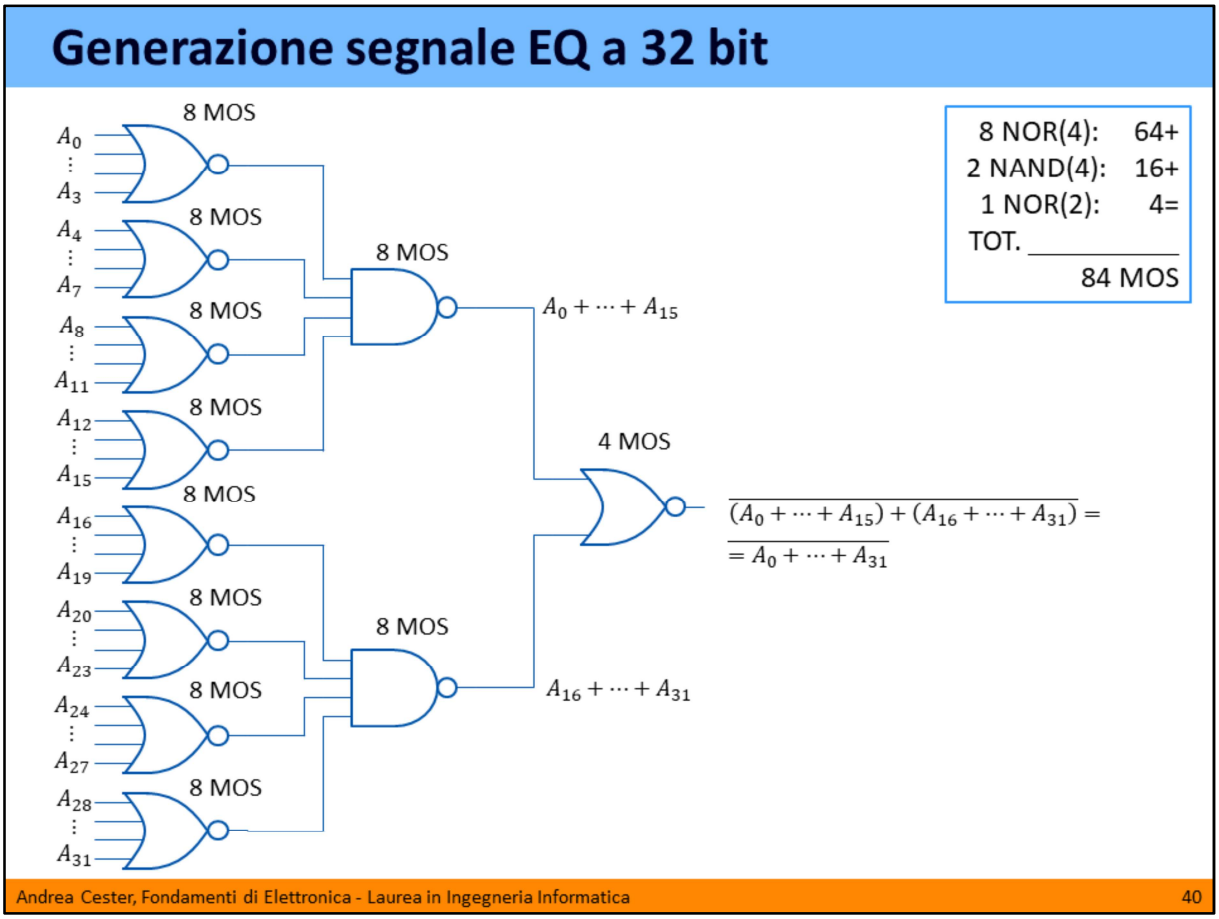
| | |
|---|---|
| 4 NOR(4): | 32+ |
| 2 NAND(2): | 8+ |
| 1 NOR(2): | 4= |
| TOT. | |
| | 44 MOS |




Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 39

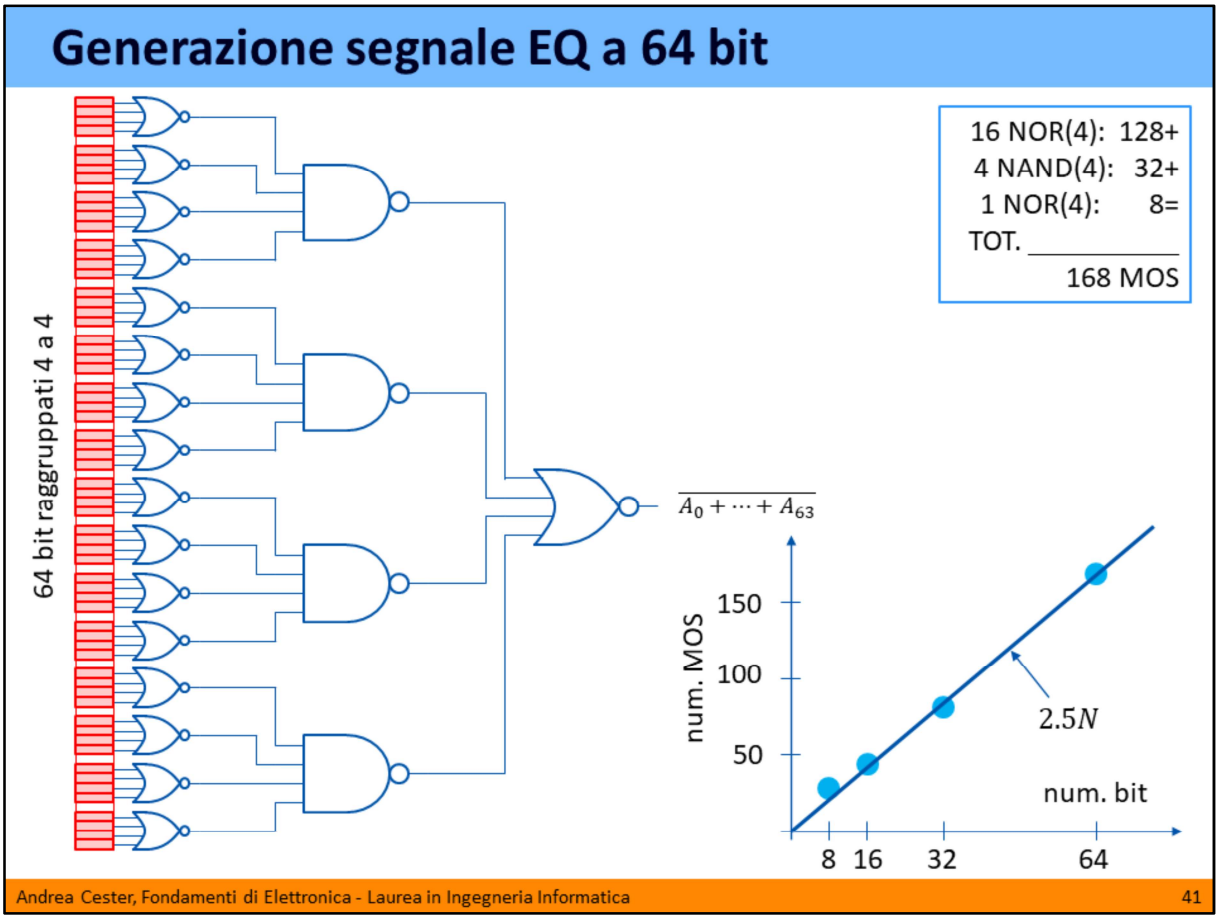
*Porta NOR a 16 ingressi*

La strategia è simile all'esempio precedente. L'unica differenza è l'impiego di 4 porte NOR a 4 ingressi ciascuna nel primo stadio. Anche in questo caso il calcolo del numero dei MOS riportato in figura e in tabella si riferisce all'implementazione in logica statica complementare.




*Porta NOR a 32 ingressi*

In questo terzo esempio, volendo raddoppiare il numero di ingressi e mantenere la stessa topologia dei precedenti esempi, si deve adottare porte NAND a quattro ingressi nel secondo stadio. Il calcolo del numero dei MOS riportato in figura e in tabella si riferisce sempre all'implementazione in logica statica complementare.




*Porta NOR a 64 ingressi*

Come ultimo esempio, una porta NOR a 64 ingressi può essere realizzata con struttura ad albero usando tutte porte a 4 ingressi (8 MOS ciascuna): 16 NOR nel primo stadio, 4 NAND nel secondo, 1 NOR nell'ultimo stadio. In logica statica complementare ciò richiede 168 MOS.
Nel grafico vediamo l'andamento del numero di MOS richiesti in funzione del numero di bit nei quattro esempi appena fatti.
Va precisato che il tipo di implementazione adottata è solo un esempio a scopo di stimare come aumenta l'occupazione di are in funzione del numero di bit.

## Numero di MOS stimato per un sommatore

F.A. A pass-T/TG: 12N+12P = 24 MOS

| Architettura | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| **Ripple carry** | 192 | 384 | 768 | 1536 | 24N |
| **Sottrattore (MUX)** | 66 | 132 | 258 | 514 | 8N+2 |
| **(add/sub/comp)** | 28+2 | 44+2 | 84+2 | 128+2 | 2.5N+2 (circa) |
| **Tot.** | **288** | **562** | **1112** | **2180** | **34.5N+4 (circa)** |

**Circa il 70% dei MOS usati servono per l'implementazione dei FA del sommatore, cuore di tutto il sistema**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 42

*Esempio: stima del numero di MOS in un sommatore/sottrattore/comparatore ripple carry*

Supponiamo di realizzare un sommatore/sottrattore/comparatore a N bit in architetture ripple carry usando full-adder in architettura pass-T/TG e usando per MUX e altri blocchi logici l'implementazione adottata nei precedenti esempi.
Vogliamo calcolare la quantità di MOS richiesta in funzione del numero di bit.

Il cuore del blocco aritmetico sotto studio è il sommatore, composto da N full-adder in architettura ripple-carry. Adottano l'implementazione a TG vista precedentemente, sono necessari 24 MOS per full-adder. Il numero totale di MOS per il sommatore è 24N.

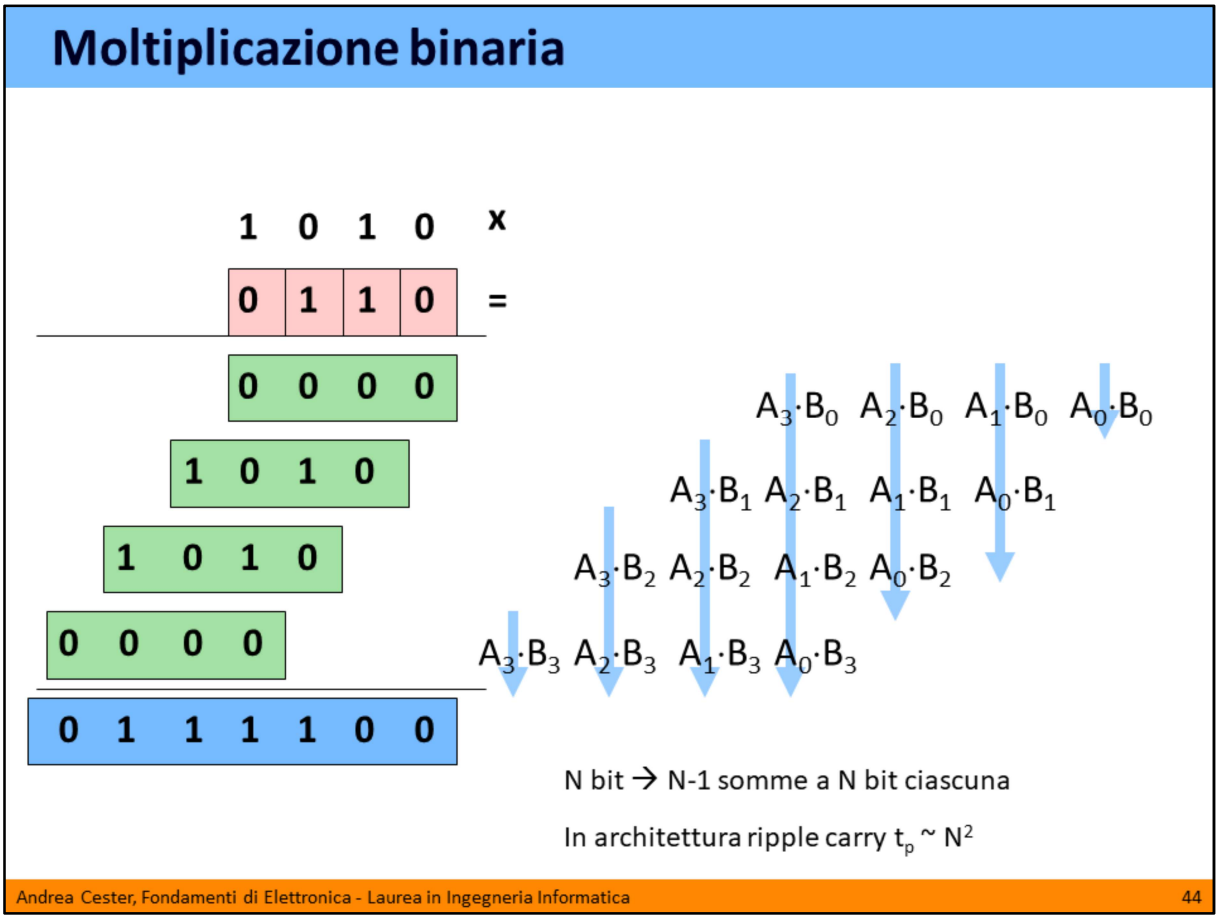
Per aggiungere la funzionalità di sottrattore, sono necessari N MUX a due ingressi ciascuno. Usando MUX a TG abbiamo bisogno di due TG e due invertitori per ciascun MUX (ricordiamo che non serve l'invertitore all'ingresso 0, dove entra B negato). È necessario anche un invertitore aggiuntivo per negare il segnale di controllo (comune a tutti i MUX). In totale abbiamo 8N+2 MOS per il vettore di MUX.

Se vogliamo aggiungere anche la funzionalità di comparazione, bisogna inserire un invertitore per negare il bit più significativo della somma (2 MOS) e quelli necessari per la porta NOR (circa 2.5N).

Sommando i vari contributi, otteniamo approssimativamente 34.5N+4 MOS. Nella tabella sono riportati i valori trovati negli esempi precedenti per 8, 16, 32, 64 bit.
Sebbene il numero esatto dipende dal tipo di implementazioni delle porte usate (NOR, MUX, FA, …), ci sono due risultati importanti:

1. la dipendenza lineare del numero di MOS dal numero di bit.
2. Circa il 70% dei MOS sono impiegati per realizzare i FA del sommatore.

# MOLTIPLICATORE




**Moltiplicatore**

Il moltiplicatore è un altro componente chiave di una ALU. Esistono diverse strategie di implementazione. Nel seguito ne verrà mostrata una a titolo di esempio, detta moltiplicatore a conservazione del riporto (*carry save*).

*Moltiplicazione in base 2*

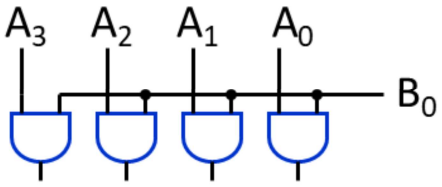
L'operazione di moltiplicazione tra due numeri A e B a N bit è eseguita in due fasi:
1) Prodotti parziali. Il numero A viene moltiplicato per ciascuna cifra binaria di B.
2) Somma dei prodotti. Gli N prodotti parziali vengono sommati in diagonale eseguendo N-1 somme.
Nell'implementazione di un moltiplicatore, queste due fasi corrispondono a due stadi principali in cui è diviso un moltiplicatore.

## Moltiplicatore: prodotti parziali

| A | B | A x B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

A x B = A and B

**È possibile generare tutte le moltiplicazioni in parallelo mediante una semplice porta AND**




Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 45

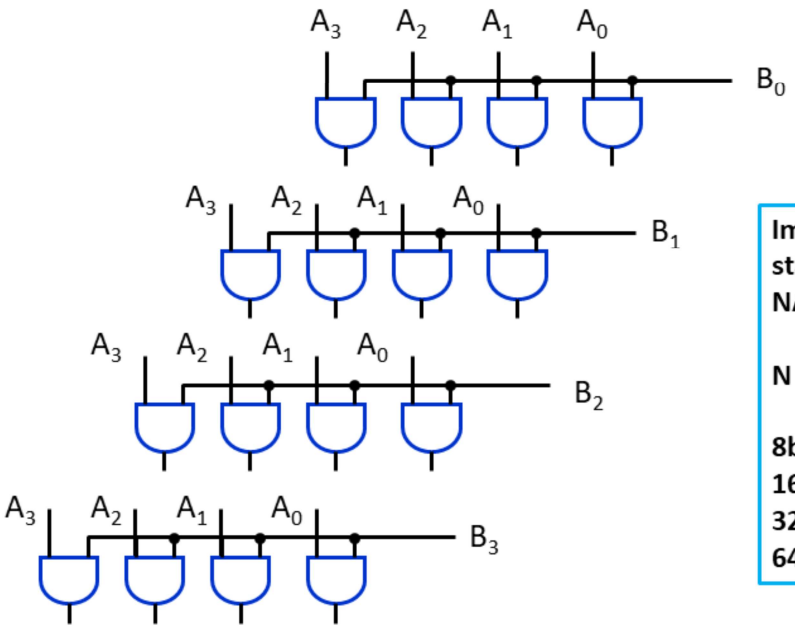
*Prodotti parziali*

Il prodotto parziale è ottenuto moltiplicando ciascuno degli N bit del numero B per il numero A (sempre a N bit).
Se $A_n$ e $B_n$ sono i due bit n-esimi di A e B, il loro prodotto ha la stessa tabella di verità della funzione AND. Cioè $A_n \cdot B_n = 1$ se e solo se $A_n = B_n = 1$.

Ad esempio, se N = 4 il prodotto di A e il bit $B_0$ di B, equivale alle 4 porte AND mostrate in figura. Il risultato sarà identico ad A se $B_0 = 1$, altrimenti sarà 0.

## Moltiplicatore: prodotti parziali

**Calcolo dei prodotti $A_i$ x $B_j$**



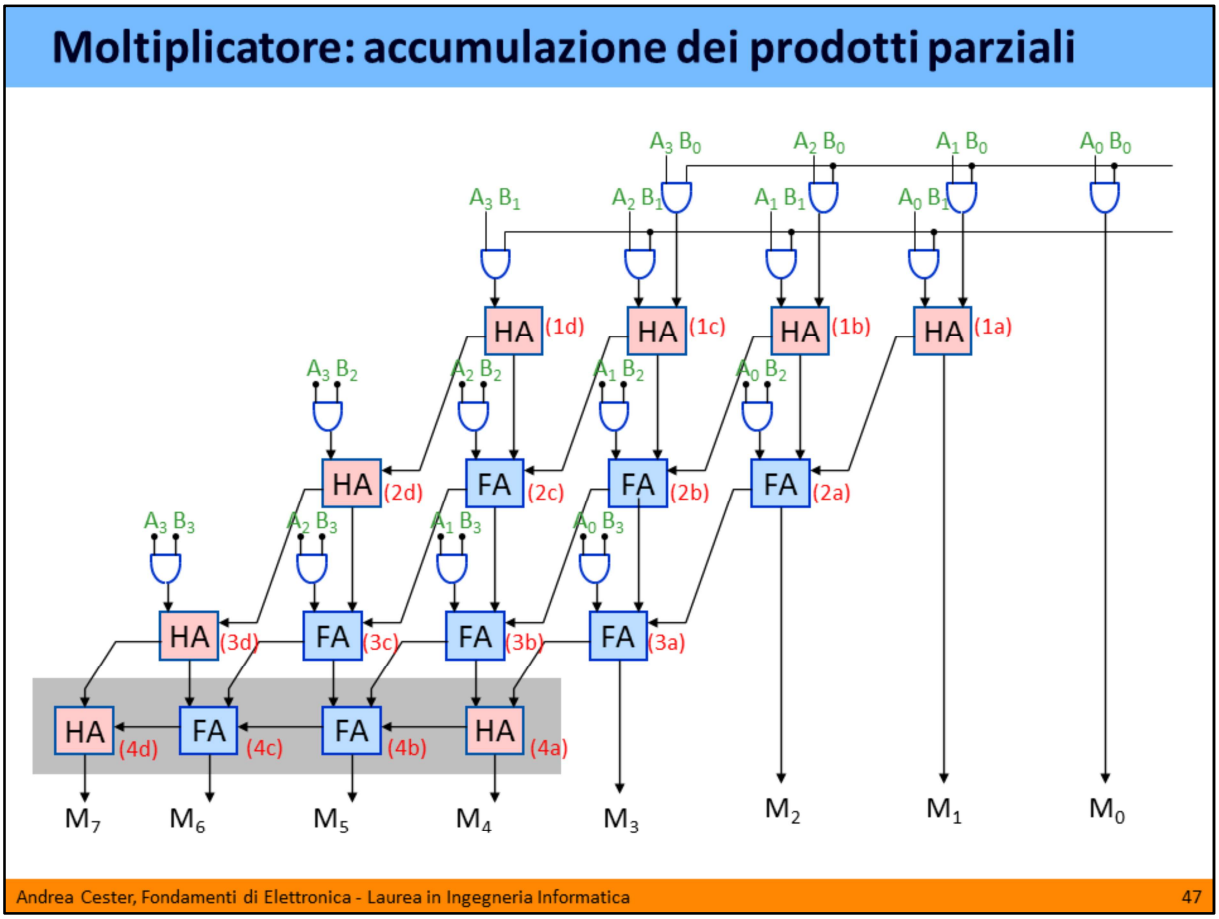

**Implementazione in logica statica complementare:**
**NAND + NOT: 4 + 2 = 6 MOS**

**N bit → $6N^2$ MOS:**

**8bit → 384 MOS**
**16bit → 1536 MOS**
**32bit → 6144 MOS**
**64bit → 24576 MOS**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 46

Poiché ciascun bit di B deve essere moltiplicato per il numero A, otteniamo una matrice di prodotti parziali a cui corrisponde una matrice di N x N porte AND a due ingressi. Ciascuna porta AND riceve in ingresso un bit di A e uno di B implementando tutte le possibili combinazioni.
Nel circuito disegnato, le porte AND nella riga k-esima ricevono in ingresso il bit $B_k$ e ciascuna porta lo moltiplica per un diverso bit di A.




**Somma dei prodotti parziali**

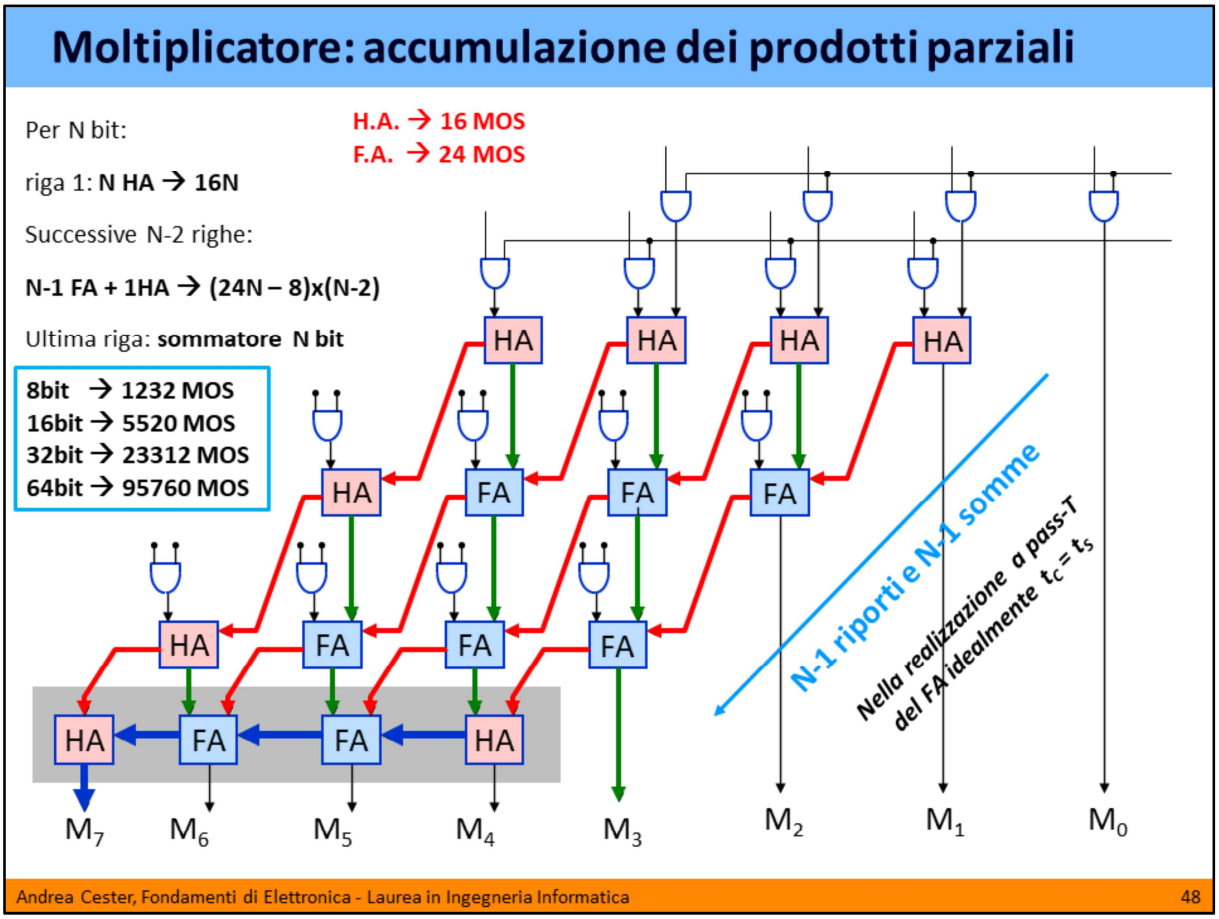
I prodotti parziali devono essere sommati per diagonali, ovvero è necessario traslare a sinistra di un bit ciascuna riga di prodotti parziali. È facile verificare guardando lo schema blocchi in figura che il bit k-esimo del prodotto, corrisponde alla somma di tutti i prodotti parziali $A_nB_m$ tali che n+m = k.

Il *bit* $M_0$ del prodotto equivale al prodotto $A_0B_0$.

Il *bit* $M_1$ del prodotto equivale alla somma $A_1B_0 + A_0B_1$. Il riporto in ingresso per questa somma è sempre 0, quindi è possibile usare un HA per eseguire la somma stessa. La somma genera un riporto che definiamo $C_1$ che deve essere sommato nella colonna successiva.

Il bit $M_2$ del prodotto equivale alla somma: $A_2B_0 + A_1B_1 + A_0B_2$. A questo risultato va sommato anche il riporto $C_1$ della somma precedente. Mentre i prodotti parziali $A_2B_0$, $A_1B_1$ e $A_0B_2$ sono subito disponibili, il riporto $C_{1A}$ richiede il tempo di calcolo di un HA (HA 1° in figura). Il modo ottimale di procedere è eseguire prima la somma $A_2B_0 + A_1B_1$ mediante l'HA 1b che lavora in parallelo all'HA 1a. Successivamente usare un FA per eseguire la somma tra l'uscita del HA 1b e il prodotto parziale $A_0B_2$, usando come riporto in ingresso $C_1$. Dovendo eseguire due somme avremo in uscita due riporti (uno per ciascuna somma), che chiamiamo $C_{1B}$ e $C_{2A}$ (il pedice indica il nome del FA o HA che lo genera).

Il bit $M_3$ del prodotto è dato dalla somma: $A_3B_0 + A_2B_1 + A_1B_2 + A_0B_3$ a cui vanno aggiunti i riporti $C_{1B}$ e $C_{2A}$. Procedendo in modo analogo al precedente, sommiamo prima $A_3B_0 + A_2B_1$ mediante l'HA 1c, producendo la somma $S_{1C}$ e il riporto $C_{1C}$. A questo punto usiamo un FA (2b) per sommare $S_{1C}$ con $A_1B_2$ usando come riporto di ingresso $C_{1B}$ (che nel frattempo è pronto, poiché richiede il tempo di calcolo di un HA). Infine sommiamo la somma ottenuta ($S_{2B}$) con $A_0B_3$ mediante il FA 3a, usando come riporto di ingresso $C_{2A}$, che richiede il tempo di calcolo di un HA e un FA, come $S_{2B}$.




Procedendo in questo modo andiamo a calcolare tutti gli altri bit.
Ad esempio $M_4 = A_3B_1 + A_2B_2 + A_1B_3 + C_{1C} + C_{2B} + C_{3A}$. I tre riporti vengono sommati non appena pronti. In particolare, $C_{1C}$ richiede il tempo di calcolo di un HA, $C_{2B}$ il tempo di un HA e un FA, $C_{3A}$ il tempo di calcolo di una HA e due FA.
La somma viene quindi eseguita in quest'ordina:

$$M_4 = \{\{[(A_3B_1 + 0)_{HA} + A_2B_2 + C_{1C}]_{FA} + A_1B_3 + C_{2B}\}_{FA} + C_{3A}\}_{HA}$$

I pedici sulle parentesi indicano il tipo di circuito che esegue la somma.
I due FA e i due HA usati nella somma, generano i riporti nel seguente ordine temporale: $C_{1D}$, $C_{2C}$, $C_{3B}$ e $C_{4A}$.
Il bit $M_5$ è dato dalla somma: $A_3B_2 + A_2B_3 + C_{1D} + C_{2C} + C_{3B} + C_{4A}$
Tenendo conto dei tempi di calcolo la somma viene fatta nel seguente ordine:

$$M5 = \{[(A_3B_2 + C_{1D})_{HA} + A_2B_3 + C_{2C}]_{FA} + C_{3B} + C_{4A}\}_{FA}$$

E così via fino a $M_7$.
Dallo schema in figura osserviamo il cammino di propagazione di somma (verde) e riporto (rosso). Con N stadi, abbiamo N-1 somme iniziali che procedono in sequenza. Ciascun riporto viene sommato nello stadio successivo. Queste N-1 somme generano i bit $M_3M_2M_1M_0$ del prodotto.
I bit da $M_4$ a $M_7$ sono generati da un sommatore finale, che in figura è stato rappresentato in architettura ripple carry.

Il nome *carry-save* (*conservazione del riporto*), si riferisce proprio a questa tecnica in cui i riporti non vengono propagati immediatamente lungo i singoli stadi del sommatore (in modo orizzontale), ma vengono trattenuti e sommati in uno stadio successivo. Ovviamente ciò richiede uno stadio solo nell'ultimo stadio del circuito.

## Numero di MOS stimato per un moltiplicatore

F.A. A pass-T/TG: 12N+12P = 24 MOS

H.A. A pass-T/TG: 8N+8P = 16 MOS

| Sommatore (int) | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Ripple carry | 192 | 384 | 768 | 1536 | 24N |

| Moltiplicatore (int) | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Matrice di AND | 384 | 1536 | 6144 | 24576 | $6N^2$ |
| Somme parziali (HA) | 224 | 480 | 992 | 2016 | $32(N-1)$ |
| Somme parziali (FA) | 1008 | 5040 | 22320 | 93744 | $24(N-1)(N-2)$ |
| Sub.Tot. | 1616 | 7056 | 29456 | 120336 | $30N^2-40N+16$ |
| **Tot. (ripple carry finale)** | **1808** | **7440** | **30224** | **121872** | $30N^2-16N+16$ |

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 49

*Stima del numero di MOSFET*

Assumendo, ad esempio, di realizzare FA e HA con pass-T e le porte AND in logica statica complementare (come illustrato negli esempi pre4cedenti), per moltiplicare due numeri a N bit sono necessari:

- $N^2$ porte AND per generare i prodotti parziali. Usando 6 MOS per porta, risultano $6N^2$ MOS.
- N HA nella prima riga di prodotti parziali, e 1 HA nelle successive righe (tranne la somma finale). In totale sono 2N – 2 HA. Realizzando gli HA a TG abbiamo 16 MOS per HA e 32(N – 1) MOS in totale.
- N – 1 FA nelle successive N – 1 righe tranne l'ultima che realizza la somma finale. Usando l'implementazione a TG con 24 MOS per ciascun FA, risultano in totale 24(N – 1)(N – 2) MOS

Escludendo la somma finale, i MOS necessari sono in totale:

$$6N^2 + 32(N-1) + 24(N-1)(N-2) = 30N^2 - 40N + 16$$

Assumendo di realizzare l'ultima somma in architettura ripple carry, il numero di MOS totale è:

$$30N^2 - 16N + 16$$

La tabella mostra il numero di MOS per un moltiplicatore carry-save con 8, 16, 32 e 64 bit. È facile constatare che la maggior parte dei MOS è necessaria per realizzare i FA e le porte AND, che sono i contributi da cui trae origine il termine $N^2$.

## Moltiplicatore: somma dei prodotti parziali

- Per il calcolo del prodotto è necessario aspettare:
  - Il tempo di propagazione attraverso le porte AND (array di prodotti bit a bit)
  - Il tempo di propagazione di (N – 1) riporti (o somme)
  - Il tempo di propagazione dell'ultimo stadio sommatore

$$T_{MULT} = t_{AND} + (N-1)\max(t_C, t_S) + T_{SUM}$$

**Se il sommatore dell'ultimo stadio è realizzato in architettura ripple-carry, il tempo totale si moltiplicazione è:**

$$T_{MULT} = t_{AND} + (N-1)\max(t_C, t_S) + (N-1)t_C + t_S$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 50

*Tempo di calcolo del prodotto*

Per il calcolo del prodotto è necessario aspettare:

- Il tempo di propagazione attraverso le porte AND (array di prodotti bit a bit), che richiede un tempo $t_{AND}$
- Il tempo di propagazione di (N – 1) riporti (o somme), che richiede un tempo (N–1) $\max(t_C, t_S)$
- Il tempo di propagazione dell'ultimo stadio sommatore, che richiede un tempo $(N-1)\cdot t_C + t_S$

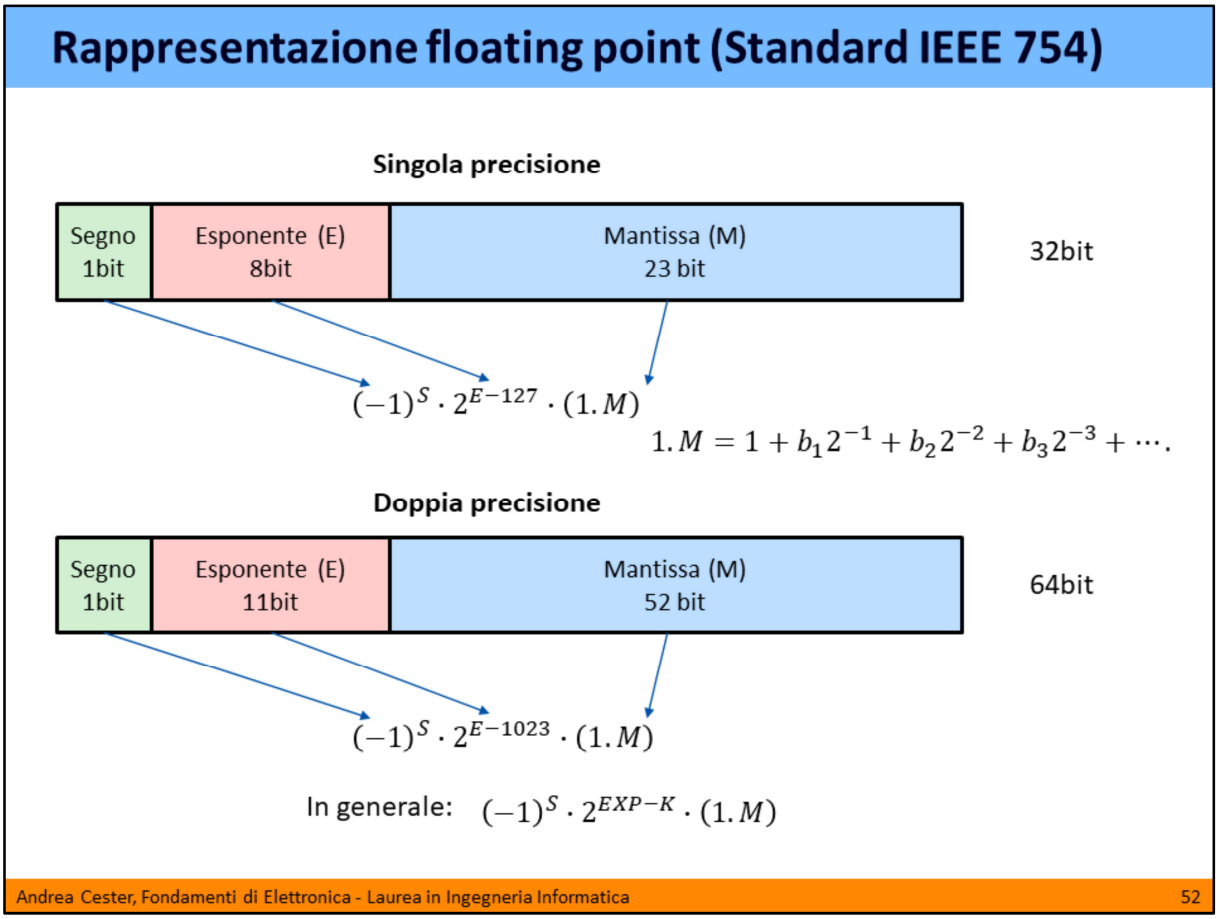
Il tempo di calcolo totale è:

$$t_{MULT} = t_{AND} + (N-1)\cdot \max(t_C, t_S) + (N-1)\cdot t_C + t_S$$

Supponendo che $t_C$ sia circa uguale a $t_S$ l'espressine si semplifica in:

$$t_{MULT} = t_{AND} + (2N-1)\cdot t_C$$

# OPERAZIONI FLOATING POINT: SOMMA/SOTTRAZIONE




**Rappresentazione dei numeri in floating point (Standard IEEE 754)**

I numeri floating point (virgola mobile ) sono rappresentati seguendo lo standard internazionale IEEE 754, che suddivide i bit in tre parti: segno (S), esponente (E) e mantissa (M).
Il valore numerico memorizzato è il seguente:

$$valore = (-1)^S \cdot (1 + M) \cdot 2^{E-bias}$$

S è il bit di segno: S = 0 significa un numero positivo; S = 1 significa un numero negativo.
Invece, il numero di bit usati per E e M e il valore di *bias* dipendono dalla precisione usata.

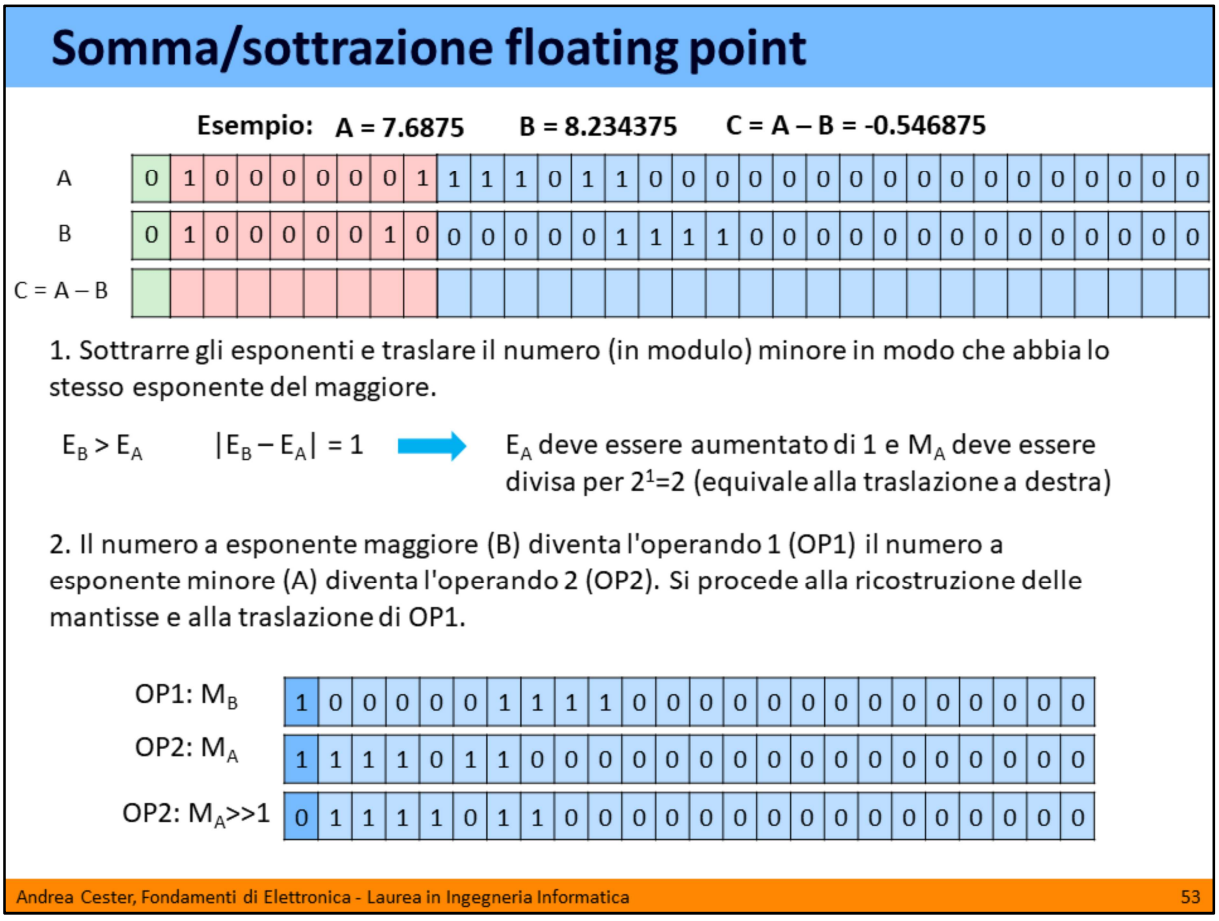
*Singola precisione*

- bias = 127
- E è un campo a 8 bit che memorizza un intero senza segno (da 0 a 255). La potenza di 2 che moltiplica la mantissa è (E – bias) e può variare da -127 a 128
- M è un campo a 23 bit che memorizza la parte frazionaria della mantissa (assumendo che la parte intera sia sempre 1), che porta i bit effettivi a 24. Per questo motivo si usa anche rappresentare la mantissa estesa come 1.M.

*Doppia precisione*

- bias = 1023
- E è un campo a 11 bit che memorizza un intero senza segno (da 0 a 2048). La potenza di 2 varia quindi da -1023 a 1024.
- M è un campo a 53 bit che memorizza la parte frazionaria della mantissa.

A parte il numero di bit per ciascun campo, la filosofia di rappresentazione è la stessa.




**Sommatore floating-point**

*Esempio di somma/sottrazione di numeri in singola precisione*

Dati i numeri A = 7.6875 e B = 8.234375, vogliamo trovare la differenza C = A – B.
Innanzitutto esprimiamo i due numeri nel formato IEEE 754:

$$A = 1.921875 \cdot 2^2 \quad e \quad B = 1.029296875 \cdot 2^3$$

Esprimiamo le mantisse di A e B in formato binario:

$$A = (1.111011)_b 0 \cdot 2^2 \quad e \quad B = (1.000001111)_b \cdot 2^3$$

Che forniscono:

$$S_A = 0 \quad E_A = bias + 2 = 129 = 10000001_b \quad M_A = 111011_b$$
$$S_B = 0 \quad E_B = bias + 3 = 10000010_b \quad M_B = 000001111_b$$

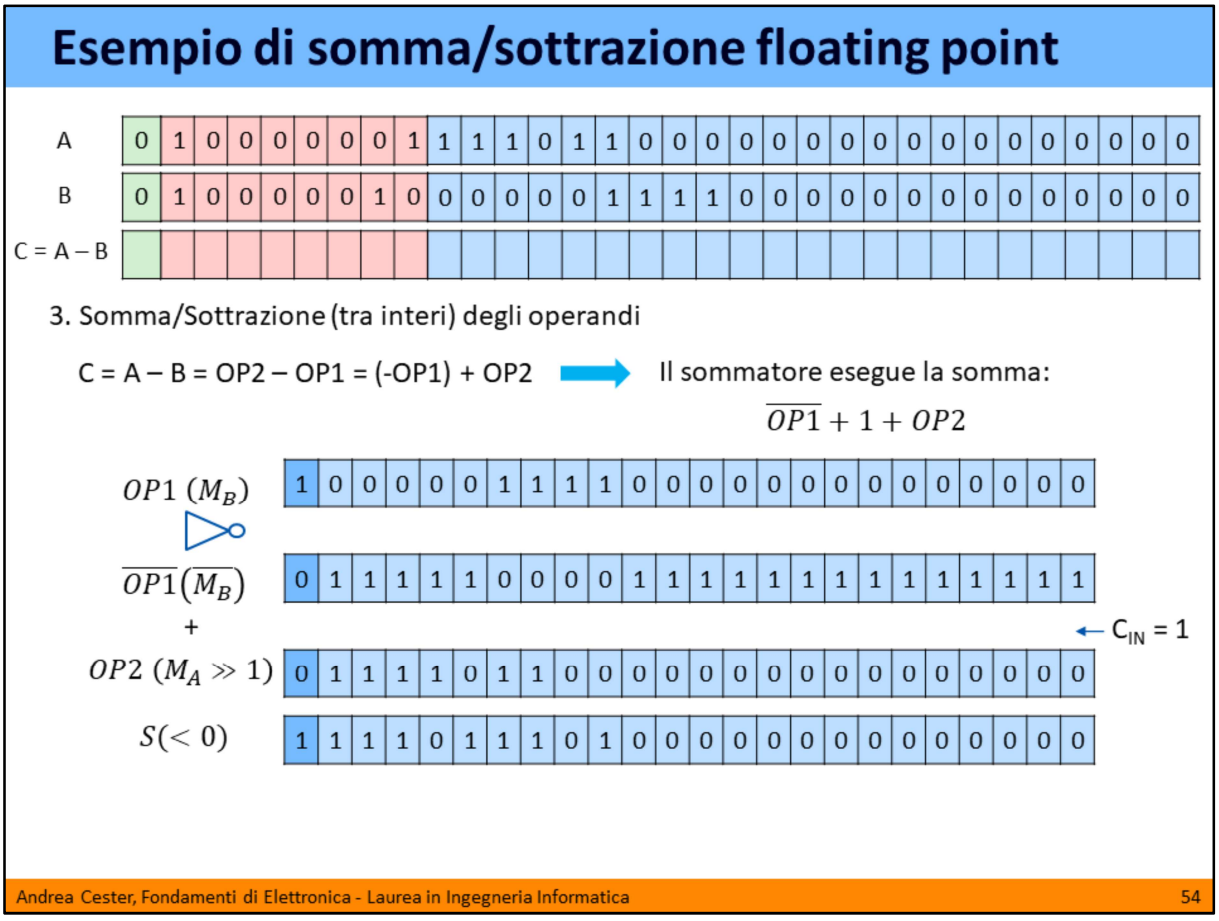
L'algoritmo di somma è il seguente:

*1) Allineamento degli esponenti*
Non potendo sommare le mantisse se i numeri hanno ordini di grandezza diversi, il sistema confronta i due esponenti e prende come riferimento quello più grande (che definiamo operando 1, OP1). La mantissa del numero con l'esponente più piccolo (OP2) viene traslata verso destra di un numero di posizioni pari alla differenza tra i due esponenti.
Nell'esempio illustrato, il numero minore è A (esponente 2). Quindi: OP1 = B, OP2 = A, esponente di A deve essere incrementato di 1 per renderlo uguale a quello di B.

*2) Ricostruzione e allineamento delle mantisse.*
L'aumento dell'esponente di A di una quantità k (=1 nel nostro esempio) corrisponde alla divisione per $2^k$ della corrispondente mantissa:

$$A = (1.111011)_b \cdot 2^2 = (0.1111011)_b \cdot 2^3$$

È necessario quindi ricostruire le mantisse complete aggiungendo un 1 come MSB e traslare quella del numero più piccolo (A nel nostro caso) k bit a destra inserendo 0 al posto dell'MSB. Nell'esempio riportato, la mantissa di A diventa: $(1.M_A)>>1 = 0.1111011_b$



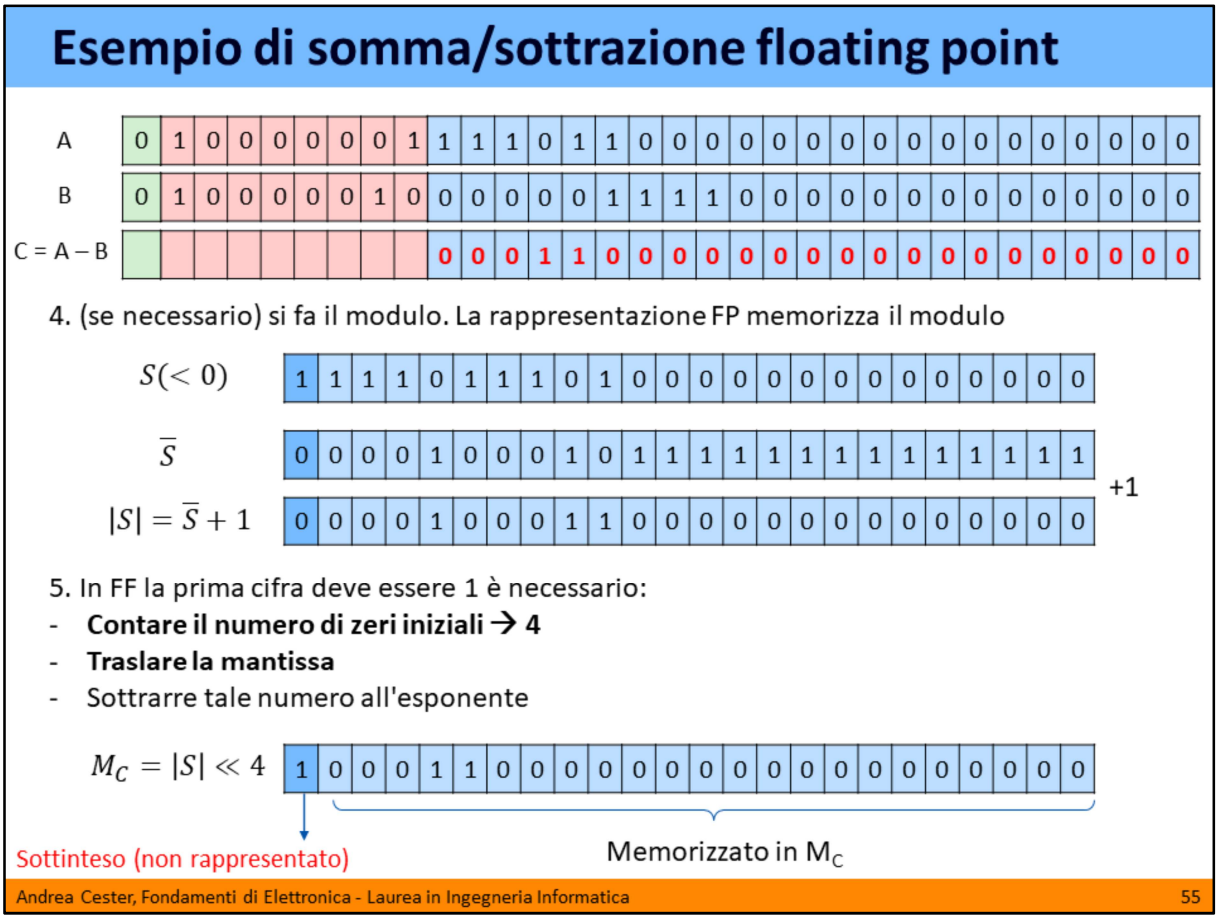

*3) Somma algebrica delle mantisse*

Avendo lo stesso esponente, le mantisse possono essere sommate (o sottratte a seconda dei loro segni). Nell'esempio studiato vogliamo calcolare C = A – B. Essendo B positivo, ciò equivale a sommare il complemento a due di B (negazione di B e $C_{IN}$ = 1).

Il risultato della somma è:

$$1.000001111 + 0.111101100 = 1.111011101$$

ed è negativo.




*4) Calcolo del modulo e del segno del risultato.*
Poiché il formato IEEE 754 memorizza il modulo della mantissa e riserva un bit per il segno è necessario valutare il segno del risultato e calcolarne il modulo.
In questo caso il segno è negativo, quindi $S_C$ = 1. Il modulo si esegue negando il risultato e sommando 1 (complemento a 2):

$$1.11101110100 \dots 0 \rightarrow 0.00010001011 \dots 1 \rightarrow 0.00010001100 \dots 0$$

*5) Normalizzazione del risultato.*
Lo standard IEEE 754 prevede che la mantissa sia normalizzata in modo che la cifra intera sia 1. è necessario traslare il numero intero di una quantità di cifre pari al numero di zeri iniziali (4 in questo caso):

$$0.00010001100 \dots 0 \ll 4 \rightarrow 1.0001100 \dots 0$$

La mantissa finale risulta:

$$1.\, M_C = 1.0001100 \dots 0 \rightarrow M_C = 0001100 \dots 0$$

## Esempio di somma/sottrazione floating point

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| C = A – B | | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

5. In FF la prima cifra deve essere 1 è necessario:
- Contare il numero di zeri iniziali
- Traslare la mantissa
- **Sottrarre tale numero all'esponente**

4 zeri

$M_C = |S| \ll 4$ | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$\max(E_A, E_B) = E_B$ | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | $E_B(3) + 127 = 130$

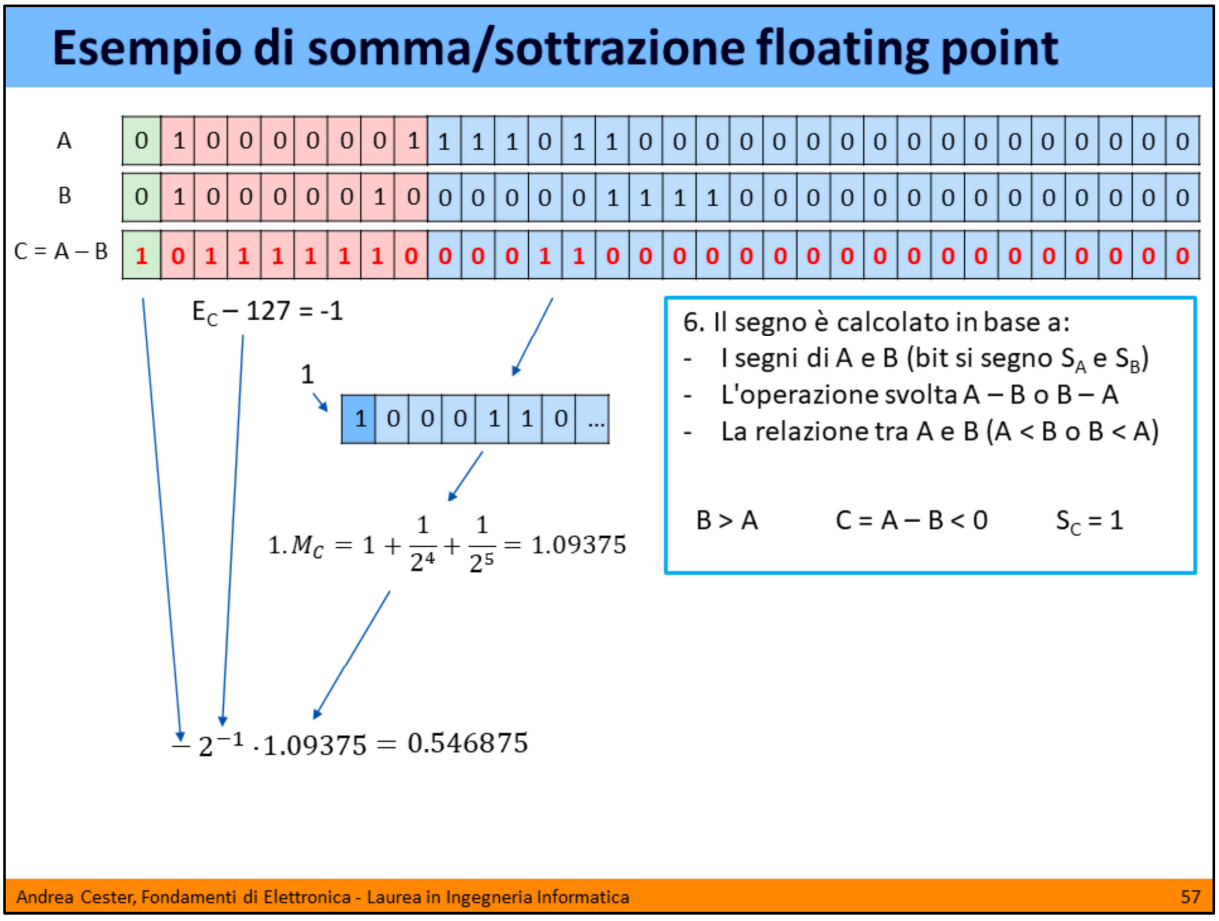
$-4$ | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |

$E_C = E_B - 4$ | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | $E_C(-1) + 127 = 126$



La traslazione a sinistra di 4 cifre corrisponde a una moltiplicazione per $2^4$. L'esponente del risultato deve essere quindi ridotto di 4 unità.

$$E_C = E_B - 4 = 126 = 01111110_b$$




Otteniamo infine la rappresentazione di C:

$$C = (-1)^{S_C} \cdot (1.M_C) \cdot 2^{E_C - bi}$$
$$S_C = 1$$
$$M_C = 0001100 \ldots 0$$
$$E_C = 01111110$$

Verifichiamo che:

$$1.M_C = 1 + 0\frac{1}{2} + 0\frac{1}{4} + 0\frac{1}{8} + 1\frac{1}{16} + 1\frac{1}{32} = 1.09375$$
$$C = (-1)^1 \cdot 1.09375 \cdot 2^{-1} = -0.546875 = A - B$$

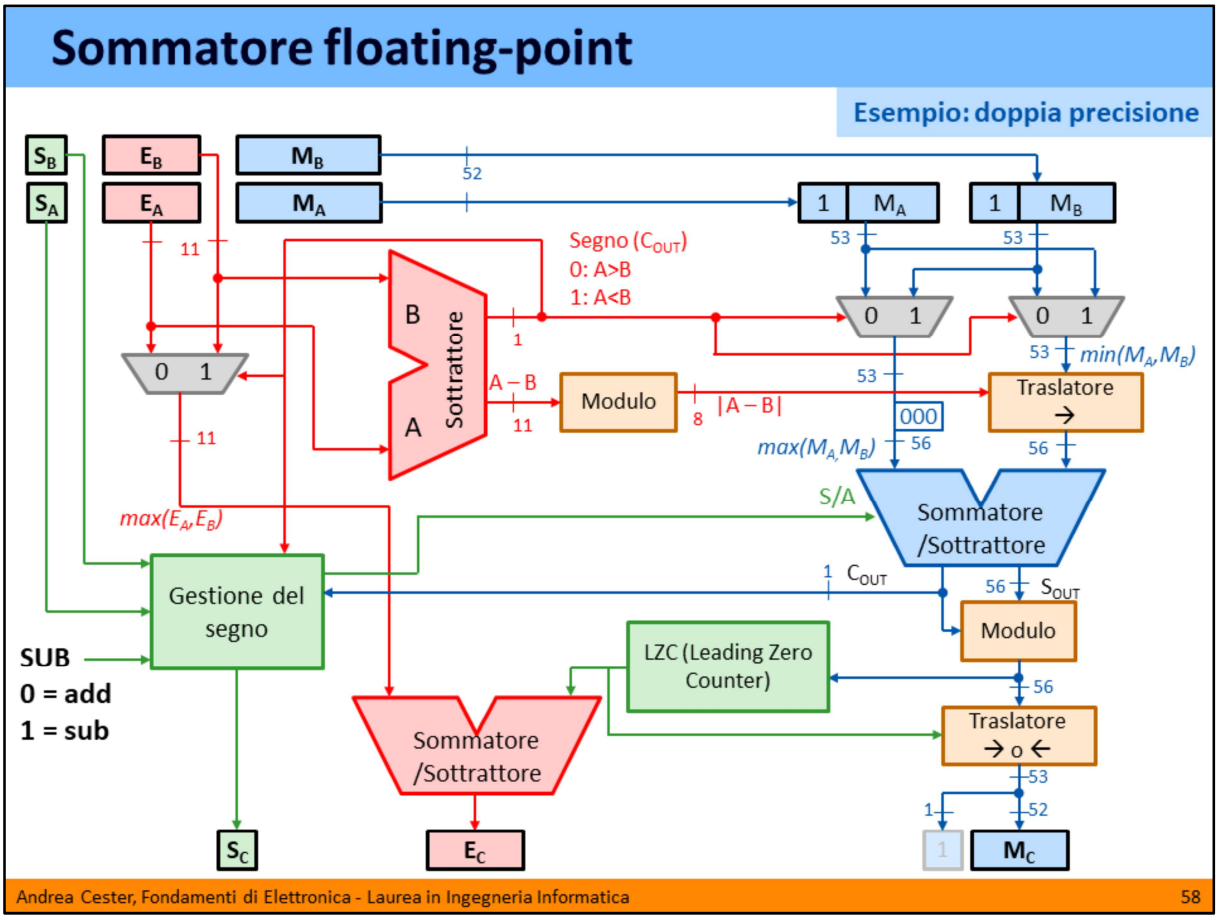
L'algoritmo si somma che abbiamo discusso è ciò che deve fare un sommatore in virgola mobile. L'esempio è stato fatto per praticità con due numeri in singola precisione, ma lo stesso procedimento si usa per la doppia precisione.

Nelle unità floating-point (FPU, floating-point unit) dei microprocessori, l'algoritmo discusso, deve essere implementato a livello hardware. Le operazioni elementari svolte sono elencate di seguito:

1. *Allineamento degli esponenti:* Confronto tra interi (sottrazione, comparazione, modulo), selezione dell'esponente (multiplexer)
2. *Ricostruzione e allineamento delle mantisse:* Selezione di OP1 e OP2 (multiplexer), allineamento (traslatore)
3. *Somma algebrica delle mantisse:* Sommatore/sottrattore
4. *Calcolo del modulo e del segno del risultato:* Modulo
5. *Normalizzazione del risultato:* Conteggio degli zeri e traslazione finale del risultato.

Abbiamo già visto esempi di l'implementazione di questi blocchi.

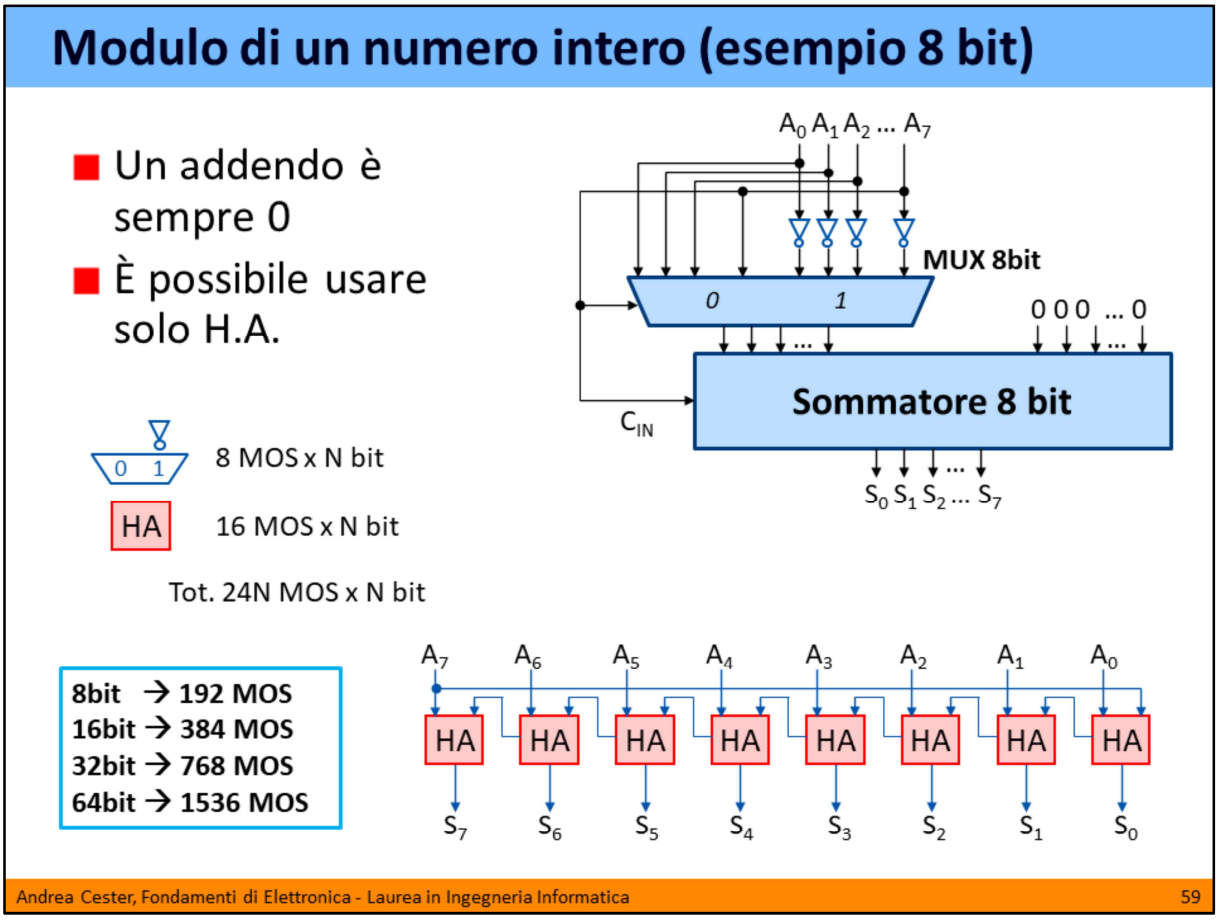
Nel seguito analizzeremo lo schema a blocchi di un sommatore in virgola mobile e, a titolo di esempio, proveremo ad implementare i blocchi utilizzando gli esempi implementativi visti finora, dando una stima del numero di MOS.




*Struttura semplificata di un sommatore floating-point*

Gli ingressi del sommatore sono i tre campi dei numeri A e B che rappresentano segno, esponente e mantissa ($S_A$, $S_B$, $E_A$, $E_B$, $M_A$, $M_B$) e un segnale SUB che indica se desideriamo fare la somma A+B (SUB = 0) o la sottrazione A–B (SUB= 1).

- Il sottrattore a 11 bit in alto a sinistra esegue la differenza A-B e fornisce in uscita 11 bit di somma/sottrazione e il riporto $C_{OUT}$. Il riporto è usato per controllare un vettore di 11 MUX che selezionano il maggiore di $E_A$ e $E_B$. La differenza A-B entra in un blocco che esegue il modulo. |A-B| sarà usato per allineare le mantisse nel passaggio successivo.
- Sul lato destro vediamo tutte le operazioni da svolgere sulle mantisse. Iniziando dall'alto, si ricostruiscono le mantisse aggiungendo un 1 come MSB (estendendo i bit da 52 a 53). Successivamente una coppia di vettori di MUX (24 bit ciascuno) selezionano OP1 (operando maggiore) e OP2 (operando minore) usando come ingresso di selezione il riporto di uscita del sottrattore.
- OP2 (il minore dei due) deve essere traslato verso l'LSB di un numero di bit pari a |A-B|. Gli 11 bit di uscita del sottrattore costituiscono i segnali di controllo dei MUX che compongono il traslatore. Prima di eseguire questa operazione i bit di OP1 e OP2 sono estesi a 56 aggiungendo 3 zeri dalla parte dell'LSB. Ciò incrementa la precisione dell'operazione di somma/differenza poiché permette di perdere meno bit di OP2 durante la traslazione.
- Il sommatore/sottrattore a 56 bit esegue quindi la somma o la differenza di A e B. La scelta tra somma e differenza è fatta da un apposito circuito di controllo che, noti i segni $S_A$ e $S_B$ e noto il segnale SUB esegue l'operazione richiesta (A + B, A – B, -A + B o -A – B). Lo stesso blocco di controllo genera il segno $S_C$, nota l'operazione da svolgere e la relazione d'ordine tra A e B.
- Le uscite del sommatore S e $C_{OUT}$ entrano nel blocco che esegue il modulo della mantissa.
- Infine, il risultato del modulo è l'ingresso del blocco LZC (Leeding-Zero Counter) che conta il numero di zeri iniziali. Tale valore è usato per traslare la mantissa da una parte (generando $M_C$), e sommarsi all'esponente dall'altra (generando $E_C$).




*Modulo di un numero intero*

L'operazione modulo può essere svolta negando il segnale e sommando 1, se negativo. Il circuito mostrato in alto in questo esempio, usa un vettore di multiplexer per selezionare il numero a N bit (N = 8 in figura) in versione diretta o negata. La selezione è fatta in base al bit di segno (MSB di A). L'uscita dei MUX è usata come addendo del successivo sommatore. Il secondo addendo del sommatore è posto a 0, mentre il riporto $C_{IN}$ è pilotato dal bit di segno di A.

Se $A \geq 0$, allora $A_{N-1} = 0$. Il MUX seleziona A in forma diretta, il sommatore ha $C_{IN} = 0$ e la sua uscita vale $S = A + 0 = A$.

Se $A < 0$, allora $A_{N-1} = 1$. Il MUX seleziona A in forma negata , il sommatore ha $C_{IN} = 1$ e la sua uscita vale:

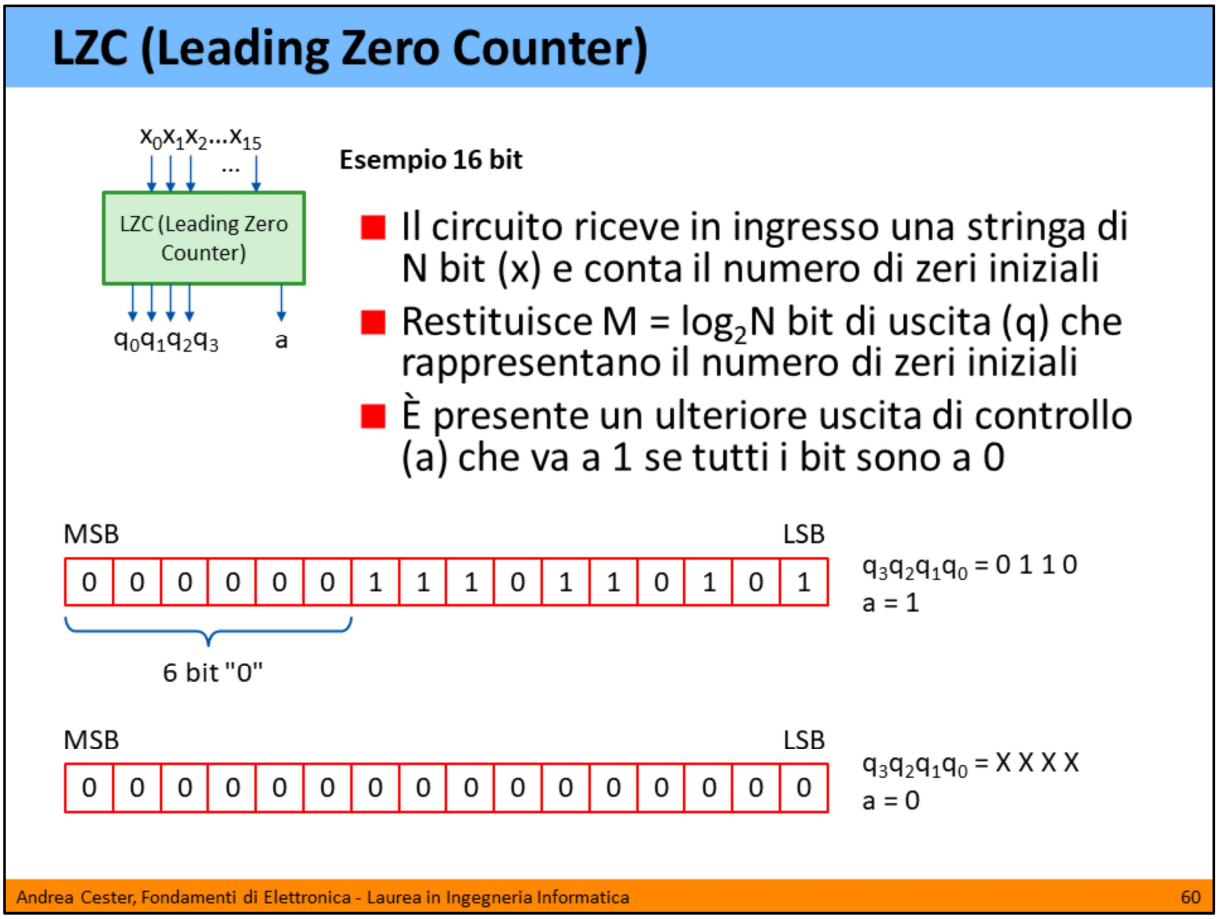
$$S = \overline{A} + 0 + 1$$

che rappresenta il complemento a 2 di A.

Lo schema in alto rappresenta solo la struttura a blocchi. Se per il sommatore adottiamo l'architettura ripple carry, ciascun FA ha uno degli ingressi a 0 e svolge la funzione:

$$S_n = A_n \oplus 0 \oplus C_{n-1} = A_n \oplus C_{n-1} \quad e \quad C_n = A_n \cdot 0 + A_n \cdot C_{n-1} + 0 \cdot C_{n-1} = A_n \cdot C_{n-1}$$

che corrisponde alla somma di soli due bit. Se il sommatore è dedicato solo alla funzione modulo, è possibile adottare HA invece che FA, semplificando il circuito.
Implementando il MUX come nel sommatore/sottrattore e gli HA a pass-T, servono 8 MOS per il MUX e 16 per l'HA, per un totale di 24 MOS per bit (trascuriamo per semplicità i due MOS per negare il segnale di controllo dei MUX).




*LZC (Leading Zero Counter)*

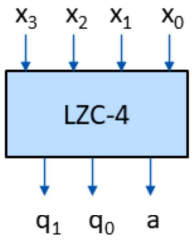
Il Leading Zero Counter (LZC) ha il compito di contare il numero di bit 0 iniziali dalla parte del MSB. Il circuito riceve in ingresso una stringa di N bit e deve fornire in uscita la codifica binaria del numero di 0 iniziali della stringa. In generale se N è il numero di bit della stringa di ingresso il circuito ha $\log_2$N+1 uscite.

Se, ad esempio forniamo in ingresso 16 bit, l'uscita è composta da 5 segnali (a, $q_0$, $q_1$, $q_2$ e $q_3$):

- Il segnale *a* vale 1 se esiste un bit a 1 nella stringa. Vale 0 se i biot della stringa sono tutti nulli.
- Gli altri quattro segnali $q_0$, $q_1$, $q_2$ e $q_3$ forniscono la codifica binaria del numero di zeri iniziali. Il valore delle uscite q ha significato solo se a = 1. In caso contrario il valore di q può essere indeterminato.

## Leading Zero Counter: tabella di verità a 4 bit

| $x_3$ | $x_2$ | $x_1$ | $x_0$ | $q_1$ | $q_0$ | a |
|---|---|---|---|---|---|---|
| 1 | X | X | X | 0 | 0 | 1 |
| 0 | 1 | X | X | 0 | 1 | 1 |
| 0 | 0 | 1 | X | 1 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | X | X | 0 |




Mappa di Karnaugh per $q_1$

| $x_1x_0$ / $x_3x_2$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | 1 | 1 | 1 |
| 01 | | | | |
| 11 | | | | |
| 10 | | | | |

Mappa di Karnaugh per $q_0$

| $x_1x_0$ / $x_3x_2$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | 1 | | |
| 01 | 1 | 1 | 1 | 1 |
| 11 | | | | |
| 10 | | | | |

Mappa di Karnaugh per a

| $x_1x_0$ / $x_3x_2$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 1 | 1 |
| 01 | 1 | 1 | 1 | 1 |
| 11 | 1 | 1 | 1 | 1 |
| 10 | 1 | 1 | 1 | 1 |

$$q_1 = \overline{x_3} \cdot \overline{x_2} = \overline{x_3 + x_2}$$

$$q_0 = \overline{x_3} \cdot x_2 + \overline{x_3} \cdot \overline{x_1}$$

$$a = x_3 + x_2 + x_1 + x_0$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 61

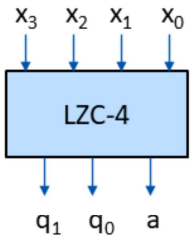
*LZC a 4 bit*

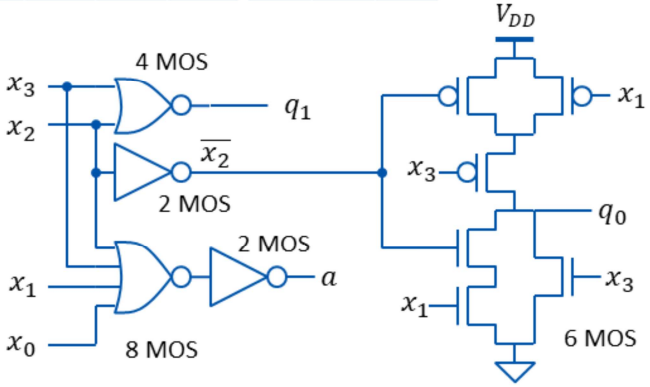
Come esempio vediamo l'implementazione di un LZC a 4 bit. Definiamo $x_3$, $x_2$, $x_1$ e $x_0$ i segnali di ingresso e $a$, $q_1$ e $q_0$ quelli di uscita.
In figura è mostrata la tabella di verità. La X presente in alcune cella indica una condizione di "*don't-care*" cioè di indifferenza. Se X compare tra i valori di ingresso, significa che l'uscita specificata non dipende dal valore di quello specifico ingresso. Se X compare in uno dei valori di uscita, significa che la corrispondente uscita può avere un valore arbitrario per quella corrispondente configurazione degli ingressi. Queste condizioni possono essere usate per semplificare la struttura delle porte logiche e ridurre il numero di MOS.
Le tre figure in basso mostrano le mappe di Karnaugh per le tre uscite. Le caselle vuote sottintendono uno 0. L'implementazione delle funzioni può essere fatta trattando le condizioni di indifferenza come 1 o come 0 in modo arbitrario. Volendo semplificare e ridurre il numero di porte logiche, il criterio da adottare è di includere le caselle X nei raggruppamenti se ciò fa aumentare il numero di celle raggruppate.
In base a questo criterio, ciascuna uscita può essere scritta come:

$$q_1 = \overline{x_2 + x_3}$$
$$q_0 = x_2 \cdot \overline{x_3} + \overline{x_1} \cdot \overline{x_3}$$
$$a = x_3 + x_2 + x_1 + x_0$$

## Leading Zero Counter: tabella di verità a 4 bit

| $x_3$ | $x_2$ | $x_1$ | $x_0$ | $q_1$ | $q_0$ | a |
|---|---|---|---|---|---|---|
| 1 | X | X | X | 0 | 0 | 1 |
| 0 | 1 | X | X | 0 | 1 | 1 |
| 0 | 0 | 1 | X | 1 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | X | X | 0 |




$$q_0 = \overline{x_3} \cdot x_2 + \overline{x_3} \cdot \overline{x_1} = \\ = \overline{x_3} \cdot (x_2 + \overline{x_1}) = \\ = \overline{x_3 + \overline{x_2} \cdot x_1}$$

$$q_1 = \overline{x_3 + x_2}$$

$$a = x_3 + x_2 + x_1 + x_0$$

**Totale: 22 MOS**



Un esempio di implementazione in logica statica complementare è mostrato in figura con quattro porte logiche elementari NOT, NAND e NOR e una porta logica arbitraria. Per questa realizzazione l'uscita $q_0$ è stata riscritta:

$$q_0 = x_2 \cdot \overline{x_3} + \overline{x_1} \cdot \overline{x_3} = \overline{x_3} \cdot (x_2 + \overline{x_1}) = \overline{x_3 + \overline{x_2 + \overline{x_1}}} = \overline{x_3 + \overline{x_2} \cdot x_1}$$

Il numero di MOS per ciascuna porta è indicato in figura accanto a ciascuna porta logica. In totale abbiamo 22 MOS.

*LCZ a 2 bit*

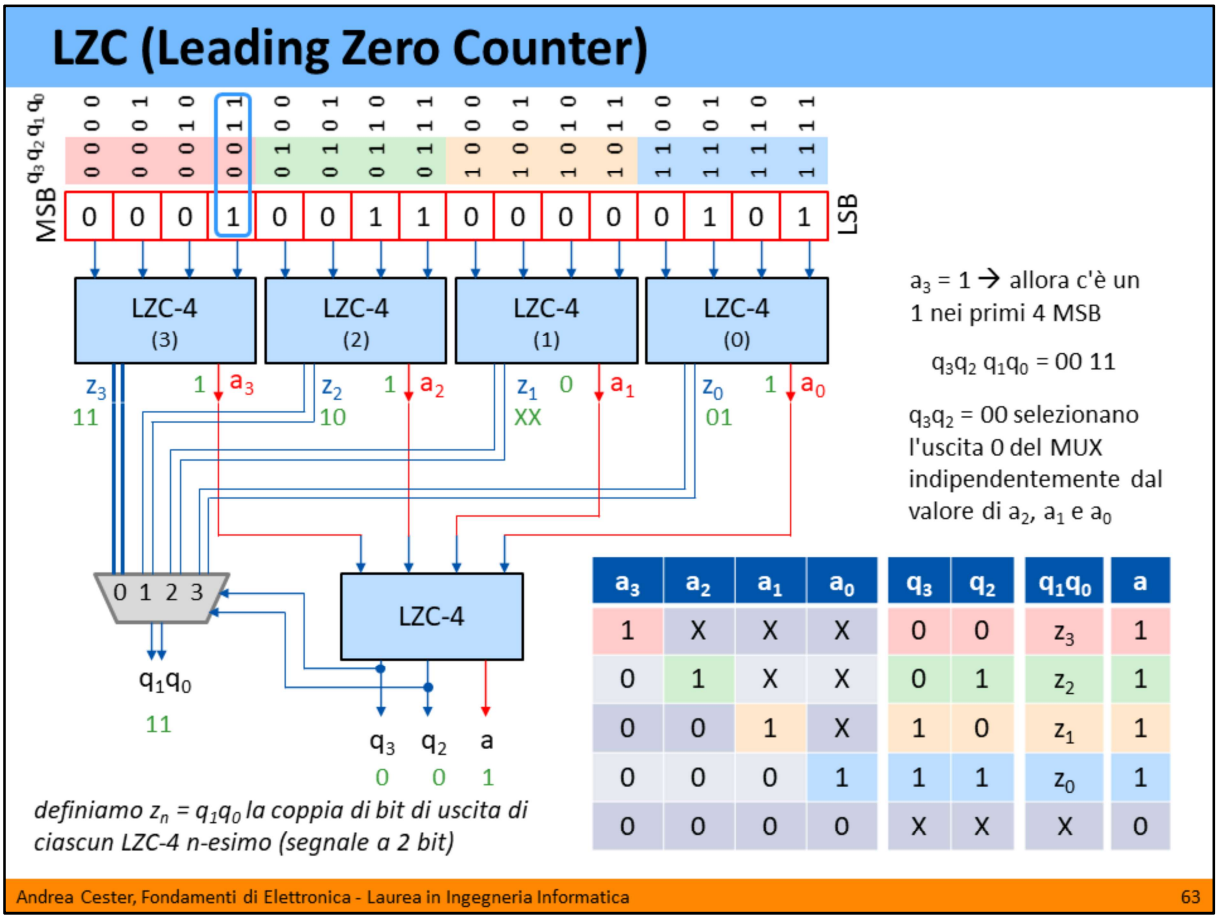
La funzione logica ha due ingressi ($x_0$ e $x_1$) e due uscite ($q_0$ e a). Esistono 4 casi per $x_0$ e $x_1$:

$$x_1 = 0 \; e \; x_0 = 0 \quad \rightarrow \quad a = 0, q_0 = X$$
$$x_1 = 0 \; e \; x_0 = 1 \quad \rightarrow \quad a = 1, q_0 = 1$$
$$x_1 = 1 \; e \; x_0 = 0 \quad \rightarrow \quad a = 1, q_0 = 0$$
$$x_1 = 1 \; e \; x_0 = 1 \quad \rightarrow \quad a = 1, q_0 = 0$$

Da ciò ricaviamo:

$$q_0 = \overline{x_1}$$
$$a = x_1 + x_0 = \overline{\overline{x_1} \cdot \overline{x_0}}$$

che possono essere implementate con 2 invertitori (4 MOS) e una porta NAND a 2 ingressi (4 MOS) per un totale di 8MOS




| $a_3$ | $a_2$ | $a_1$ | $a_0$ | $q_3$ | $q_2$ | $q_1q_0$ | a |
|---|---|---|---|---|---|---|---|
| 1 | X | X | X | 0 | 0 | $z_3$ | 1 |
| 0 | 1 | X | X | 0 | 1 | $z_2$ | 1 |
| 0 | 0 | 1 | X | 1 | 0 | $z_1$ | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 | $z_0$ | 1 |
| 0 | 0 | 0 | 0 | X | X | X | 0 |

*LZC a N bit*

La realizzazione di un LCZ con un numero N di bit segue la stessa filosofia. Il problema che sorge è il ritardo delle porte logiche. Basti considerare che l'uscita *a* è ottenuta dalla OR di tutti i bit. Usando ad esempio 16 bit come in figura, il ritardo di una porta logica a singolo stadio a N ingressi diventa intollerabile.

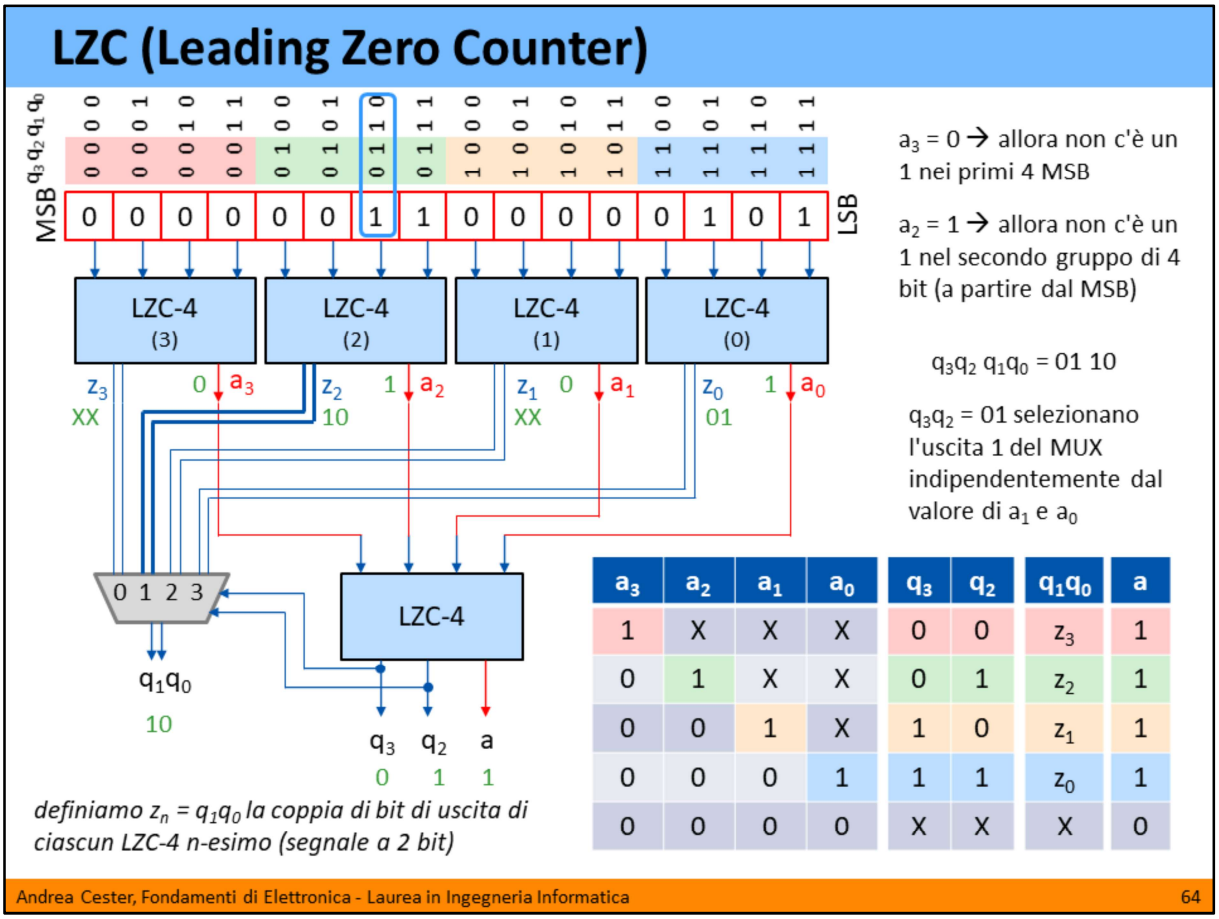
Per ovviare al problema è possibile usare una configurazione ad albero. La figura mostra l'implementazione di un LZC a 16 bit raggruppando i bit in quattro gruppi da 4.

Le stringe di bit in alto identificano il valore che devono avere le uscite $q_0$, $q_1$, $q_2$ e $q_3$ dell'LZC complessivo (a 16 bit) se il primo 1 della stringa si trova nella corrispondente posizione. Ad esempio se la stringa di ingresso è quella in figura, in cui il primo 1 appare come quarto bit (a partire dall'MSB), abbiamo 3 zeri e deve risultare $q_3q_2q_1q_0 = 0011$.

Il primo stadio è formato da 4 LZC a quattro bit che calcolano il numero di zeri iniziali di ciascun gruppo. Questi sono numerati da 0 a 3. Per semplicità e compattezza indichiamo con $z_0$, $z_1$, $z_2$ e $z_3$, il vettore $q_1q_0$ di ciascun LCZ. $z_n$ rappresenta, dunque, un numero a 2 bit. Le corrispondenti uscite $a_0$, $a_1$, $a_2$, $a_3$ indicano l'eventuale ricorrenza di un bit 1 all'interno del gruppo corrispondente.

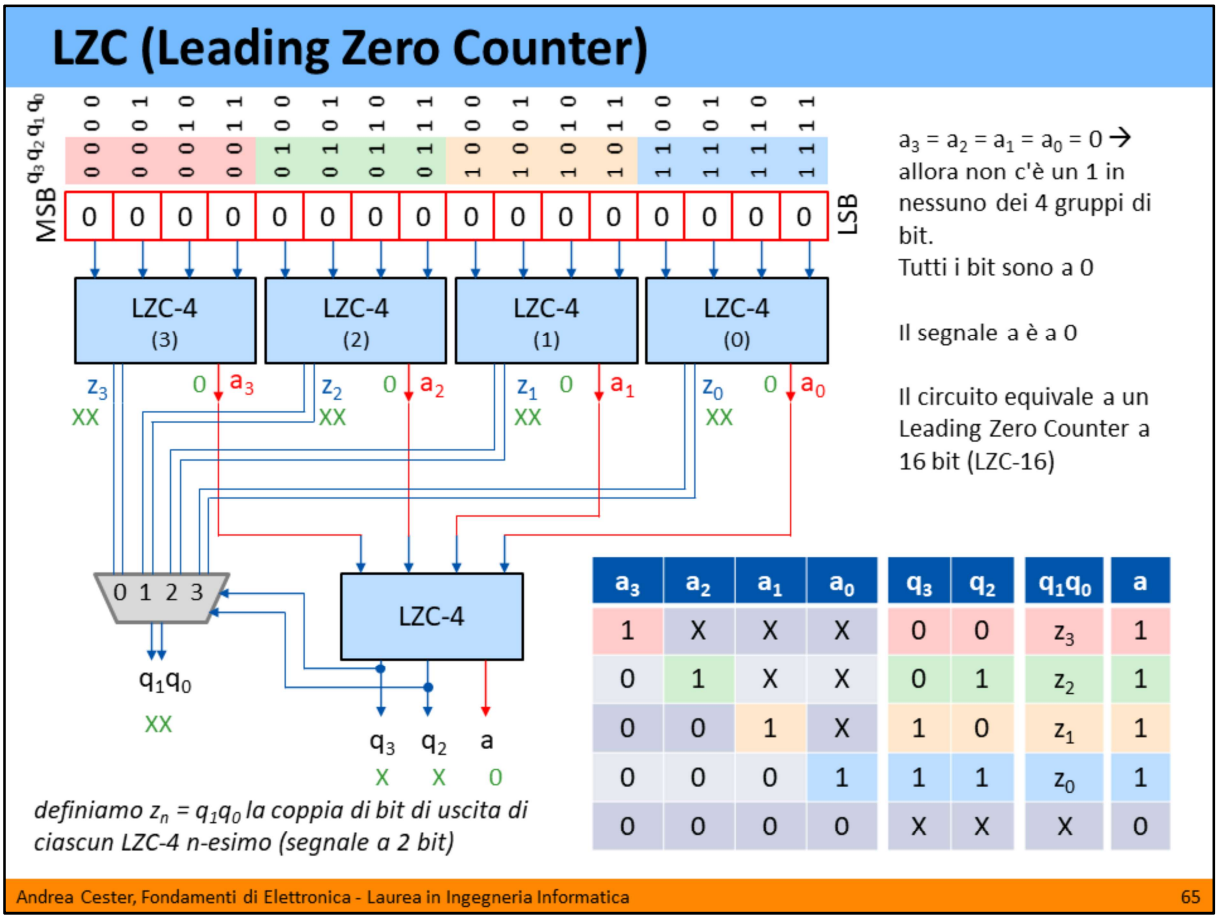
Ad esempio se $a_3 = 1$, indica che nei primi quattro bit c'è un 1 il numero $q_3q_2q_1q_0$ è strettamente minore di 4 e significa che $q_3q_2 = 00$. Se invece $a_3 = 0$ significa che non ci sono zeri nei primi 4 MSB. In altre parole il primo 1 che si trova nella stringa $a_3a_2a_1a_0$ identifica il gruppo di quattro bit in cui si trova il primo 1.

Il secondo stadio è formato da un MUX a 4 ingressi e un ulteriore LZC a 4 ingressi che ha come ingressi i bit $a_n$ dei 4 LCZ del primo stadio. Quest'ultimo LZC conta quanti zeri ci sono nella stringa $a_3a_2a_1a_0$. Con la stringa di bit mostrata in figura $a_3 = 1$ le uscite dell'ultimo LZC sono $q_3q_2 = 00$ e a = 1. I bit $q_1q_0$ coincidono in questo caso con $z_3$, cioè la posizione del primo 1 all'interno dei primi 4 bit. Le uscite dei MUX sono selezionate dai bit $q_3q_2$, che in questo caso selezionano l'ingresso 0 ($z_3$).




| $a_3$ | $a_2$ | $a_1$ | $a_0$ | $q_3$ | $q_2$ | $q_1q_0$ | a |
|---|---|---|---|---|---|---|---|
| 1 | X | X | X | 0 | 0 | $z_3$ | 1 |
| 0 | 1 | X | X | 0 | 1 | $z_2$ | 1 |
| 0 | 0 | 1 | X | 1 | 0 | $z_1$ | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 | $z_0$ | 1 |
| 0 | 0 | 0 | 0 | X | X | X | 0 |

In questo secondo esempio vediamo che nella stringa di ingresso il primo 1 compare nel secondo blocco. In questo caso il vettore di bit $a_3a_2a_1a_0$ vale 0101 e indica che il primo 1 si trova nel secondo blocco. Il blocco LZC del secondo stadio fornisce in uscita $a$ = 1 e $q_3q_2$ = 01. Ciò porta alla selezione dell'ingresso 1 del MUX che collega all'uscita $q_1q_0$ il dato di ingresso $z_2$, che in questo caso vale 10.
Notiamo che in questo caso, essendo a3 = 0, il valore si z3 è irrilevante poiché non è collegato all'uscita del MUX.




| $a_3$ | $a_2$ | $a_1$ | $a_0$ | $q_3$ | $q_2$ | $q_1q_0$ | a |
|---|---|---|---|---|---|---|---|
| 1 | X | X | X | 0 | 0 | $z_3$ | 1 |
| 0 | 1 | X | X | 0 | 1 | $z_2$ | 1 |
| 0 | 0 | 1 | X | 1 | 0 | $z_1$ | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 | $z_0$ | 1 |
| 0 | 0 | 0 | 0 | X | X | X | 0 |

In quest'ultimo esempio vediamo cosa succede se la stringa di 16 bit in ingresso è tutta nulla. In questo caso ciascun LZC del primo stadio porrà $a_n$ = 0. Il secondo stadio riceve in ingresso una stringa nulla, ponendo a sua volta $a$ = 0.In questo caso il valore delle uscite $q$ è irrilevante

Con la stessa filosofia è possibile realizzare un LZC a 8 bit usando nel primo stadio due LZC a 4 bit e nel secondo stadio un LZC da 2 bit.

Se volessimo realizzare invece un LZC da 64 bit è possibile raggruppare i bit in 16 gruppi da 4 bit (servono quindi 16 LZC). Ciascuno di questi produce in uscita un segnale $a_n$ che indica se è presente un 1 all'interno del gruppo corrispondente. I 16 bit an possono essere poi gestiti da un LZC a 16 bit (implementato ad albero come nel caso appena visto). Serviranno quindi 3 stadi di LZC.

Come esercizio, provare a realizzare lo schema a blocchi di un LZC a 8, 32 e 64 bit usando LZC da 2 o da 4 bit.

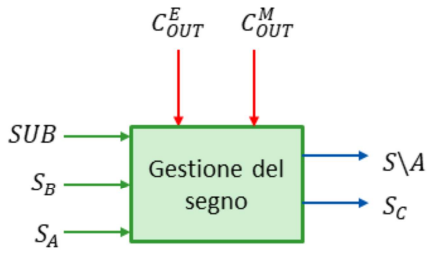
# Stima del numero di MOS

| Sommatore | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Ripple carry | 192 | 384 | 768 | 1536 | 24N |
| Sottrattore (MUX) | 66 | 132 | 258 | 514 | 8N+2 |
| (add/sub/comp) | 28+2 | 44+2 | 84+2 | 128+2 | 2.5N+2 (circa) |
| Tot. | **288** | **562** | **1112** | **2180** | **34.5N+4 (circa)** |

| Circuito | 8 bit | 16 bit | 32bit | 64bit | Nbit |
|---|---|---|---|---|---|
| Traslatore | 134 | 328 | 778 | 1804 | 4N(1+logN)+2logN |
| Modulo (int) | 192 | 384 | 768 | 1536 | 24N |
| LZC | 72 | 154 | 356 | 726 | ~12(N-2) |

## Gestione del segno

- Deve generare i segnali di controllo e di uscita ($S_C$) in base a:
  - L'operazione richiesta (SUB)
  - Il segno degli operandi ($S_A$ e $S_B$)
  - L'ordine degli operandi (ricavato dal confronto degli esponenti, $C_{OUT}^{E}$)
  - Il riporto di uscita della somma/differenza delle mantisse (ricavato dal riporto di uscita $C_{OUT}^{M}$)
- È una funzione logica a 5 ingressi e 2 uscite
- Occupazione di area (qualche decina di MOS)




Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 67

*Gestione del segno*

La gestione del segno serve a selezionare da un lato il tipo di operazione da fare (somma o sottrazione) e il segno del risultato finale. Questo blocco di controllo deve generare i segnali:

- S/A che controlla il sommatore/sottrazione delle mantisse (S/A = 0 per sommare OP1+OP2, S/A = 1 per sottrare OP1 – OP2).
- Il segno di uscita $S_C$.

Le informazioni necessarie sono:

- I segni $S_A$ e $S_B$ degli operandi
- La relazione d'ordine tra A e B (riporto di uscita $C_{OUT}^{E}$ della sottrazione dei riporti $E_A$ - $E_B$) per poter associare il segno giusto a OP1 e OP2.
- L'operazione che si desidera svolgere (SUB)

Solo come esempio, riportiamo di seguito un possibile algoritmo di calcolo dei segnali S/A e $S_C$.

*1) Identificazione del tipo di operazione e trasformazione in somma.*

Se SUB = 0 vogliamo eseguire l'operazione C = A + B, se SUB = 1 svolgiamo l'operazione C = A – B. Nel secondo caso possiamo riscrivere la differenza come:

$$C = A + (-B) = A + B' \quad con \quad B' = -B$$

Ciò equivale a complementare il segno $S_B$ se SUB = 1

$$S'_B = \overline{SUB} \cdot S_B + SUB \cdot \overline{S_B} = SUB \oplus B$$

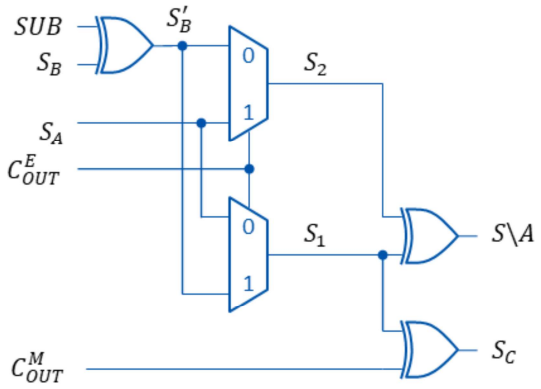
*2) Identificazione degli operandi OP1 (maggiore) e OP2 (minore).*

A tale scopo è necessario conoscere la relazione d'ordina (cioè il riporto $C_{OUT}^{E}$. Detti $S_1$ e $S_2$ i segni di OP1 e OP2:

$$S_1 = \overline{C_{OU}^{E}} \cdot S_A + C_{OUT}^{E} \cdot S_B \quad e \quad S_2 = \overline{C_{OUT}^{E}} \cdot S_B + C_{OUT}^{E} \cdot S_A$$

In questo modo dobbiamo svolgere l'operazione OP1+OP2. I segno di OP1 e OP2 è identificato dai segni $S_1$ e $S_2$.

## Gestione del segno (esempio di implementazione)




| | |
|---|---|
| 3 XOR → | 3x10 = 30 MOS |
| 2 MUX → | 2x12 = 24 MOS |
| TOT. | 54 MOS |

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 68

*3) Calcolo di S/A e $S_C$.*

Un modo possibile è generare i segnali in modo tale da ricondurre la operazioni sulle mantisse a un operazione del tipo $X \pm Y$ in cui X e Y siano due numeri postivi, dato che le mantisse rappresentano i moduli dei due numeri da sommare. Ci sono quattro possibili casi:

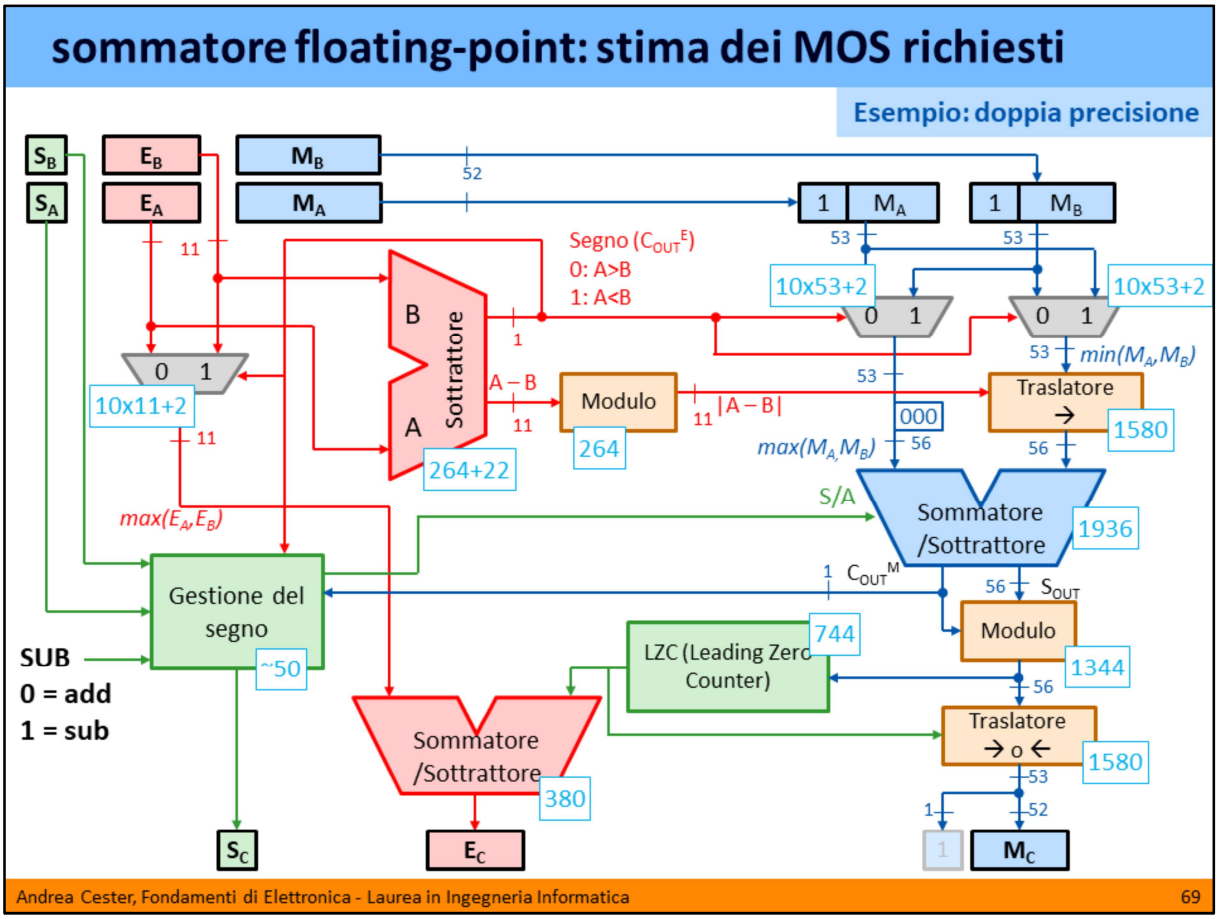
- $S_1$ = 0, $S_2$ = 0 (OP1>0, OP2>0): C = $M_1$ + $M_2$ è la somma delle mantisse (positive), quindi: S/A = 0, $S_C$ = $C_{OUT}^M$ = 0
- $S_1$ = 0, $S_2$ = 1 (OP1>0, OP2<0): C = $M_1$ – $M_2$ è la differenza delle mantisse, quindi: S/A = 1, $S_C$ = $C_{OUT}^M$
- $S_1$ = 1, $S_2$ = 0 (OP1<0, OP2>0): possiamo riscrivere la somma come: -(-OP1 – OP2). -OP1 è positivo e OP2 è positivo. C = $M_1$ – $M_2$ è la differenza delle mantisse, quindi: S/A = 1. Avendo raccolto il segno meno, $S_C$ deve essere invertito: $S_C = \overline{C_{OUT}^M}$ .
- $S_1$ = 1, $S_2$ = 1 (OP1<0, OP2<0): scriviamo anche in questo caso: -(-OP1 + (-OP2)). -OP1 e -OP2 sono positivi e ci riconduciamo alla somma delle mantisse: C = $M_1$ + $M_2$. Abbiamo, dunque, S/A = 0, ma dobbiamo invertire il segno del riporto: $S_C = \overline{C_{OUT}^M}$ = 1.

Mettendo insieme i risultati, abbiamo:

$$S\backslash A = \overline{S_1} \cdot S_2 + S_1 \cdot \overline{S_2} = S_1 \oplus S_2$$

$$S_C = \overline{S_1} \cdot \overline{S_2} \cdot C_{OUT}^M + \overline{S_1} \cdot S_2 \cdot C_{OUT}^M + S_1 \cdot \overline{S_2} \cdot \overline{C_{OUT}^M} + S_1 \cdot S_2 \cdot \overline{C_{OUT}^M} = \overline{S_1} \cdot C_{OUT}^M + S_1 \cdot \overline{C_{OUT}^M}$$
$$= S_1 \oplus C_{OUT}^M$$

L'implementazione del blocco logico può essere fatta con MUX e porte XOR impiegando circa una cinquantina di MOS, trascurabile rispetto al resto del sommatore FP.




*Stima del numero di MOS richiesti*

La figura mostra il numero di MOS richiesti per ciascun modulo.
NOTE:

- Le implementazioni usate sono come negli esempi delle precedenti slide e non si riferiscono a numeri usati in progetti reali, che impiegano architetture più complesse. Alcune saranno viste in seguito.
- Il sommatore dei riporti esegue sempre e solo la differenza. Il numero $E_B$ è sempre negato e il suo riporto di ingresso è settato a 1. Sono necessari 24N MOS per il sommatore e 2N MOS per gli invertitori. N = 11 bit per la doppia precisione e 8 per la singola precisione.
- Per LCZ e traslatori con numero di bit che non sia una potenza di 2 il numero di MOS è una stima basata sui calcoli precedenti, arrotondando logN all'intero successivo. Ad esempio per il traslatore a 57 bit si arrotonda log57 a 6.

## sommatore floating-point: stima dei MOS richiesti

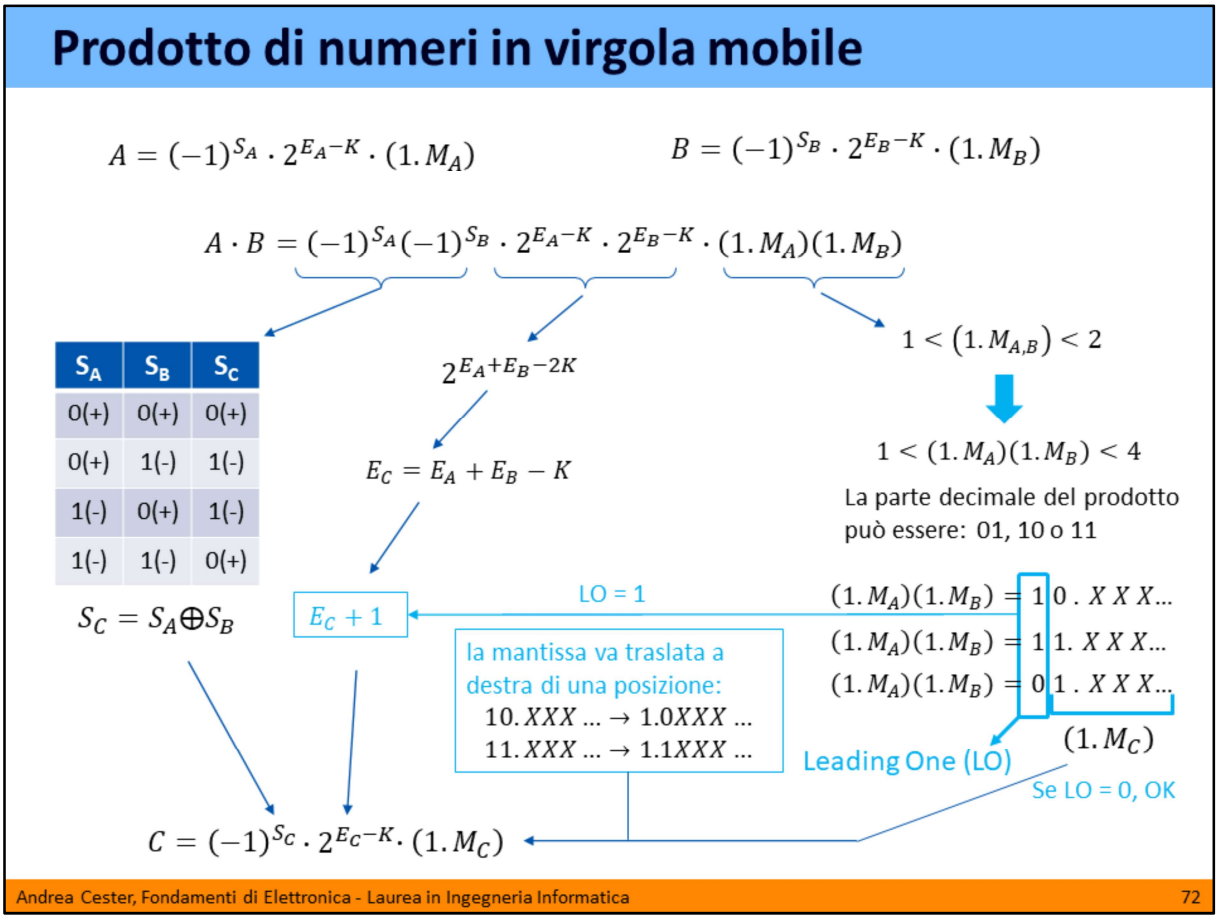
| Singola precisione (32 bit) | |
|---|---|
| MUX: | 1x82 |
| | 2x242 |
| Sottrattore (exp): | 208 |
| Modulo: | 1x192 |
| | 1x648 |
| Traslatore: | 2x658 |
| Somm./Sottr.: | 1x936 |
| | 1x256 |
| LZC: | 356 |
| Segno: | ~50 |
| TOT. | 4528 MOS |
| di cui: | 2240 in sommatori |
| | 1316 in traslatori |
| | 972 in MUX e controllo |

| Doppia precisione (64 bit) | |
|---|---|
| MUX: | 1x112 |
| | 2x532 |
| Sottrattore (exp): | 286 |
| Modulo: | 1x264 |
| | 1x1344 |
| Traslatore: | 2x1580 |
| Somm./Sottr.: | 1x1936 |
| | 1x380 |
| LZC: | 744 |
| Segno: | ~50 |
| TOT. | 9340 MOS |
| di cui: | 4210 in sommatori |
| | 3160 in traslatori |
| | 1970 in MUX e controllo |



Vediamo qui il confronto tra la singola e doppia precisione. Raddoppiando la precisione, raddoppia anche il numero di MOS.
Notiamo anche che circa metà dei MOS sono impiegati per eseguire le operazioni di somma.

# OPERAZIONI FLOATING POINT: MOLTIPLICAZIONE




| $S_A$ | $S_B$ | $S_C$ |
|---|---|---|
| 0(+) | 0(+) | 0(+) |
| 0(+) | 1(-) | 1(-) |
| 1(-) | 0(+) | 1(-) |
| 1(-) | 1(-) | 0(+) |

**Moltiplicatore in virgola mobile**

*Algoritmo di moltiplicazione in virgola mobile*

Siano dati due numeri in virgola mobile rappresentato secondo lo standard IEEE 754:

$$A = (-1)^{S_A}(1.M_A)2^{E_A-K} \quad e \quad (-1)^{S_B}(1.M_B)2^{E_B-K}$$

Il prodotto dei numeri è:

$$C = (-1)^{S_C}(1.M_C)2^{E_C-K}$$

Ricordiamo che l'esponente è memorizzato sommato a un offset K (127 in singola precisione e 1023 in doppia precisione).

*Bit di segno*

Il bit del segno è SC = SA + SB, cioè la somma dei bit A e B che è data dalla XOR di A e B (come mostra anche la tabella di verità):

$$S_C = S_A \oplus S_B$$

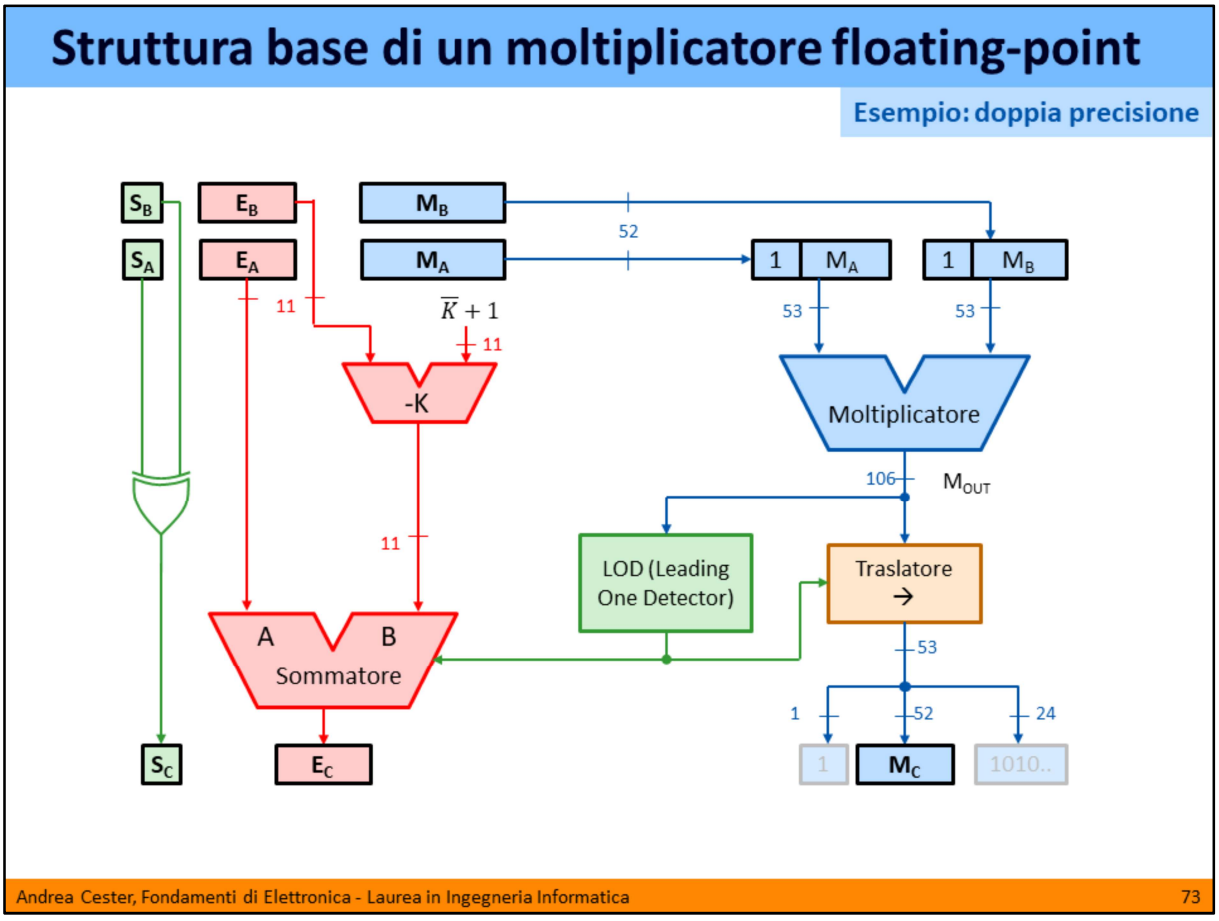
*Mantissa*

La mantissa si ricava dal prodotto delle mantisse e richiede per prima cosa l'estensione della mantissa aggiungendo il bit 1 dalla parte del MSB (come per la somma). In base allo standard IEEE 754 il valore di mantissa (1.M) è sempre maggiore o uguale a 1, ma strettamente minore di 2. Di conseguenza la mantissa di C (prodotto di A e B) soddisfa la seguente relazione:

$$1 \leq (1.M_C) < 4$$

Di conseguenza i bit interi (a sinistra della virgola) daranno due e potranno avere i seguenti valori: 01, 10, 11. Se il primo bit è a 0, allora la mantissa ($1.M_C$) soddisfa già lo standard IEEE 754. Negli altri due casi è necessario spostare la virgola a sinistra, traslando a destra i bit del risultato del prodotto. La moltiplicazione delle mantisse si svolge nei seguenti passi:

1. Ricostruzione e moltiplicazione delle mantisse ($1.M_A$) e ($1.M_B$).
2. Normalizzazione. È necessario leggere il primo MSB. Se è 1 il risultato va normalizzato traslandolo a destra (divisione per 2). Ciò richiede di incrementare di uno l'esponente.




*Esponente*

L'esponente di C è dato dalla somma degli esponenti. Tuttavia ricordiamo che lo standard IEEE 754 richiede che l'esponente sia sommato all'offset K (127 in singola precisione e 1023 in doppia precisione). Sommare direttamente $E_A$ e $E_B$ significa includere l'offset due volte. Pertanto il campo $E_C$ dell'esponente di C è necessario sommare i valori contenuti in $E_A$ e in $E_B$ e sottrarre K:

$$\begin{matrix} esponente\ A: E_A - K \\ esponente\ B: E_B - K \end{matrix} \rightarrow esponente\ C: E_C - K = E_A - K + E_B - K \rightarrow E_C = E_A - E_B - K$$

Infine è necessario sommare 1 all'esponente se la mantissa è stata traslata.

*Struttura del moltiplicatore*

La figura mostra lo schema base di un moltiplicatore in virgola mobile:

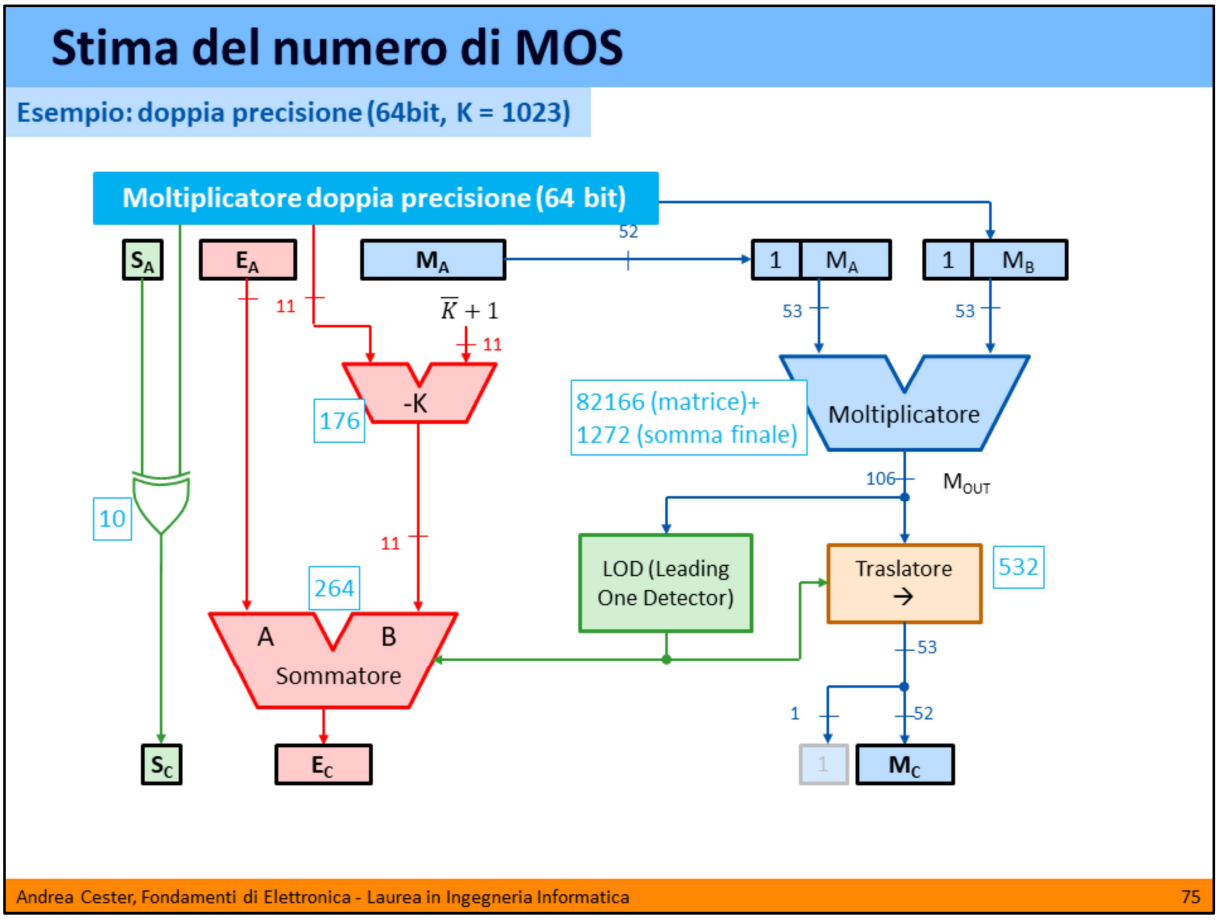
- Il bit di segno è calcolato con una semplice porta XOR.
- Gli esponenti sono sommati, dopo che a uno di essi ($E_B$ in questo esempio) è stato sottratto il modulo.
- Le mantisse sono estese da 52 bit a 53 bit e moltiplicate mediante il moltiplicatore a 53 bit. Il risultato finale è a 106 bit, ma dovrà essere troncato a 53.
- Il bit più significativo della moltiplicazione (che corrispondo al riporto di uscita dell'ultimo stadio di somma finale del moltiplicatore viene testato da un circuito (LDO, Leading One Detector). In questo caso non è necessario contare i bit a 1, ma è sufficiente verificare se il primo bit è o no a 1. Se questo bit è a 1 allora è necessario traslare a destra di un bit il risultato della somma.
- Il traslatore è controllato da questo bit. Se uguale a 1 trasla di un bit, altrimenti non vi è traslazione.
- Il bit più significativo del prodotto delle mantisse controlla anche il riporto di ingresso del sommatore dei riporti, per sommare 1 qualora la mantissa di C venga divisa per 2.

# Stima del numero di MOS

| Sommatore | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Ripple carry | 192 | 384 | 768 | 1536 | 24N |
| Sottrattore (MUX) | 66 | 132 | 258 | 514 | 8N+2 |
| (add/sub/comp) | 28+2 | 44+2 | 84+2 | 128+2 | 2.5N+2 (circa) |
| Tot. | **288** | **562** | **1112** | **2180** | **34.5N+4 (circa)** |

| Circuito | 8 bit | 16 bit | 32bit | 64bit | Nbit |
|---|---|---|---|---|---|
| Traslatore | 134 | 328 | 778 | 1804 | 4N(1+logN)+2logN |
| Modulo (int) | 192 | 384 | 768 | 1536 | 24N |
| LZC | 72 | 154 | 356 | 726 | ~12(N-2) |

| Moltiplicatore (int) | 8 bit | 16 bit | 32bit | 64bit | N bit |
|---|---|---|---|---|---|
| Matrice di AND | 384 | 1536 | 6144 | 24576 | $6N^2$ |
| Somme parziali (HA) | 224 | 480 | 992 | 2016 | $32(N-1)$ |
| Somme parziali (FA) | 1008 | 5040 | 22320 | 93744 | $24(N-1)(N-2)$ |
| Sub.Tot. | 1616 | 7056 | 29456 | 120336 | $30N^2 - 40N + 16$ |
| Tot. (ripple carry finale) | **1808** | **7440** | **30224** | **121872** | $30N^2 - 16N + 16$ |




In figura vediamo una stima dei MOS richiesti per realizzare il moltiplicatore in doppia precisione usando gli schemi circuitali introdotti negli esempi precedenti.

# sommatore floating-point: stima dei MOS richiesti

**Moltiplicatore doppia precisione (64 bit)**

| | |
|---|---|
| Sommatori: | 1x176 |
| | 1x264 |
| Segno: | 1x10 |
| Traslatore: | 1x532 |
| Moltiplicatore: | |
| - AND + somme parz.: | 82166 |
| - Sommatore finale: | 1272 |
| TOT. | 83148 MOS |
| di cui 1712 in sommatori | |

| | |
|---|---|
| Som./sotr.Comp. int. 64 bit: | 2180 |
| Sommatore FP 64 bit: | 9340 |
| Moltiplicatore int. 64 bit: | 121872 |
| Moltiplicatore FP 64 bit | 83148 |
| Rototraslatore 64 bit: | 1804 |
| TOT. | 218344 MOS |

**sommatore doppia precisione (64 bit)**

| | |
|---|---|
| MUX: | 1x112 |
| | 2x532 |
| Sottrattore (exp): | 286 |
| Modulo: | 1x264 |
| | 1x1344 |
| Traslatore: | 2x1580 |
| Somm./Sottr.: | 1x1936 |
| | 1x380 |
| LZC: | 744 |
| Segno: | ~50 |
| TOT. | 9340 MOS |
| di cui: | 4210 in sommatori |
| | 3160 in traslatori |
| | 1970 in MUX e controllo |